

King's College London
Department of Mathematics
Submission Cover Sheet for Coursework



The following cover sheet must be completed and submitted with any dissertation, project, coursework essay or report submitted as part of formal assessment for degree in the Mathematics Department.

You are not required to write your name on your work

Candidate number (this is found on your student record account)	AF28307
Module Code and Title	7CCMFM50 - MSc Financial Mathematics Project
Title of Project/Coursework	Hedging & Indifference Pricing of Terminal Claims on the VIX

Declaration

By submitting this assignment I agree to the following statements:

I have read and understand the King's College London Academic Honesty and Integrity Statement that I signed upon entry to this programme of study.

I declare that the content of this submission is my own work.

I understand that plagiarism is a serious examination offence, an allegation of which can result in action being taken under the College's Misconduct regulations.

Your work may be used as an example of good practice for future students to refer to. If chosen, your work will be made available either via KEATS or by paper copy. Your work will remain anonymous; neither the specific mark nor any individual feedback will be shared. Participation is entirely optional and will not affect your mark. If you consent to your submission being used in this way please add an X in the box to the right.	X
--	---

Hedging & Indifference Pricing of Terminal Claims on the VIX

by

Michael K. Osinowo

Department of Mathematics
King's College London
The Strand, London WC2R 2LS
United Kingdom

Email: michael.osinowo@kcl.ac.uk

Tel: +44 (0)7064055189

30 August 2026

REPORT SUBMITTED IN PARTIAL FULFILMENT OF
THE REQUIREMENTS FOR THE DEGREE OF MSc IN
FINANCIAL MATHEMATICS IN THE UNIVERSITY OF
LONDON

Abstract

We study the indifference pricing and hedging of terminal claims on the non-traded VIX index under transaction costs, finite liquidity and budget constraints. Risk preferences are represented by an entropic objective, whose cash invariance reduces each indifference price to a difference of two convex solves. We first model the log-VIX as a discrete-time Ornstein–Uhlenbeck process estimated from historical data, express the trading constraints as linear inequalities, and show that the resulting problem can be solved as an exponential conic program. Indifference buy and sell prices for a digital claim are evaluated across a range of strikes.

We then challenge the law used under Ornstein–Uhlenbeck by implementing and calibrating the Quadratic Rough Heston model. This is done to address some of the limitations identified for the initial approach. The log-normal terminal law is estimated under the physical measure and is not required to agree with the VIX option quotes. It also cannot reproduce the upward-sloping VIX smile, and says nothing about the S&P 500 despite the VIX being derived from the SPX option surface. We re-solve the same optimisation over Monte Carlo scenarios from the joint law with SPX options admitted to the hedging universe.

We evaluate the indifference prices at varying strikes under three configurations to separate the effect of the model change and extended traded universe. Results show the choice of model impacts the bid-ask spread which is explained by a change in the underlying law and adding SPX options narrows the spread at the right tail under the Quadratic Rough Heston the tail, where the leverage correlation of the joint law makes deep SPX puts an effective substitute for thin VIX quotes, while leaving it unchanged near the money. Both effects are smaller than that of the risk-aversion parameter.

Acknowledgements

Thanks to all who have helped make this possible.

Contents

1	Introduction	6
2	Preliminaries: Pricing and Hedging in Incomplete Markets	9
2.1	Market Incompleteness	10
2.1.1	Complete and Incomplete Markets	10
2.1.2	No-Arbitrage and the Price Interval	10
2.2	Preferences and Risk Measures	11
2.2.1	Convex Risk Measures	11
2.2.2	From Utility to the Entropic Risk Measure	12
2.3	Utility Indifference Pricing	13
2.3.1	Origins, Definition and Main Results	13
2.4	Discretising the Expectation	15
2.4.1	Integral Quadrature and Epi-Convergence	15
2.4.2	Monte Carlo and Sample Average Approximation	16
2.4.3	The Discretised Problem as an Exponential Conic Program	17
2.5	The SPX, the VIX and Forward Variance	17
2.5.1	The SPX Option Surface	17
2.5.2	The VIX and the Forward Variance Curve	18
2.6	Models for the Terminal Law	18
2.6.1	The Log-OU Model	19

2.6.2	Stochastic Volatility and the Joint SPX/VIX Problem .	20
2.6.3	The Quadratic Rough Heston Model	20
2.7	Summary	22
3	Indifference Pricing under the Log-OU Model	23
3.1	Data and Model Estimation	23
3.1.1	Datasets	23
3.1.2	Parameter Estimation	24
3.1.3	Terminal Moments and the Median of the VIX	24
3.2	The Portfolio Optimisation Problem	26
3.2.1	Formulation	26
3.2.2	Linear Reformulation of the Cost and Budget Constraints	27
3.2.3	Quadrature Discretisation	28
3.2.4	Convexity of the Discretised Problem	28
3.3	Indifference Pricing	29
3.3.1	The Cash-Invariance Reduction	30
3.3.2	The Digital Claim	31
3.4	Numerical Implementation	32
3.5	Results	32
3.5.1	Optimal Portfolio and Terminal Wealth	32
3.5.2	Indifference Prices and Bid–Ask Spreads	33
4	Joint SPX–VIX Law: Calibration and Extended Hedging	36
4.1	Overview	36
4.2	Calibration and Simulation of the QRH Model	37
4.2.1	Computing the Forward Variance	37
4.2.2	Simulating the SPX and VIX Paths	39

4.2.3	QRH Calibration: Optimisation, Objective and Algorithm	41
4.2.4	Calibration Results	44
4.3	Portfolio Optimisation under the Joint Law	45
4.3.1	Scenario Set and the Extended Tradable Universe . . .	45
4.3.2	The Discretised Problem	46
4.4	Results and Comparison	46
4.4.1	Optimal Portfolio and Terminal Wealth	46
4.4.2	Indifference Prices: Model Effect vs. Tradable Universe Extension Effect	48
5	Conclusion	51
A	Python Code	53
A.1	ProjectTask.ipynb	55
A.2	ProjectTask_QRH.ipynb	58
A.3	optimizer.py	64
A.4	qrh_utils.py	66
A.5	qrh_calibration.py	71
A.6	qrh_optimizer.py	73

Chapter 1

Introduction

Portfolio optimization and hedging are central problems in modern finance, and the classical treatment beginning with Merton [44] assumes that contingent claims can be exactly replicated by trading in the available assets. That assumption fails in most markets of practical interest. Illiquidity, transaction costs, bid-ask spreads, portfolio constraints and non-traded underlyings all render the market incomplete, and in an incomplete market the arbitrage-free price is not unique. No-arbitrage alone pins the value of a claim down only to an interval, which is typically far too wide to be of use. Selecting a single price from that interval requires additional input, and utility-based indifference pricing supplies it in the form of the risk preferences of a single agent. The indifference price is the amount of cash that leaves the agent exactly as well off, under the selected risk measure, whether or not the agent takes on the claim. It is effectively a subjective price, and the gap between the buying and the selling price measures the residual risk that the agent cannot hedge away.

This project applies that framework to claims on the CBOE Volatility Index. The VIX is a relevant test case because it is not a traded asset, so a claim written on it cannot be hedged in its own underlying, and the available instruments are exchange-traded call and put options on the VIX, quoted with wide spreads and limited depth at each strike. In this setting the agent encounters incompleteness from a non-traded underlying, proportional transaction costs, and finite liquidity. We consider static hedging, in which the portfolio is chosen once at time $t = 0$ and held to maturity, and we compute indifference buying and selling prices for a digital claim on the VIX across a range of strikes. Chapter 2 reviews the theory this rests on, in particular the results of Hodges and Neuberger [40], Pennanen [48] on

existence in illiquid discrete-time markets, and Roux and Xu [55] on the exponential case, together with the convex risk measure machinery of Föllmer and Schied [24] and the quadrature discretization of Pennanen and Koivu [49].

Computing the indifference price depends on the agent’s preferences, represented by the utility function or risk measure, and on the probability law of the terminal payoffs, which in this context requires modelling the log VIX as a discrete-time Ornstein–Uhlenbeck (OU) process estimated by least squares from historical data. The OU model is convenient as it admits closed-form expressions for the mean and variance at maturity, which allows for the construction of a quadrature grid and a convex program that solves the resulting optimization problem. Although the OU model serves the base objective and is tractable, it does have its limitations as it cannot reproduce the VIX smile, it is estimated under the physical measure while being used to value options quoted under a risk-neutral one, and it ignores the S&P 500 despite the VIX being derived from SPX options.

The main contribution of this work is to remove those restrictions. We replace the log-OU specification with the Quadratic Rough Heston (QRH) model of Gatheral, Jusselin and Rosenbaum [29], simulated using the quadratic rough hybrid scheme of Bourgey and Gatheral [9] and calibrated jointly to the SPX and VIX implied volatility surfaces observed on the quote date. Calibration of this model gives a joint risk-neutral law for the pair (S_T, VIX_T) , which allows SPX options to be included in the set of hedging instruments alongside VIX options and makes it possible to price claims whose payoff depends on both indices at once.

Specifically, this project makes three contributions:

1. It assesses whether the log-OU model is an adequate description of the market in which the hedging portfolio is actually traded, comparing its implied terminal density against the risk-neutral density recovered from VIX option quotes, and finds that it is not.
2. It implements and calibrates the quadratic rough Heston model to joint SPX and VIX smiles, and reformulates the portfolio optimization problem over Monte Carlo scenarios drawn from the resulting joint law, showing that convexity is preserved.
3. It measures the impact on the indifference bid–ask spread of including SPX options in the tradable universe, and the sensitivity of the

indifference price to the choice of model, against its sensitivity to the risk-aversion parameter.

This report is a resubmission. An earlier version of this project was submitted for this module and examined in September 2025 [46], and is referred to throughout as the initial submission. That version covered the material of Chapter 3, the log-OU law, the quadrature discretisation and the exponential conic solve, and took as its third part a change of preferences rather than a change of model. The entropic objective was replaced by a power disutility, a non-negativity constraint on terminal wealth was imposed, and the indifference price was recovered by a line search over wealth. The present report retains the baseline of Chapter 3, whose estimation, discretisation and solved values are unchanged, removes the power-utility extension entirely, and replaces it with the calibration and extended hedging work of Chapter 4. Chapter 2 is new. Material carried over from [46] is identified as such at the point at which it is used.

The rest of this project is organised as follows. Chapter 2 reviews pricing and hedging in incomplete markets, covering utility indifference pricing, convex risk measures, static hedging under liquidity constraints, the numerical methods employed here, and the literature on modelling the VIX and on the joint SPX/VIX calibration problem. Chapter 3 formulates and solves the portfolio optimization and indifference pricing problem for VIX claims under the log-OU model, using integral quadrature and exponential conic programming, and assesses the adequacy of that law against the market-implied density. Chapter 4 implements and calibrates the quadratic rough Heston model, uses the calibrated joint law to price and hedge with both SPX and VIX options, and compares the results against the baseline. Chapter 5 concludes.

Chapter 2

Preliminaries: Pricing and Hedging in Incomplete Markets

This chapter covers the theory the rest of this project rests on, where we formally state results and the assumptions that accompany them, and discuss how each fits into the objective of this project. It is organised around the three questions that have to be answered before an indifference price can be computed at all: how a claim should be valued when it cannot be replicated (Sections 2.1–2.3), how the expectation in the objective should be evaluated numerically (Section 2.4), and what probability law the terminal payoffs should be evaluated under (Sections 2.5–2.6).

Throughout, we work on a probability space $(\Omega, \mathcal{F}, \mathbb{P})$ carrying all the randomness in the model. Trading takes place at a single date $t = 0$ and positions are held to a fixed maturity T , so the only filtration we need is the trivial one at time 0 together with $\mathcal{F}_T$. We write $J = \{0, 1, \dots, |J|\}$ for the index set of tradable assets, with $j = 0$ the risk-free asset, $x \in \mathbb{R}^J$ for the vector of positions, P^j for the payoff of asset j at T , and c for the payoff of the claim being priced. $\mathbb{Q}$ denotes a risk-neutral (martingale) measure and $\mathbb{P}$ the physical measure.

One notational point should be settled here, since this chapter is the only one in which both objects appear. Following the project brief we write $\xi_t = \log \text{VIX}_t$ for the log-VIX, which is the state variable of Chapter 3. We also write $\xi_t(u) = \mathbb{E}_t^{\mathbb{Q}}[V_u]$ for the forward variance curve, which is the state variable of Chapter 4. The two are distinguished by the horizon argument: the log-VIX never carries one and the forward variance curve always does. They are introduced in Sections 2.6.1 and 2.5.2 respectively and are not used

together anywhere in this thesis.

2.1 Market Incompleteness

2.1.1 Complete and Incomplete Markets

In the context of indifference pricing, a market is *complete* if every contingent claim can be exactly replicated by trading in the available assets, and *incomplete* otherwise. In our static setting the set of claims that can be manufactured from the traded assets is

$$\mathcal{C} = \left\{ \sum_{j \in J} P^j x^j : x \in \mathbb{R}^J \right\} \subseteq L^0(\Omega, \mathcal{F}_T, \mathbb{P}), \quad (2.1)$$

and the market is complete precisely when $\mathcal{C} = L^0$. A claim $c \notin \mathcal{C}$ means that we cannot replicate the payoff of c from our set of tradable assets.

There are four primary sources of incompleteness. The first is the VIX being a *non-tradable underlying*, as it is an index computed from S&P 500 option prices rather than an asset that can be bought and held, the details are properly established in the CBOE VIX whitepaper and an accompanying paper by Avellaneda respectively [14, 2]. A claim on the VIX cannot be delta-hedged in its own underlying and only VIX futures and options are available as instruments. The second is *proportional transaction costs*, since every instrument is quoted with a bid-ask spread and the agent buys at the ask and sells at the bid [40, 55]. The third is *finite liquidity*, only a limited quantity is available at the top of the limit order book at each strike, so positions are bounded above by the displayed depth [41, 33]. The fourth is *discrete trading*, and in our case the extreme version of it, since the portfolio is chosen once at $t = 0$ and held to maturity. The first is the main failure of replication in this context, while the other three are frictions that make replication fail generally and are present in every market, including those that are theoretically considered complete. The problem in this project has all four at once.

2.1.2 No-Arbitrage and the Price Interval

An *arbitrage* is a portfolio that costs nothing to set up and at some terminal state in the future yields a non-negative payoff that is strictly positive with

positive probability. In the linear market model this is the condition

$$\mathcal{C} \cap L_+^0 = \{0\}, \quad (2.2)$$

that is, the only attainable claim with a non-negative payoff is the zero claim. The Fundamental Theorem of Asset Pricing states that (2.2) holds if and only if the set

$$\mathcal{M} = \{ \mathbb{Q} \sim \mathbb{P} : \mathbb{E}_{\mathbb{Q}}[P^j] = p^j \quad \forall j \in J \} \quad (2.3)$$

of equivalent martingale measures is non-empty, and the market is complete if and only if $\mathcal{M}$ is a singleton [35, 36, 17, 21].

In an incomplete market no-arbitrage is not violated, what is lost is uniqueness of $\mathcal{M}$ because it now contains more than one measure, and instead of a single price the theory delivers only the no-arbitrage interval

$$\pi_{\text{sub}}(c) = \inf_{\mathbb{Q} \in \mathcal{M}} \mathbb{E}_{\mathbb{Q}}[c] \quad \text{and} \quad \pi_{\text{sup}}(c) = \sup_{\mathbb{Q} \in \mathcal{M}} \mathbb{E}_{\mathbb{Q}}[c], \quad (2.4)$$

the sub and super hedging bounds as detailed in [23]. Any price strictly inside $[\pi_{\text{sub}}, \pi_{\text{sup}}]$ is consistent with no-arbitrage. The bounds are correct, but they do not serve our application well, because this approach takes no account of the risk preferences of the agent. That is the entire motivation for what follows.

2.2 Preferences and Risk Measures

2.2.1 Convex Risk Measures

Definition 2.1 (Convex risk measure). *Let $\mathcal{X}$ be a linear space of real random variables. A map $\rho : \mathcal{X} \rightarrow \mathbb{R}$ is a convex risk measure if, for all $X, Y \in \mathcal{X}$, $c \in \mathbb{R}$, and $\alpha \in [0, 1]$,*

$$\begin{aligned} (\textbf{Monotonicity}) \quad & X \geq Y \text{ a.s.} \Rightarrow \rho(X) \leq \rho(Y), \\ (\textbf{Convexity}) \quad & \rho(\alpha X + (1 - \alpha)Y) \leq \alpha \rho(X) + (1 - \alpha)\rho(Y), \\ (\textbf{Cash invariance}) \quad & \rho(X + c) = \rho(X) - c. \end{aligned}$$

The three axioms are due to Artzner, Delbaen, Eber and Heath [1] in the coherent case and were relaxed to the convex case by Föllmer and Schied [24] and independently by Frittelli and Rosazza Gianin [26], see [25] for the full treatment. Monotonicity says a uniformly better payoff cannot be

riskier, and is what makes the composition argument of Section 3.2.4 work. Cash invariance says that adding a sure amount of cash reduces the capital requirement by exactly that amount, and it is the property that collapses the indifference price to a difference of two optimal values in Section 2.3. Convexity is the one worth spelling out.

Informally, convexity implies that the risk of the blend $\alpha X + (1 - \alpha)Y$ of two positions is no worse than the blend of their risks meaning diversification does not increase risk. In a pricing and optimization context it means that every local minimum is then a global minimum and the optimal value is well defined for the resulting convex program from our problem formulation, this conclusion takes from [10]. This is why we explicitly check convexity in Section 3.2.4.

2.2.2 From Utility to the Entropic Risk Measure

A *utility function* $u : \mathbb{R} \rightarrow \mathbb{R}$ is an increasing, concave function that ranks random wealth by the expected utility criterion $X \mapsto \mathbb{E}[u(X)]$; increasing says more wealth is preferred, and concave says the agent is risk-averse, since Jensen's inequality gives $\mathbb{E}[u(X)] \leq u(\mathbb{E}[X])$. The specification used throughout this thesis is exponential (CARA) utility $u(x) = -e^{-\lambda x}$, whose absolute risk aversion coefficient $-u''/u'$ is the constant λ [53, 4]. We work only with the exponential case. A *risk measure* takes the opposite sign convention and assigns to a random position a number interpreted as a capital requirement $\rho(X)$, is the amount of cash that has to be added to X to make it acceptable [1]. The two are views of the same concept and the bridge between them is the certainty equivalent.

Definition 2.2 (Certainty equivalent). *Let u be a strictly increasing utility function and X a random payoff. The certainty equivalent of X is the sure amount $\text{CE}(X)$ that the agent regards as exactly as good as X (adapted from [56, 25]):*

$$u(\text{CE}(X)) = \mathbb{E}[u(X)], \quad \text{that is} \quad \text{CE}(X) = u^{-1}(\mathbb{E}[u(X)]). \quad (2.5)$$

The associated risk measure is $\rho(X) := -\text{CE}(X)$.

The certainty equivalent is the natural route from preferences to prices because it converts a distribution into a single number denominated in cash, which is the unit a price has to be quoted in. Taking $u(x) = -e^{-\lambda x}$ in (2.5) gives the *entropic risk measure*

$$\rho_\lambda(X) = \frac{1}{\lambda} \log \mathbb{E}[e^{-\lambda X}], \quad (2.6)$$

which is the objective used in Chapters 3 and 4. It is applied to the agent's net outflow at maturity c and divided by the wealth normalisation $\hat{w}$, giving $\mathcal{V}(c) = \frac{1}{\lambda} \log \mathbb{E}[\exp(\lambda c / \hat{w})]$. A larger outflow is worse, so $\mathcal{V}$ is non-decreasing in c where ρ in Definition 2.1 is non-increasing in wealth, the two are the same object read with opposite signs, $\mathcal{V}(c) = \rho_\lambda(-c / \hat{w})$. Three properties make this the preferred objective of the indifference pricing literature [12]. It satisfies all three axioms of Definition 2.1, and in particular it is the only standard certainty equivalent that is cash-invariant. It is also defined for negative wealth, so no positivity constraint has to be imposed.

A consequence of constant absolute risk aversion is that the optimal positions taken do not depend on the wealth of the agent. Power utility removes that feature, but it loses cash invariance, and the problem would then require a line search over wealth rather than two convex solves. The exponential specification is used throughout and the power case lies outside the scope of this thesis. Other utility specifications exist such as the mean–variance criteria in incomplete markets [58], SAHARA utilities which interpolate between the two families [16], and the neural approximation of the whole hedging problem in deep hedging [3].

2.3 Utility Indifference Pricing

2.3.1 Origins, Definition and Main Results

Indifference pricing was introduced by Hodges and Neuberger [40], who observed that with proportional transaction costs the Black–Scholes replication argument breaks down, and instead proposed to define the price of an option as the cash addition that leaves an optimally investing agent indifferent to writing it. Davis et al. [19] present a more rigorous coverage in continuous time, formulating the value function as the solution of an HJB variational inequality and showing that the optimal policy has a no-transaction region where the agent does not rebalance at all. Their formulation sets the basis for later developments.

A second line of work established what the price actually is in the exponential case. Rouge and El Karoui [54] and Delbaen et al. [20] showed that the exponential indifference price can be written as a difference of relative entropies,

$$\pi_s(c) = \frac{1}{\lambda} \left[\inf_{\mathbb{Q} \in \mathcal{M}} \{ \mathbb{E}_{\mathbb{Q}}[\lambda c] + H(\mathbb{Q} | \mathbb{P}) \} - \inf_{\mathbb{Q} \in \mathcal{M}} H(\mathbb{Q} | \mathbb{P}) \right], \quad (2.7)$$

so that in the limit of small claims the pricing measure is the minimal entropy martingale measure. Becherer [5] extended this to general semimartingale settings. The underlying utility maximisation theory is due to Kramkov and Schachermayer [42, 43], and Henderson [37] and Henderson and Hobson [38] address claims on non-traded assets, showing that closed form representations are generally lost once utility is not exponential.

The results most relevant to this project work in discrete time and drop the frictionless assumption. Pennanen [48] studies optimal investment and claim valuation in illiquid markets, where the cost of trading is a convex rather than linear function of the position, and proves existence of optimal strategies and lower semicontinuity of the optimal value function under a condition that generalises no-arbitrage. All results are obtained by the direct method, without duality. Lower semicontinuity here is the nonlinear-market equivalent of the closed set of attainable claims that no-arbitrage requires in the classical model [17], without it the infimum in (2.8) might be approached but never reached, and the optimal portfolio would not be well defined. Convex duality for the same problem is developed in [51], and Roux and Xu [55] treat the exponential disutility case under proportional transaction costs. On the applied side, Armstrong, Pennanen and Rakwongwan [41] price index options by static hedging under finite liquidity, and Pennanen and Rakwongwan [50] extend this to semi-static strategies. The formulation in Chapter 3 is closest to [41], the main difference being that the underlying of the claim is itself not traded.

Adapting from the literature covered we formulate the definition of the indifference price (2.3), particularly results from [48] and specifications of [52]. Let $\mathcal{D}$ be the set of feasible positions, bounded by the depth available in the market, $S(x)$ the cost of setting up the position x , and define the optimal value of the hedging problem for an agent with wealth w and liability c as

$$\varphi(w, c) = \inf_{x \in \mathcal{D}} \left\{ \mathcal{V} \left(c - \sum_{j \in J} P^j x^j \right) \mid S(x) \leq w \right\}. \quad (2.8)$$

Definition 2.3 (Indifference price). *For an agent with initial wealth $\bar{w}$ and existing liability $\bar{c}$, the indifference selling price (2.9) of a claim c is the smallest premium that leaves the agent no worse off after selling, and the indifference buying price (2.10) is the largest premium the agent would pay for it:*

$$\pi_s(\bar{w}, \bar{c}; c) = \inf \left\{ w \in \mathbb{R} : \varphi(\bar{w} + w, \bar{c} + c) \leq \varphi(\bar{w}, \bar{c}) \right\}, \quad (2.9)$$

$$\pi_b(\bar{w}, \bar{c}; c) = \sup \left\{ w \in \mathbb{R} : \varphi(\bar{w} - w, \bar{c} - c) \leq \varphi(\bar{w}, \bar{c}) \right\}. \quad (2.10)$$

The price is subjective by design as it depends on the agent's risk measure, risk aversion, wealth and existing book. Also, due to the cash-invariance property, the indifference price equation can be further simplified which we cover in Section 3.3.1 representing the indifference selling and buying price respectively as

$$\pi_s = \hat{w}(\varphi(0, \bar{c} + c) - \varphi(0, \bar{c})), \quad \pi_b = \hat{w}(\varphi(0, \bar{c}) - \varphi(0, \bar{c} - c)). \quad (2.11)$$

Each price is then two convex solves, and the wealth level drops out completely. This highlights the advantage of using an entropic objective.

An important property of these prices follow from convexity and monotonicity of $\mathcal{V}$ in the optimal value function (2.8). Stated here because it serves as the benchmark/standard for evaluation of results of Sections 3.5 and 4.4.2 and they are stated in this form in [12, 37].

Proposition 2.1. *Let $\mathcal{V}$ be a convex, non-decreasing, cash-invariant risk measure and let φ be as in (2.8). Then $\pi_b(c) \leq \pi_s(c)$, so the quoted spread is never crossed.*

This property is standard and well known in practice as a crossed spread is an indication that something is wrong in the formulation or the solution, and it is tested for every price obtained later on.

2.4 Discretising the Expectation

Both formulations in this project reduce to minimising an expectation of a convex function of the decision variables. The two subsequent chapters differ in how that expectation is evaluated, this section introduces both methods and the trade-off between them.

2.4.1 Integral Quadrature and Epi-Convergence

The *integral quadrature* is used to approximate an integral by a finite weighted sum of function evaluations at chosen nodes,

$$\mathbb{E}[f(x, \xi_T)] = \int_{\mathbb{R}} f(x, \xi) p_{\xi_T}(\xi) d\xi \approx \sum_{i=1}^M \omega_i f(x, \xi_i), \quad \omega_i = p_{\xi_T}(\xi_i) \Delta \xi_i, \quad (2.12)$$

where $\{\xi_i\}_{i=1}^M$ are the nodes and $\{\omega_i\}$ the weights. The nodes can be constructed by any convention but this project implements equally spaced nodes weighted by the density are the simplest, and Gauss–Hermite nodes are the natural alternative for a Gaussian integrand [49].

Integral quadrature is the right tool for Chapter 3 for two reasons. First, the integrand there depends on the random state through the scalar ξ_T , and the log-OU model of Section 2.6.1 has its density in closed form, so a one-dimensional deterministic rule is easy to work with. Second, the approximation is consistent. Pennanen and Koivu [49] show that quadrature discretisations of convex stochastic programs *epi-converge* to the original problem, so that optimal values converge and every cluster point of a sequence of discretised optimal solutions is optimal for the true problem. The same discretisation is used in the static hedging problems of [41, 50], which is the closest published precedent for our formulation.

2.4.2 Monte Carlo and Sample Average Approximation

Monte Carlo simulation replaces the deterministic nodes and weights by N independent draws from the law of the state and equal weights $\omega_i = 1/N$,

$$\mathbb{E}[f(x, \zeta)] \approx \frac{1}{N} \sum_{i=1}^N f(x, \zeta^{(i)}), \quad \zeta^{(1)}, \dots, \zeta^{(N)} \text{ i.i.d. } \sim \text{law of } \zeta. \quad (2.13)$$

The law of large numbers gives almost sure convergence, and the central limit theorem gives an error of order $\mathcal{O}(N^{-1/2})$ with a constant that does not depend on the dimension of ζ , extensively covered in [31]. Applied to an optimisation problem this is the sample average approximation, whose convergence theory mirrors the epi-convergence result above but with stochastic instead of the deterministic error [57].

Monte Carlo is required in Chapter 4 because the QRH model employed is non-Markovian, has no closed-form terminal density, and the object we need is the *joint* law of (S_T, VIX_T) , which is two-dimensional and can only be obtained by simulating the path.

2.4.3 The Discretised Problem as an Exponential Conic Program

For discretisations, the expectation in the entropic objective is replaced by a finite weighted sum making the objective take the form:

$$\frac{1}{\lambda} \log \sum_i \omega_i \exp(g_i(x)), \quad g_i(x) \text{ affine in } x, \quad (2.14)$$

which is a log-sum-exp of affine functions. This is convex and non-decreasing in each g_i , and is representable in the exponential cone, along side the linear inequality constraints imposed by the limit order book, the discretised problem then becomes an *exponential conic program*, solvable by an interior-point method [10, 45]. In this regards, every optimisation reported in this project is an instance of (2.14), the two chapters differ only in how the weights ω_i are generated and in how many terms the sum has.

2.5 The SPX, the VIX and Forward Variance

2.5.1 The SPX Option Surface

The S&P 500 (SPX) is a capitalisation-weighted index of 500 large US listed companies and is the most heavily traded equity index option market in the world. In the context of this project, the main property of interest is the *option surface* which is the collection of European call and put prices $C(K, T)$, $P(K, T)$ across strikes and maturities, equivalently expressed as an implied volatility surface. There are two key facts about the option surface relevant to this project, The first is that it is downward sloping in strike, the well-documented equity skew. The second is that it determines the risk-neutral marginal density of the underlying at maturity, by Breeden and Litzenberger [11],

$$p_{S_T}(K) = e^{rT} \frac{\partial^2 C(K, T)}{\partial K^2}, \quad (2.15)$$

where p_{S_T} is the density of S_T and K the level at which it is evaluated. Equation (2.15) is a no-arbitrage identity that applies to both the SPX and VIX chain as it recovers the market's risk-neutral density of VIX_T , which is the benchmark against which the model densities of Section 2.6 are assessed.

2.5.2 The VIX and the Forward Variance Curve

In simple terms the VIX is the market's price for thirty days of S&P 500 volatility. Formally it is the square root of a discretised 30-day variance swap rate on the SPX, computed by Cboe from a chain of out-of-the-money SPX options [15, 14] as

$$\text{VIX}_t^2 = \frac{2}{\delta} \sum_i \frac{\Delta K_i}{K_i^2} e^{r\delta} Q(K_i) - \frac{1}{\delta} \left(\frac{F}{K_0} - 1 \right)^2, \quad \delta = 30/365, \quad (2.16)$$

where $Q(K_i)$ is the mid price of the out-of-the-money option at strike K_i and F the forward. It is convenient to introduce the *forward variance curve*

$$\xi_t(u) := \mathbb{E}_t^{\mathbb{Q}}[V_u], \quad u \geq t, \quad (2.17)$$

where V_u is the instantaneous variance of the SPX at time u , $\xi_t(u)$ is the price today of a contract paying the realised variance at u , and is observable from the option surface. In the continuous limit (2.16) is exactly the average of the forward variance curve over the next thirty days, see [13] for details on the continuous limit derivation,

$$\text{VIX}_t^2 = \frac{1}{\delta} \int_t^{t+\delta} \xi_t(u) du, \quad (2.18)$$

which is the representation used throughout Chapter 4 and the reason forward-variance models are the natural class for the joint problem [7].

The main take-away for this project setup is, the VIX is a *functional of the SPX option surface*, not a primitive asset and since (2.18) shows that VIX_T and S_T share the same variance process, any model of the VIX that says nothing about the SPX is discarding information, and any hedging universe restricted to VIX options is smaller than it needs to be.

2.6 Models for the Terminal Law

The remaining input is the law under which the terminal payoffs are evaluated. This section sets out the two specifications used in this project, the simple one that Chapter 3 adopts and the one that Chapter 4 replaces it with, and states the case for the replacement.

2.6.1 The Log-OU Model

The *Ornstein–Uhlenbeck* (OU) process is the Gaussian mean-reverting diffusion, with dynamics

$$d\xi_t = a(\bar{\xi} - \xi_t) dt + \sigma dW_t, \quad (2.19)$$

with $a > 0$ the speed of mean reversion, $\bar{\xi}$ the mean level and σ the diffusion coefficient. Applied to $\xi_t = \log \text{VIX}_t$, sampling (2.19) at a fixed daily interval gives the AR(1) form $\xi_t - \xi_{t-1} = -a(\xi_{t-1} - \bar{\xi}) + \sigma \varepsilon_t$, estimated by ordinary least squares in Section 3.1.2. Working in logs keeps the VIX positive while leaving the dynamics Gaussian, and mean reversion is the stylised fact of volatility time series that this method captures [2, 59].

This methodology employed in Chapter 3 is used to express our resulting terminal law in closed form. which is exactly what the quadrature rule (2.12) needs. Alternatives to the log-OU approach exist, Grünbichler and Longstaff [32] model the volatility level directly with a square-root process, which keeps positivity without taking logs but has a non-central chi-squared transition density, Detemple and Osakwe [22] price volatility options under a range of mean-reverting configs, and jump and tempered-stable extensions are explored in [59, 60]. We are going ahead with the log-OU model in Chapter 3 because the Gaussian terminal density makes the quadrature-based convex program a straightforward solve, and because it is the specification provided in the project brief [52].

Limitations of a VIX-Only OU Model

The model has four main limitations. First, the *measure mismatch*, the parameters in (2.19) are estimated from historical VIX data, that is under the real world measure $\mathbb{P}$, and the resulting terminal law is then used to value a portfolio of options quoted under the risk-neutral measure $\mathbb{Q}$. No change of measure appears anywhere in the argument. Second, the model is *not calibrated to the hedging instruments*. The VIX option quotes already contain a risk-neutral density via (2.15), and the log-OU model ignores this. Third, a lognormal terminal law *cannot reproduce the upward-sloping VIX smile*, which is part of the known stylised facts of the VIX option market. Fourth, and most significant is that the model *ignores information from the SPX*. It also restricts the hedging universe to VIX options when SPX options provides a proxy to manage residual risk when pricing VIX claim.

An alternative to the OU has to be risk-neutral and calibrated to traded

instruments, to reproduce the upward-sloping VIX smile, to be consistent with the SPX surface, and to deliver a *joint* law of (S_T, VIX_T) rather than a marginal. That is what we introduce in the following section.

2.6.2 Stochastic Volatility and the Joint SPX/VIX Problem

A *stochastic volatility* model makes the instantaneous variance of the underlying a random process, which is what allows a single model to generate a non-flat implied volatility surface. The canonical example is Heston [39],

$$\frac{dS_t}{S_t} = \sqrt{V_t} dW_t, \quad dV_t = \kappa(\theta - V_t) dt + \eta\sqrt{V_t} dB_t, \quad d\langle W, B \rangle_t = \rho dt, \quad (2.20)$$

where the correlation $\rho < 0$ produces the equity skew. This class is relevant to Chapter 4, where the objective is a convex risk measure evaluated on payoffs depending on both S_T and VIX_T , so their joint distribution is required. Since (2.18) expresses VIX_T as an average of the forward variance curve at T , a model of the variance process delivers both quantities from the same simulated path.

Fitting both surfaces at once is hard, and for a long time the *joint SPX/VIX calibration problem* was regarded as the outstanding open problem in volatility modelling [34]. Matching the steep short-dated SPX skew requires a large volatility of volatility, and a large volatility of volatility produces a VIX smile that is too high and too flat relative to what is quoted. Continuous models in the Heston family could not solve this, and attempts to force a fit drive $|\rho|$ towards 1 and η to unreasonable values [29]. Guyon [34] showed that a joint fit exists by solving a non-parametric optimal-transport problem, which settles the existence question without giving a dynamic model.

2.6.3 The Quadratic Rough Heston Model

The *quadratic rough Heston* (QRH) model of Gatheral, Jusselin and Rosenbaum [29] is the specification we adopt. In the form used by Bourgey and Gatheral [9],

$$\frac{dS_t}{S_t} = -\sqrt{V_t} dW_t, \quad V_t = Y_t^2 + c, \quad Y_t = \bar{Y} + \int_{-\infty}^t \kappa(t-s)\sqrt{V_s} dW_s, \quad (2.21)$$

with the gamma kernel

$$\kappa(\tau) = \frac{\nu}{\Gamma(\alpha)} \tau^{\alpha-1} e^{-\lambda_\kappa \tau}, \quad \alpha = H + \frac{1}{2}, \quad (2.22)$$

giving four parameters $(H, \nu, \lambda_\kappa, c)$ plus the level $\bar{Y}$. The structural difference from (2.20) is that V is driven by the same Brownian motion as the price and is a quadratic functional of a kernel-weighted average of past returns. There is no correlation parameter. This model embeds the Zumbach effect in its set up, which is the observation that past returns predict future volatility asymmetrically, and the model arrives as a scaling limit of the quadratic Hawkes processes of Blanc, Donier and Bouchaud [8] through the super-Heston construction of Dandapani, Jusselin and Rosenbaum [18].

Quadratic feedback produces heavy-tailed volatility and an upward-sloping VIX smile without an unreasonably large volatility of volatility, which is how the model resolves the tension in (2.20). Gatheral, Jusselin and Rosenbaum [29] were the first to fit both surfaces with so few parameters.

Equation (2.21) is non-Markovian, so there is no transition density to step forward and prices are obtained by simulating the path. What makes this practical [9] is that the conditional forward process $y_t(u) = \mathbb{E}_t[Y_u]$ follows the dynamics of an affine forward variance model [30]. Knowing this, we can adapt the hybrid quadratic-exponential scheme of Gatheral [27], built for that class and derived from the hybrid scheme of Bennedsen, Lunde and Pakkanen [6]. That is the *quadratic rough hybrid* scheme, which we implement in Section 4.2.2. The terminal VIX is computed from the same shocks that produce the price path, using

$$\text{VIX}_T^2 = \frac{1}{\delta} \int_0^\delta (y_T(T+u)^2 + c) (1 + K_0(\delta - u)) du. \quad (2.23)$$

where K_0 is the integral of the resolvent of the squared kernel $\kappa^2(\tau)$.

The key take-away relevant to this project is that a single underlying process produces both S_T and VIX_T , so each path is a draw from their joint law. That joint sample is the scenario set for the portfolio problem of Chapter 4, and it is what allows us to include SPX options into the set of tradable instruments $\mathcal{D}$, extending the specifications of [52].

As for the limitations relevant to this project, Bourgey and Gatheral [9] find that the VIX smile is persistently misfit at low strikes, and that the model does not reproduce the empirical skew-stickiness ratio and they conclude the specification may be too simple. We adopt this model because it is

parsimonious and has a detailed simulation scheme and reference implementation which we use as a benchmark to evaluate calibration results in Section 4.2.4.

2.7 Summary

Two distinct literatures have been reviewed and they do not directly intersect. First, the pricing in incomplete markets literature [40, 48, 55, 41, 50] takes the terminal law of the underlying as given, and in applications that law is usually a simple log-normal or a low-dimensional diffusion estimated under $\mathbb{P}$. Its relevance is in modelling frictions, constraints and preferences. Second is the joint SPX/VIX calibration literature [34, 29, 9], it produces terminal laws that are realistic and consistent with quoted prices, but it works throughout in a frictionless, complete-market setting where a price is an expectation under a calibrated measure and hedging is assumed rather than solved.

What the illiquid-market side supplies is the formulation of Chapter 3; what the calibration side supplies is the law of Chapter 4; and the contribution is to solve the first problem under the second law, with a hedging universe widened to match. That makes it possible to measure two quantities neither literature reports on its own and answer the questions how much of the indifference spread is removed by admitting SPX options into the tradable universe, and how the sensitivity of the price to the choice of model compares with its sensitivity to the risk-aversion parameter.

Chapter 3

Indifference Pricing under the Log-OU Model

This chapter formulates and solves the portfolio optimisation and indifference pricing problem for a terminal claim on the VIX under the log-OU law of Section 2.6.1, discretised by the integral quadrature of Section 2.4.1. It reproduces the standard formulation and provides the baseline against which Chapter 4 is measured. Section 3.1 estimates the model, Section 3.2 sets up the optimisation, Section 3.3 derives the indifference price, and Sections 3.4–3.5 report the implementation and the results.

The formulation, the estimation and the numerical results of this chapter are carried over from the initial submission [46]. This chapter focuses on implementation and results only. The estimated parameters, the quadrature grid, the solved portfolios and the reported indifference prices are unchanged.

3.1 Data and Model Estimation

3.1.1 Datasets

For the numerical computations, we use two primary datasets:

- A historical time series of daily opening prices of the CBOE VIX index, covering 01/02/1990 to 11/04/2024. This dataset is used to estimate the log-OU model for $\xi_t = \ln(\text{VIX}_t)$.

- A cross-sectional snapshot of best bid and offer prices of call and put options on the VIX index, together with the corresponding bid and ask sizes across strikes, on the date 21/06/2024. This dataset defines the set of tradable assets and the feasible trading positions at $t = 0$.

All options share the maturity date 17/07/2024, which is 18 trading days after the quote date. The risk-free rate for both lending and borrowing is taken to be zero throughout, as specified in the project brief [52].

3.1.2 Parameter Estimation

We estimate the discrete-time OU specification of Section 2.6.1,

$$\Delta\xi_t \equiv \xi_t - \xi_{t-1} = -a(\xi_{t-1} - \bar{\xi}) + \sigma \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, 1), \quad (3.1)$$

which rearranges to the AR(1) form

$$\xi_t = a\bar{\xi} + (1 - a)\xi_{t-1} + \sigma\varepsilon_t. \quad (3.2)$$

The parameters $(a, \bar{\xi})$ are estimated from (3.2) by Ordinary Least Squares over the full sample of daily opening prices. The diffusion term σ is then computed as the standard deviation of the residuals of the fitted AR(1) model,

$$\sigma = \sqrt{\frac{1}{T-1} \sum_{t=1}^T r_t^2}, \quad r_t = \xi_t - a\bar{\xi} - (1 - a)\xi_{t-1}. \quad (3.3)$$

The estimated values are given in Table 3.1.

a	$\bar{\xi}$	σ
0.024495	2.909862	0.076587

Table 3.1: Estimated log-OU parameters, daily VIX opening prices 01/02/1990–11/04/2024.

3.1.3 Terminal Moments and the Median of the VIX

The terminal law (3.10) follows by unrolling the recursion (3.2):

$$\xi_T = a\bar{\xi} + (1-a)\xi_{T-1} + \sigma\epsilon_T, \quad \mathbb{E}[\epsilon_t] = 0 \quad (3.4)$$

$$\xi_T = a\bar{\xi} + (1-a)a\bar{\xi} + (1-a)^2\xi_{T-2} + (1-a)\sigma\epsilon_{T-1} + \sigma\epsilon_T \quad (3.5)$$

$$\xi_T = a\bar{\xi} \sum_{t=0}^{T-1} (1-a)^t + (1-a)^T \xi_0 + \sigma \sum_{t=0}^{T-1} (1-a)^t \epsilon_{T-t} \quad (3.6)$$

$$\xi_T = a\bar{\xi} \frac{1 - (1-a)^T}{a} + (1-a)^T \xi_0 + \sigma \sum_{t=0}^{T-1} (1-a)^t \epsilon_{T-t} \quad (3.7)$$

Therefore the expectation is

$$\begin{aligned} \mathbb{E}[\xi_T] &= a\bar{\xi} \frac{1 - (1-a)^T}{a} + (1-a)^T \xi_0 \\ &= \bar{\xi} + (1-a)^T (\xi_0 - \bar{\xi}) \end{aligned} \quad (3.8)$$

and, since the ϵ_t are independent with unit variance, the variance and standard deviation are

$$\begin{aligned} \mathbb{V}[\xi_T] &= \sigma^2 \sum_{t=0}^{T-1} (1-a)^{2t} \mathbb{V}[\epsilon_{T-t}] \\ &= \sigma^2 \frac{1 - (1-a)^{2T}}{1 - (1-a)^2} \\ \sigma(\xi_T) &= \sqrt{\mathbb{V}[\xi_T]} \end{aligned} \quad (3.9)$$

Taking $T = 19$, corresponding to the 18 trading days from 21/06/2024 to 17/07/2024, and substituting the estimates of Table 3.1 gives the values in Table 3.2.

$\mathbb{E}[\xi_T]$	$\sigma(\xi_T)$
2.70289	0.26756

Table 3.2: Mean and standard deviation of the log-VIX at maturity.

The terminal law of the log VIX is given as

$$\xi_T \sim \mathcal{N}\left(\bar{\xi} + (1-a)^T (\xi_0 - \bar{\xi}), \sigma^2 \frac{1 - (1-a)^{2T}}{1 - (1-a)^2}\right), \quad (3.10)$$

and the median m of the VIX at maturity is given by solving for m in:

$$\begin{aligned}\mathbb{P}(\text{VIX}_T \leq m) &= \mathbb{P}(\xi_T \leq \ln m) \\ &= \Phi\left(\frac{\ln m - \mathbb{E}[\xi_T]}{\sigma(\xi_T)}\right) = \frac{1}{2},\end{aligned}$$

and is evaluated as

$$\begin{aligned}\text{Med}[\text{VIX}_T] &= \exp\left(\sigma(\xi_T)\Phi^{-1}(1/2) + \mathbb{E}[\xi_T]\right) \\ &= \exp(\mathbb{E}[\xi_T]) \\ &= 14.8609.\end{aligned}$$

This distribution is what Section 3.2.3 uses to construct the terminal price grid and the quadrature weights.

3.2 The Portfolio Optimisation Problem

3.2.1 Formulation

The problem is to hedge an arbitrary terminal claim on the VIX index with payoff $c(\xi_T)$ given an initial wealth w . To hedge the terminal claim, we select the optimal positions $x \in \mathbb{R}^J$ in a set of risky assets and one risk-free asset. The positions x^j for $j \in \{0, \dots, J\}$ are determined at $t = 0$ and held until $t = T$, making this a *static hedging* problem.

Each position in the risky assets x^j , $j \in J \setminus \{0\}$, has an associated cost $S^j(x^j)$, where $0 \in J$ denotes the risk-free asset, which has no transaction cost. The agent buys at the ask p_a^j and sells at the bid p_b^j , so

$$S^j(x^j) = \begin{cases} p_a^j x^j & \text{if } x^j \geq 0, \\ p_b^j x^j & \text{if } x^j \leq 0, \end{cases} \quad (3.11)$$

and the total cost at $t = 0$ is bounded above by the initial wealth w . For each tradable asset, the maximum buying and selling volume is constrained to the volume available at $t = 0$ at the offer (q_a^j) and the bid (q_b^j) respectively,

giving the set of feasible portfolios

$$D := \prod_{j \in J} [-q_b^j, q_a^j]. \quad (3.12)$$

The set of tradable assets comprises European call and put options on the VIX index at varying strikes, all sharing the maturity date 17/07/2024. The payoff at maturity of each risky asset is $P^j(\xi_T)$, and the payoff of the risk-free asset is $P^0 = 1$. The optimisation problem is then

$$\min_{x \in D} \quad \mathcal{V} \left(c(\xi_T) - \sum_{j \in J} P^j(\xi_T) x^j \right) \quad \text{s.t.} \quad \sum_{j \in J} S^j(x^j) \leq w, \quad (3.13)$$

where $\mathcal{V}$ is the entropic risk measure of Section 2.2.2, parameterised by the risk-aversion parameter λ and the wealth normalisation $\hat{w}$,

$$\mathcal{V}(c) = \frac{1}{\lambda} \ln \mathbb{E} [\exp(\lambda c / \hat{w})]. \quad (3.14)$$

3.2.2 Linear Reformulation of the Cost and Budget Constraints

The cost function (3.11) is piecewise linear and therefore non-differentiable at the origin, which common interior point solvers do not support. It is made linear by decomposing each position into ancillary buy and sell variables (see [47, Section 10.2.1]),

$$x^j = x_+^j - x_-^j, \quad x_+^j \geq 0, \quad x_-^j \geq 0, \quad (3.15)$$

so that the cost becomes the linear function $S(x) = p_a^\top x_+ - p_b^\top x_-$. Bounding x_+ and x_- separately by the displayed depth then enforces both the feasible set (3.12) and the requirement that the buy and sell quantity for every risky asset cannot exceed the quantity available on the limit order book at $t = 0$. Problem (3.13) becomes

$$\begin{aligned} \min_{x_+, x_-} \quad & \mathcal{V} \left(c(\xi_T) - \sum_{j \in J} P^j(\xi_T) x^j \right) \\ \text{s.t.} \quad & p_a^\top x_+ - p_b^\top x_- \leq w \\ & x^j = x_+^j - x_-^j \\ & 0 \leq x_+^j \leq q_a^j \\ & 0 \leq x_-^j \leq q_b^j. \end{aligned}$$

3.2.3 Quadrature Discretisation

The objective (3.14) requires the expectation of the random function

$$f(x, \xi) = \exp \left(\frac{\lambda}{\hat{w}} \left(c(\xi) - \sum_{j \in J} P^j(\xi) x^j \right) \right),$$

for which no closed-form expression exists. Since f depends on the random state only through the scalar ξ_T , whose density is known in closed form from (3.10), the integral quadrature rule (2.12) applies directly, and with it the epi-convergence guarantee of [49] discussed in Section 2.4.1.

We take $M = 100$ equally spaced nodes $\{\xi_i\}_{i=1}^M$ spanning $[\mu_T - 9\sigma_T, \mu_T + 9\sigma_T]$, with $\mu_T = \mathbb{E}[\xi_T]$ and $\sigma_T = \sigma(\xi_T)$ from Table 3.2 and spacing $\Delta\xi = 18\sigma_T/(M - 1)$. The weights are $\omega_i = p_{\xi_T}(\xi_i)\Delta\xi$, where p_{ξ_T} is the Gaussian density of (3.10). A nine standard deviation window captures the tails to well beyond the range of quoted strikes, and $M = 100$ is the resolution specified in the project brief [52]. The discretised problem is

$$\begin{aligned} \min_{x_+, x_-} \quad & \frac{1}{\lambda} \ln \sum_{i=1}^M \omega_i \exp \left(\frac{\lambda}{\hat{w}} \left(c(\xi_i) - \sum_{j \in J} P^j(\xi_i) x^j \right) \right) \\ \text{s.t.} \quad & x^j = x_+^j - x_-^j, \\ & 0 \leq x_+^j \leq q_a^j, \\ & 0 \leq x_-^j \leq q_b^j. \end{aligned} \tag{3.16}$$

The budget constraint of (3.13) does not appear among the constraints of (3.16). It has not been dropped: the cash-invariance reduction of Section 3.3.1 shows that the optimal cash holding always exhausts the budget, so the constraint binds with equality and can be substituted out, after which the cost of the position enters the objective through the term $S(x)/\hat{w}$ carried inside $W_T(x)$. Imposing it separately would be redundant, and the implementation of Section 3.4 does not.

3.2.4 Convexity of the Discretised Problem

Existence of an optimal solution requires the objective to be convex. We establish this in two steps: first for $\mathcal{V}$ itself, then for the discretised objective in (3.16).

Convexity of $\mathcal{V}$. For random variables $X, Y \in L^0$ and $\theta \in [0, 1]$, Hölder's inequality with conjugate exponents $1/\theta$ and $1/(1 - \theta)$ gives

$$\begin{aligned} \mathbb{E} \left[\exp \left(\frac{\lambda}{w} (\theta X + (1 - \theta)Y) \right) \right] &= \mathbb{E} \left[\exp \left(\frac{\lambda}{w} X \right)^\theta \exp \left(\frac{\lambda}{w} Y \right)^{1-\theta} \right] \\ &\leq \left(\mathbb{E} \left[\exp \left(\frac{\lambda}{w} X \right) \right] \right)^\theta \left(\mathbb{E} \left[\exp \left(\frac{\lambda}{w} Y \right) \right] \right)^{1-\theta}. \end{aligned}$$

Taking $\frac{1}{\lambda} \ln$ of both sides, which is increasing and therefore preserves the inequality, and using $\ln(u^\theta v^{1-\theta}) = \theta \ln u + (1 - \theta) \ln v$,

$$\begin{aligned} \frac{1}{\lambda} \ln \mathbb{E} \left[\exp \left(\frac{\lambda}{w} (\theta X + (1 - \theta)Y) \right) \right] &\leq \theta \frac{1}{\lambda} \ln \mathbb{E} \left[\exp \left(\frac{\lambda}{w} X \right) \right] + (1 - \theta) \frac{1}{\lambda} \ln \mathbb{E} \left[\exp \left(\frac{\lambda}{w} Y \right) \right] \\ \mathcal{V}(\theta X + (1 - \theta)Y) &\leq \theta \mathcal{V}(X) + (1 - \theta) \mathcal{V}(Y), \end{aligned}$$

so $\mathcal{V} : L^0 \rightarrow \mathbb{R}$ is convex, as required by Definition 2.1.

Convexity of the discretised problem. Given the quadrature nodes and weights $\{(\xi_i, \omega_i)\}_{i=1}^M$, the scenario payoffs $P_i \in \mathbb{R}^{|J|}$ with $P_i^j = P^j(\xi_i)$, the claim values $c_i = c(\xi_i)$, and the decomposition $x = x_+ - x_-$, the discretised objective is

$$f(x_+, x_-) = \frac{1}{\lambda} \log \left(\sum_{i=1}^M \omega_i \exp \left(\frac{\lambda}{w} [c_i - P_i^\top (x_+ - x_-)] \right) \right).$$

For each node the map $g_i(x_+, x_-) := \frac{\lambda}{w} [c_i - P_i^\top (x_+ - x_-)]$ is affine, and $h(g_1, \dots, g_M) := \log \left(\sum_i \omega_i e^{g_i} \right)$ is convex and non-decreasing in each coordinate. Therefore $f = \frac{1}{\lambda} h \circ g$ is convex by the composition rule, and since every constraint in (3.16) is linear, the discretised problem is convex. It is precisely the exponential conic program (2.14) of Section 2.4.3.

3.3 Indifference Pricing

The indifference prices π_s and π_b were defined in (2.3) in terms of the optimal value

$$\varphi(w, c) = \inf_{x \in D} \left\{ \mathcal{V} \left(c(\xi_T) - \sum_{j \in J} P^j(\xi_T) x^j \right) \left| \sum_{j \in J} S^j(x^j) \leq w \right. \right\}. \quad (3.17)$$

As stated there, computing them directly would require a search over w . This section establishes the reduction (2.11) that removes that search, and then specifies the claim to be priced.

3.3.1 The Cash-Invariance Reduction

The indifference price is the least amount of wealth $w \in \mathbb{R}$ that needs to be added to the portfolio to allow the agent to buy or sell the claim without increasing the risk of the agent's portfolio. Equivalently, it is the optimal bid–ask price for a claim c from the agent's perspective.

We first verify cash invariance for $\mathcal{V}$ directly. Writing $z(\xi_T) = c(\xi_T) - \sum_{j \in J} P^j(\xi_T)x^j$,

$$\begin{aligned} \mathcal{V}(z(\xi_T) + \bar{w}) &= \frac{1}{\lambda} \ln \mathbb{E} \left[\exp \left(\frac{\lambda}{\hat{w}} (z(\xi_T) + \bar{w}) \right) \right] \\ &= \frac{1}{\lambda} \ln \left(\exp \left(\frac{\lambda}{\hat{w}} \bar{w} \right) \cdot \mathbb{E} \left[\exp \left(\frac{\lambda}{\hat{w}} z(\xi_T) \right) \right] \right) \\ &= \mathcal{V}(z(\xi_T)) + \frac{\bar{w}}{\hat{w}}. \end{aligned}$$

The same property then carries to the optimal value (3.17). Introducing an explicit cash decision variable $w^* \in \mathbb{R}$ and writing $S(x) = \sum_{j \in J} S^j(x^j)$,

$$\varphi(\bar{w}, \bar{c}) = \inf_{x \in D, w^* \in \mathbb{R}} \left\{ \mathcal{V} \left(\bar{c}(\xi_T) - \left(\sum_{j \in J} P^j(\xi_T)x^j + w^* \right) \right) \middle| S(x) + w^* \leq \bar{w} \right\}$$

which by cash invariance equals

$$\varphi(\bar{w}, \bar{c}) = \inf_{x \in D, w^* \in \mathbb{R}} \left\{ \mathcal{V} \left(\bar{c}(\xi_T) - \sum_{j \in J} P^j(\xi_T)x^j \right) - \frac{w^*}{\hat{w}} \middle| S(x) + w^* \leq \bar{w} \right\}.$$

For any $x \in D$ the objective is a decreasing function of w^* , which is bounded above by the constraint $w^* \leq \bar{w} - S(x)$. The infimum is therefore attained when the cash position reaches its maximum, $w^* = \bar{w} - S(x)$. Substituting,

$$\begin{aligned}
\varphi(\bar{w}, \bar{c}) &= \inf_{x \in D} \left\{ \mathcal{V} \left(\bar{c}(\xi_T) - \sum_{j \in J} P^j(\xi_T) x^j \right) - \frac{\bar{w} - S(x)}{\hat{w}} \right\} \\
&= \inf_{x \in D} \left\{ \mathcal{V} \left(\bar{c}(\xi_T) - \sum_{j \in J} P^j(\xi_T) x^j \right) + \frac{S(x)}{\hat{w}} \right\} - \frac{\bar{w}}{\hat{w}}.
\end{aligned}$$

The budget constraint disappears in the second line because it holds with equality after the substitution, and the budget is absorbed into the objective through the term $S(x)/\hat{w}$. This gives

$$\varphi(\bar{w}, \bar{c}) = \varphi(0, \bar{c}) - \frac{\bar{w}}{\hat{w}},$$

Applying this identity to the selling condition $\varphi(\bar{w} + w, \bar{c} + c) \leq \varphi(\bar{w}, \bar{c})$,

$$\begin{aligned}
\varphi(0, \bar{c} + c) - \frac{\bar{w} + w}{\hat{w}} &\leq \varphi(0, \bar{c}) - \frac{\bar{w}}{\hat{w}} \\
\varphi(0, \bar{c} + c) - \varphi(0, \bar{c}) &\leq \frac{w}{\hat{w}} \\
w &\geq \hat{w} (\varphi(0, \bar{c} + c) - \varphi(0, \bar{c})),
\end{aligned}$$

and symmetrically for the buying price, recovering (2.11):

$$\pi_s(\bar{w}, \bar{c}; c) = \hat{w} (\varphi(0, \bar{c} + c) - \varphi(0, \bar{c})) \quad (3.18)$$

$$\pi_b(\bar{w}, \bar{c}; c) = \hat{w} (\varphi(0, \bar{c}) - \varphi(0, \bar{c} - c)) \quad (3.19)$$

Each price is therefore two solves of the convex program (3.16), and the initial wealth $\bar{w}$ drops out entirely.

3.3.2 The Digital Claim

The terminal claim priced here is a digital call on the VIX, with payoff

$$c(\xi_T) = \$1000 \cdot \mathbf{1}_{\{\exp(\xi_T) > K\}}. \quad (3.20)$$

The option pays \$1000 if the VIX at maturity, recovered as $VIX_T = \exp(\xi_T)$, exceeds the strike K . We price it at strikes $5k$ for $k = 0, 1, \dots, 10$, which spans the range from near-certain payoff to near-certain expiry worthless and therefore exposes the full shape of the indifference price as a function of moneyness.

3.4 Numerical Implementation

All computations are implemented in Python using `numpy` and `pandas` for data handling and `scipy` for the OU parameter estimation. Problem (3.16) is solved with `CVXPY` and `MOSEK` as the interior point solver [45].

From `vix_prices.csv` we form $\xi_t = \log(VIX_t)$ and estimate the OU parameters by OLS, regressing $\Delta\xi_t$ on ξ_{t-1} to obtain a , $\bar{\xi}$ and σ . Given the maturity of 18 trading days we compute $\mathbb{E}[\xi_T]$ and $\sigma(\xi_T)$ from (3.8) and (3.9) and build the grid $\xi_i = \mu_T - 9\sigma_T + i\Delta\xi$ with $\Delta\xi = 18\sigma_T/(M-1)$ for $i = 0, \dots, M-1$.

Market inputs are taken from `vix_options_data.csv`, which provides bid-ask quotes and sizes for the universe of tradable assets. In the `PortfolioOptimizer` class (Appendix A) the cash position is embedded implicitly by expressing the total wealth excluding the claim value as $W_T(x) = \sum_{j \in J \setminus \{0\}} P^j(x^j) + (w - S(x))$. This avoids having to declare unbounded cash positions with `numpy.inf`, which the `MOSEK` interface rejects. The tradable positions are decomposed into non-negative buy and sell variables $x = x_+ - x_-$, and the linear inequality constraints enforce the budget $S(x) \leq w$ and the limit order book depths $0 \leq x_+ \leq q_a$, $0 \leq x_- \leq q_b$. The objective is expressed with `cp.log_sum_exp` applied to $\frac{\lambda}{\bar{w}}[c - P(x_+ - x_-)] + \log \omega$, which keeps it DCP-compliant and numerically stable.

3.5 Results

3.5.1 Optimal Portfolio and Terminal Wealth

We first assess how the optimal terminal wealth

$$W_T(x) = \sum_{j \in J} P^j(\xi_T) x^j$$

varies with the VIX at maturity under different risk aversion. Figure 3.1 compares $\lambda = 2$ and $\lambda = 20$, with summary statistics in Table 3.3.

For the riskier portfolio ($\lambda = 2$) the terminal wealth has a significantly steeper slope, meaning that it is more sensitive to the VIX at maturity, while the risk-averse portfolio ($\lambda = 20$) maintains a flatter profile. This is consistent with the terminal wealth densities in Figure 3.2, which show a far more concentrated distribution for the risk-averse portfolio, against a higher-variance distribution with substantial probability of large negative terminal wealth for the riskier one.

Table 3.3: Summary statistics of terminal wealth by risk aversion.

	Min(\$)	Max(\$)	Mean(\$)	Median(\$)	Std. Dev.	Kurtosis
$\lambda = 2$	-1,448,571.63	381,567.01	63,100.23	268,507.11	455,204.17	2.41
$\lambda = 20$	35,129.80	271,743.99	200,955.07	207,994.43	58,731.76	0.44

3.5.2 Indifference Prices and Bid–Ask Spreads

Figure 3.3a plots the indifference buying and selling prices of the digital claim (3.20) across strikes, and Figure 3.3b the resulting spread.



Figure 3.1: Optimal terminal wealth against the VIX at maturity for $\lambda = 2$ (blue) and $\lambda = 20$ (red).

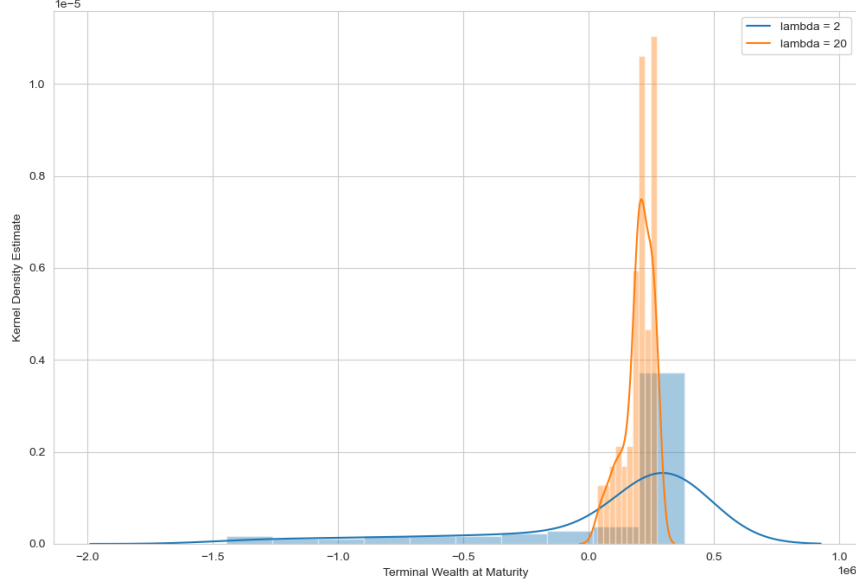
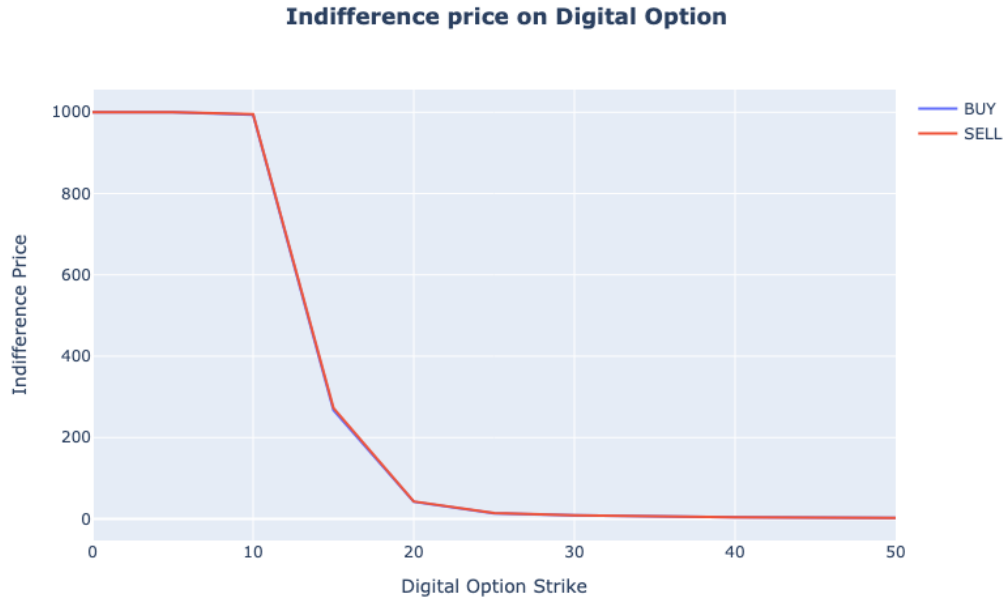


Figure 3.2: Kernel density estimate of terminal wealth for $\lambda = 2$ (blue) and $\lambda = 20$ (green).

The spread peaks near the median of the terminal distribution, where the payoff is the most uncertain. Below a strike of 10 the payoff is almost certain, hence the narrow spread and a price near \$1,000, which is what the agent is willing to pay for a claim that is effectively a cash bond. Above a strike of 25 the payoff becomes unlikely that the price approaches zero and the agent would neither pay for the claim nor offer to write it.



(a) Indifference buying and selling prices against digital strike.



(b) Indifference bid–ask spread against digital strike.

Figure 3.3: Indifference prices and spreads for the digital claim under the log-OU law.

Chapter 4

Joint SPX–VIX Law: Calibration and Extended Hedging

4.1 Overview

This chapter re-solves the problem of Chapter 3 under a different terminal law and a wider hedging universe. Three things change and it is worth naming them at the outset, because the results in Section 4.4 are designed to attribute the difference to each. The terminal law becomes the joint risk-neutral law of (S_T, VIX_T) under a quadratic rough Heston model calibrated to both surfaces, in place of the log-OU marginal of VIX_T . The discretisation becomes Monte Carlo in place of quadrature, for the reason given in Section 2.4.2 and the tradable set is extended to admit SPX options alongside VIX options.

The data and setup of the problem remain the same and all computations use the same quote date (21/06/2024) and the same claim and maturity, (17/07/2024) as Chapter 3. This makes the difference in results attributable either to the model or the tradable set, not to the inputs.

All market data used in this chapter, the SPX and VIX option chains and the VIX futures quotes on the 21/06/2024 snapshot, was obtained from Cboe DataShop.¹

¹<https://datashop.cboe.com/>

4.2 Calibration and Simulation of the QRH Model

The QRH model we implement for this section (2.21) is restated below,

$$\frac{dS_t}{S_t} = -\sqrt{V_t} dW_t, \quad V_t = Y_t^2 + c, \quad Y_t = \bar{Y} + \int_{-\infty}^t \kappa(t-s) \sqrt{V_s} dW_s, \quad (4.1)$$

with the gamma kernel

$$\kappa(\tau) = \frac{\nu}{\Gamma(\alpha)} \tau^{\alpha-1} e^{-\lambda \tau}, \quad \alpha = H + \frac{1}{2}, \quad (4.2)$$

and the path simulation implemented uses the quadratic rough hybrid scheme discussed in Section 2.6.3. As in Section 2.6.3, the subscript on λ_κ separates the kernel decay rate from the risk-aversion coefficient λ of the entropic objective, which reappears in Section 4.3.

In this section we discuss in detail the forward variance ($\xi_0(u) = \mathbb{E}^\mathbb{Q}[V_u]$), construction used here as a market input, the path simulation and computation of the VIX path, the implementation here is adapted from the reference implementation released by Gatheral and Bourgey alongside [9], available at [28]. The sub-sections that follow details the building blocks of the calibration of Algorithm 1 that gives the parameters we will work with to simulate the SPX and VIX for the indifference pricing problem in Section 4.3

4.2.1 Computing the Forward Variance

The initial forward variance curve $\xi_0(u) = \mathbb{E}^\mathbb{Q}[V_u]$ of (2.17) is built from the SPX option surface on the quote date (21/06/2024) and held fixed throughout the calibration. For each expiry we recover the forward and discount factor from put–call parity and filter the dataset for the OTM strip. Illiquid quotes and expiries with too few options are discarded. This leaves 55 usable expiries from three days to five and a half years.

The curve is obtained from the total variance $w(T)$ of the variance swap struck at each expiry. Since $w(T) = \mathbb{E}^\mathbb{Q}\left[\int_0^T V_s ds\right] = \int_0^T \xi_0(s) ds$ by Fubini, the two are related by

$$\xi_0(T) = \frac{dw(T)}{dT}. \quad (4.3)$$

The forward variance curve is the derivative of the total variance curve [7]. We compute $w(T)$ in the $y = \Phi(d_2)$ parametrisation of [28],

$$w(T) = \int_0^1 \sigma_{\text{BS}}(y)^2 T dy, \quad (4.4)$$

$$y = \Phi(d_2), \quad d_2 = -\frac{k}{\sigma\sqrt{T}} - \frac{\sigma\sqrt{T}}{2},$$

with $k = \log(K/F)$. The substitution maps every strike onto a point in $[0, 1]$, so the integral runs over a bounded range.

The integrand is known only at the quoted strikes so we interpolate between them with `PchipInterpolator` and integrate the result with `quad`, holding the variance constant beyond the lowest and highest quotes. PCHIP never returns a value outside the range of the surrounding quotes, so the interpolated variance stays positive.

Each total variance carries a bid–ask width, so the curve is smoothed before it is differentiated. We re-imply a volatility from the bid and from the ask at every strike and take half the resulting width as the allowance ϵ_i for expiry i . The smoothing solves (4.5) for the adjustments e_i , that give the flattest curve consistent with every quote.

$$\begin{aligned} \min_v \quad & v^\top A^{-1} v \\ \text{s.t.} \quad & v_i = w(T_i) + 2\sqrt{w(T_i)}\sqrt{T_i} e_i, \\ & |e_i| \leq \epsilon_i, \quad i = 1, \dots, 55, \end{aligned} \quad (4.5)$$

The objective $v^\top A^{-1} v$ penalises curvature, where A is the Gram matrix of the smoothing kernel φ of [28] with $A_{ij} = \varphi(T_i, T_j)$. A small ridge is added to A before solving. The constraint is the first-order expansion of $(\sigma_i + e_i)^2 T_i$, which states ϵ_i in the same units as the quoted width. The problem is solved by `L-BFGS-B` with an analytic gradient.

The fitted values v define the curve through the expansion $w(u) = \sum_j \varphi(T_j, u) Z_j$ with $Z = A^{-1} v$. Differentiating this and using (4.3) gives the forward variance,

$$\xi_0(u) = \sum_j \partial_u \varphi(T_j, u) Z_j. \quad (4.6)$$

The fit gives $V_0 = \xi_0(0) = 0.012210$, an instantaneous volatility of 11.05%.

Equipped with the curve fitted above, we can now compute the conditional forward level of the state variable, $y_0(u) = \mathbb{E}[Y_u]$, by taking the positive

root of (4.7),

$$y_0(u)^2 = \xi_0(u) - c - \int_0^u \kappa(v)^2 \xi_0(u-v) dv. \quad (4.7)$$

Where the right-hand side is negative, no real y_0 reproduces the market curve at that horizon which the optimizer penalizes in the objective 4.15.

4.2.2 Simulating the SPX and VIX Paths

The simulation runs on a uniform grid $t_n = n\Delta$ for $n = 0, \dots, N$. We set Δ so that every calibrated expiry lands on a grid point. We use two steps per calendar day, which gives $N = 234$ steps out to the last VIX expiry and 60,000 paths. Figure 4.1 shows a sample simulation resulting from the implementation of the following steps.

The curve $y_0(\cdot)$ from Section 4.2.1 is the starting point. It replaces the constant level $\bar{Y}$ in (2.21), so the model matches the observed variance term structure by construction and only $(H, \nu, \lambda_\kappa, c)$ are calibrated.

Each step is taken in four parts. We first form the conditional mean of Y from the shocks already drawn,

$$\hat{y}_n = y_0(t_n) - \sum_j b_j^* \chi_{n-j}, \quad (4.8)$$

where the coefficients b_j^* are kernel integrals computed once before the loop. We then form the conditional variance over the step as the blend

$$\bar{V}_n = \theta \hat{y}_n^2 + (1 - \theta) Y_{n-1}^2 + c, \quad \theta = \frac{1}{2H + 1}, \quad (4.9)$$

and rescale it so that its mean across paths matches $\xi_0(t_n)$. We then draw the return shock and the residual,

$$\chi_n \sim \mathcal{N}(0, \bar{V}_n \Delta), \quad \varepsilon_n \sim \mathcal{N}\left(0, \bar{V}_n (K_{0,0}(\Delta) - K_0(\Delta)^2 / \Delta)\right), \quad (4.10)$$

independently, and finally update the state and the log price,

$$Y_n = \hat{y}_n + \beta_{u\chi} \chi_n + \varepsilon_n, \quad V_n = Y_n^2 + c, \quad X_n = X_{n-1} - \frac{1}{4} (V_n + V_{n-1}) \Delta - \chi_n, \quad (4.11)$$

with $\beta_{u\chi} = K_0(\Delta) / \Delta$ and $X = \log S$. Splitting the innovation into a part proportional to χ_n and an independent residual matches the covariance between the return shock and the variance, this carries the leverage effect into

discrete time, this can be observed visually from the sample simulation in Figure 4.1.

The loop runs once, to the longest expiry. We store $S_T = \exp(X_{N_T})$ at the SPX expiry and retain the full array of shocks $\chi_1, \dots, \chi_N$. Each VIX expiry at N_i steps is then taken as the leading slice $\chi_1, \dots, \chi_{N_i}$ of the same paths, which keeps Δ identical across expiries.

The VIX at each expiry is computed from the same shocks, so every path gives a matched pair. We discretise (2.23) on a $Q = 16$ point Gauss–Legendre rule $\{(u_q, \theta_q)\}$ mapped to $[0, \delta]$ with $\delta = 30/365$. Writing y_q for the conditional forward level at horizon $T + u_q$,

$$\begin{aligned} y_q &= y_0(T + u_q) + \sum_{k=1}^{N_T} a_{k,q} \chi_k, \\ a_{k,q}^2 &= \frac{K_{0,0}(T + u_q - (k-1)\Delta) - K_{0,0}(T + u_q - k\Delta)}{\Delta}, \end{aligned} \quad (4.12)$$

and the VIX at maturity is the weighted sum

$$\text{VIX}_T = 100 \times \sqrt{\frac{1}{\delta} \sum_{q=1}^Q \theta_q (y_q^2 + c) (1 + K_0(\delta - u_q))}. \quad (4.13)$$

The coefficients $a_{k,q}$ depend on the parameters but not on the path, so the matrix $A = (a_{k,q})$ is built once per parameter vector and the whole sample of forward levels is a single matrix product.

The kernel integrals are evaluated in closed form as regularised lower incomplete gamma functions. In particular $K_{0,0}(\tau) = r P(2H, 2\lambda_\kappa \tau)$, and the integrated resolvent is the series

$$K_0(\tau) = \sum_{n \geq 1} r^n P(2Hn, 2\lambda_\kappa \tau), \quad r = \|\kappa\|_{L^2}^2 = \frac{\hat{\nu}^2}{(2\lambda_\kappa)^{2H}}. \quad (4.14)$$

The series is used because $\kappa^2 \sim \tau^{2H-1}$ is singular at the origin. We check admissibility, $r < 1$, before every simulation and reject ill parameters with a penalty instead of simulating them.

One call to this routine therefore returns S_T and the four VIX samples for a given $(H, \nu, \lambda_\kappa, c)$. Section 4.2.3 sets out the objective that scores those samples against the quoted smiles and the search used to minimise it.



Figure 4.1: Sample simulated path of the SPX and VIX using calibrated parameters.

4.2.3 QRH Calibration: Optimisation, Objective and Algorithm

Four parameters are calibrated, $(H, \nu, \lambda_\kappa, c)$, against two sets of targets on the 21/06/2024 snapshot: the SPX smile expiring 17/07/2024, and the first four VIX expiries with a usable smile, 17/07, 21/08, 18/09 and 16/10/2024. Smiles are built by the same OTM filter used for the variance curve, restricted to moneyness $[0.85, 1.10]$ for SPX and $[0.60, 2.50]$ for VIX, which leaves 111 SPX options and 28 to 33 options per VIX expiry. SPX strikes are normalised by the forward so that they are directly comparable to the simulated S_T/S_0 .

The Objective We introduce a custom objective ℓ function for our optimizer. Its first two terms are a vega-weighted least squares fit in implied volatility, the conventional form for smile calibration [29, 9]. We write σ^{mod} and σ^{mkt} for model and market implied volatilities, F for VIX futures levels

and Π for the y_0 shortfall of (4.7),

$$\begin{aligned}
\ell(H, \nu, \lambda_\kappa, c) = & \sum_j \tilde{\mathcal{V}}_j (\sigma_j^{\text{mod}} - \sigma_j^{\text{mkt}})^2 && (\text{SPX smile}) \\
& + \frac{1}{4} \sum_{i=1}^4 \sum_j \tilde{\mathcal{V}}_{ij} (\sigma_{ij}^{\text{mod}} - \sigma_{ij}^{\text{mkt}})^2 && (\text{VIX smiles}) \\
& + \frac{w_F}{4} \sum_{i=1}^4 \left(\frac{F_i^{\text{mod}}}{F_i^{\text{mkt}}} - 1 \right)^2 && (\text{VIX futures}) \\
& + w_\Pi \Pi && (y_0 \text{ shortfall}),
\end{aligned} \tag{4.15}$$

with $w_F = 5$ and $w_\Pi = 10$. The weights $\tilde{\mathcal{V}}$ are Black vegas evaluated at the market volatility and normalised to sum to one within each smile. This makes every smile contribute equally regardless of how many strikes it has, and gives less weight to strikes where a volatility error is inconsequential.

The futures term pins the level of the simulated VIX distribution, which the smiles constrain only in shape. The shortfall term lets the optimizer penalize parameters where (4.7) has no real solution.

Model implied volatilities are computed from the simulated sample directly and the model forward is set to the sample mean of the simulated values, which makes put-call parity hold exactly in the simulated measure, and volatilities are recovered by bisection.

Optimization We carry out the minimization in two stages, set out in Algorithm 1. The first stage runs `differential_evolution` on a coarse configuration of one step per day and 15,000 paths, for 30 generations with a population of 40. The local stage polishes the survivor with `Nelder-Mead` on the production configuration of 234 steps and 60,000 paths for at most 50 iterations, we then keep the best of the two objective values.

The random draws come from `default_rng` seeded once, so the same shocks are reused at every parameter vector and the loss is a deterministic function of the parameters.

Algorithm 1: Two-stage calibration of $(H, \nu, \lambda_\kappa, c)$

Input : option snapshot at $t = 0$; SPX expiry T ; VIX expiries $T_1, \dots, T_4$; bounds $\mathcal{B}$

Output: calibrated parameters $\Theta^* = (H, \nu, \lambda_\kappa, c)$

Market inputs;

Build the variance swap curve $\{(T_i, w(T_i))\}_{i=1}^{55}$ from (4.4);

Re-imply bid and ask volatilities; set the allowances $\{\epsilon_i\}$;

Fit ξ_0 by (4.5); $V_0 \leftarrow \xi_0(0)$;

Build the SPX smile and the four VIX smiles; compute vega weights $\tilde{\mathcal{V}}$;

Function $\text{Loss}(H, \nu, \lambda_\kappa, c; N \text{ steps}, M \text{ paths})$:

```
 $r \leftarrow \hat{\nu}^2 / (2\lambda_\kappa)^{2H};$ 
if  $r \geq 1$  then
  | return  $10^3(1+r)$ ; // skip inadmissible kernel
end

 $y_0(t_n) \leftarrow$  (4.7) on the grid  $t_n = n\Delta$ ; accumulate shortfall  $\Pi$ ;
Draw  $z^X, z^\epsilon \sim \mathcal{N}(0, 1)$  from the fixed seed;

for  $n \leftarrow 1$  to  $N$  do
  |  $\hat{y}_n \leftarrow y_0(t_n) + \sum_j b_j^* \chi_{n-j};$ 
  |  $\bar{V}_n \leftarrow \theta \hat{y}_n^2 + (1 - \theta) Y_{n-1}^2 + c$ , rescaled to match  $\xi_0(t_n)$ ;
  |  $\chi_n \leftarrow \sqrt{\bar{V}_n \Delta} z_n^X;$ 
  |  $\epsilon_n \leftarrow \sqrt{\bar{V}_n (K_{0,0}(\Delta) - \bar{\kappa}(\Delta)^2 / \Delta)} z_n^\epsilon;$ 
  |  $Y_n \leftarrow \hat{y}_n + \frac{\bar{\kappa}(\Delta)}{\Delta} \chi_n + \epsilon_n;$ 
  |  $V_n \leftarrow Y_n^2 + c;$ 
  |  $X_n \leftarrow X_{n-1} - \frac{1}{4}(V_n + V_{n-1})\Delta - \chi_n;$ 
end

 $S_T \leftarrow \exp(X_{N_T});$ 
for  $i \leftarrow 1$  to 4 do
  | VIX $_{T_i} \leftarrow$  (4.13) evaluated from  $\chi_{1:N_i}$ ;
end

Imply model volatilities and forwards from the samples;
return  $\ell$  of (4.15);
```

end

Stage 1 — global search ;

$\Theta^{(1)} \leftarrow \text{DifferentialEvolution}(\text{Loss}(\cdot; N=117, M=15,000), \mathcal{B},$
30 generations, population 40);

Stage 2 — local polish ;

$\Theta^{(2)} \leftarrow \text{NelderMead}(\text{Loss}(\cdot; N=234, M=60,000), \Theta^{(1)}, 50$
iterations);

return $\Theta^* = \arg \min \{\ell(\Theta^{(1)}), \ell(\Theta^{(2)})\}$

4.2.4 Calibration Results

Table 4.1 gives the calibrated parameters against the ranges Bourgey and Gatheral [9] report across four 2024–2025 quote dates. Three of the four sit inside those ranges. The exception is λ_κ , which comes out at 10.04 against a published range of $[5.0, 6.4]$, a kernel memory of about 36 days instead of 60. The squared kernel norm is $\|\kappa\|_{L^2}^2 = 0.599$, inside the admissibility bound. Post calibration the model VIX on the quote date is 13.33 against a market level of 13.20, and the simulated $\mathbb{E}[S_T/S_0] = 1.000227$, so the discretised price process stays a martingale to within Monte Carlo error.

Parameter	Calibrated	[9] range
H	0.164192	0.0233 – 0.2908
ν	1.043496	0.3676 – 1.2942
λ_κ	10.042725	5.0000 – 6.3980
c	0.003469	0.0023 – 0.0190
$\ \kappa\ _{L^2}^2$	0.599418	—
V_0 (market input)	0.012210 (11.05%)	—
VIX ₀ model / market	13.33 / 13.20	—
ℓ	1.7634×10^{-2}	—

Table 4.1: Calibrated QRH parameters, 21/06/2024 snapshot.

The fit is uneven across the two surfaces, and Table 4.2 gives the split. The SPX smile is fitted to 1.21% RMSE in implied volatility across 111 options, close to the width of the quotes themselves. The VIX smiles run from 6.26% to 14.67% RMSE, with the residual concentrated at the low strikes. Section 2.6.3 recorded that [9] find the same failure and attribute it to the specification being too simple. We observe it with a single parameter set fitted to four expiries at one quote date.

Leg	Expiry	Days	Options	IV RMSE	Future (model / market)
SPX	17/07/2024	26	111	1.21%	—
VIX	17/07/2024	26	28	14.67%	14.899 / 14.388
VIX	21/08/2024	61	31	6.26%	14.723 / 15.328
VIX	18/09/2024	89	33	12.30%	16.293 / 16.170
VIX	16/10/2024	117	33	10.39%	18.622 / 18.530

Table 4.2: Calibration fit by smile.

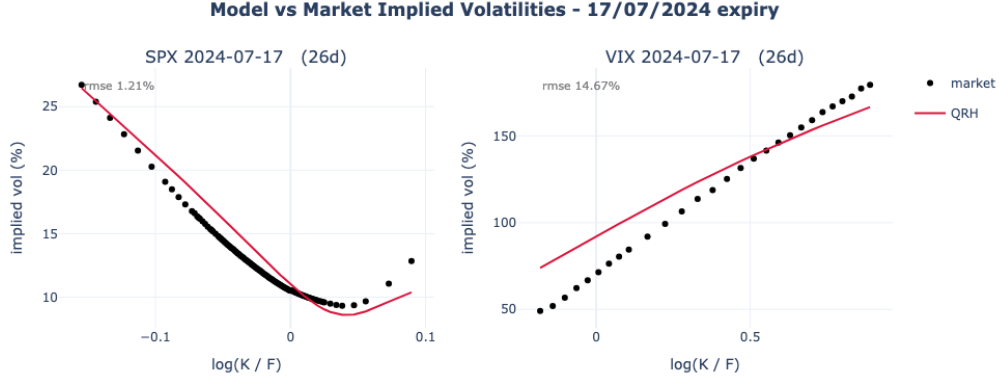


Figure 4.2: Model vs. market implied volatilities. SPX 17/07/2024 (left) and the four calibrated VIX expiries (right).

The parameters calibrated in this section are what we carry into our subsequent objective in Section 4.3.1 which is the simulated joint law of (S_T, VIX_T) .

4.3 Portfolio Optimisation under the Joint Law

4.3.1 Scenario Set and the Extended Tradable Universe

The terminal law is now the joint risk-neutral law of (S_T, VIX_T) under the calibrated model, the discretisation is Monte Carlo, and the tradable set is extended from 118 VIX contracts to 402, adding 284 SPX calls and puts at the same 17/07/2024 expiry. The VIX chain used the same as the set used in Chapter 3.

The scenario set is one QRH simulation to 17/07/2024 at two steps per day, $N = 5,000$ paths, from a fixed seed. Each path yields a pair (S_T, VIX_T) , and the payoff matrix P is built column by column from whichever underlying the contract is written on. The simulation returns S_T/S_0 with unit mean, and this is scaled by the quoted forward $F = 5,486.01$, using the forward is what keeps put–call parity holding in the model measure, and without it the optimiser sees an arbitrage opportunity and will exploit it. Also, the

realised sample has $\text{corr}(S_T, \text{VIX}_T) = -0.91$, which shows the leverage effect generated endogenously by the quadratic feedback in (2.21).

Now that we have switched to Monte Carlo for evaluating the expectation of the utility, the base and claim problems in every indifference price are solved on the same 5,000 draws since the price is a difference of two optimal values by (2.11).

4.3.2 The Discretised Problem

What does not change is the form of the problem, with equally weighted draws $\omega_i = 1/N$, the objective is still a log-sum-exp of affine functions of the positions,

$$\begin{aligned}
& \min_{x_+, x_-} \quad \frac{1}{\lambda} \log \sum_{i=1}^N \frac{1}{N} \exp \left(\frac{\lambda}{\hat{w}} W_i \right) \\
& \text{where} \quad W_i = c_i - P_i^\top (x_+ - x_-) - (w - S(x)) \\
& \text{s.t.} \quad S(x) \leq w, \\
& \quad \quad x = x_+ - x_-, \\
& \quad \quad 0 \leq x_+ \leq q_a, \\
& \quad \quad 0 \leq x_- \leq q_b,
\end{aligned} \tag{4.16}$$

so it remains an instance of (2.14) and therefore an exponential conic program as in Chapter 3. The objective is expressed with `cvxpy.log_sum_exp` applied to $\frac{\lambda}{\hat{w}} W + \log \omega$, which keeps it DCP-compliant and numerically stable, and the problem is solved by CVXPY with MOSEK.

4.4 Results and Comparison

4.4.1 Optimal Portfolio and Terminal Wealth

We solve (4.16) with no claim, $w = \hat{w} = \$100,000$, for the same two risk aversions used in Chapter 3. Both problems solve to optimality. At $\lambda = 2$ the agent takes a position in 286 of the 402 available contracts, 87 of them at the full quoted depth. At $\lambda = 20$ this falls to 193 contracts with 37 at the

depth limit.

Terminal wealth is now a function of two underlyings, so we plot a surface over both, Figure 4.4. Summary statistics are given in Table 4.3. The qualitative observation is similar to what was reported in Chapter 3 and Table 3.3, the risk-averse portfolio has the flatter terminal profile, with a standard deviation of \$2,959 at $\lambda = 20$ against \$21,991 at $\lambda = 2$. The density is also much tighter overall. This follows from the difference between the two terminal VIX laws, and in particular from the fact that the QRH law is calibrated to the prices the hedge is traded at. Both terminal distributions are compared with the observed and market-implied VIX levels in Figure 4.3, with summary statistics in Table ??.

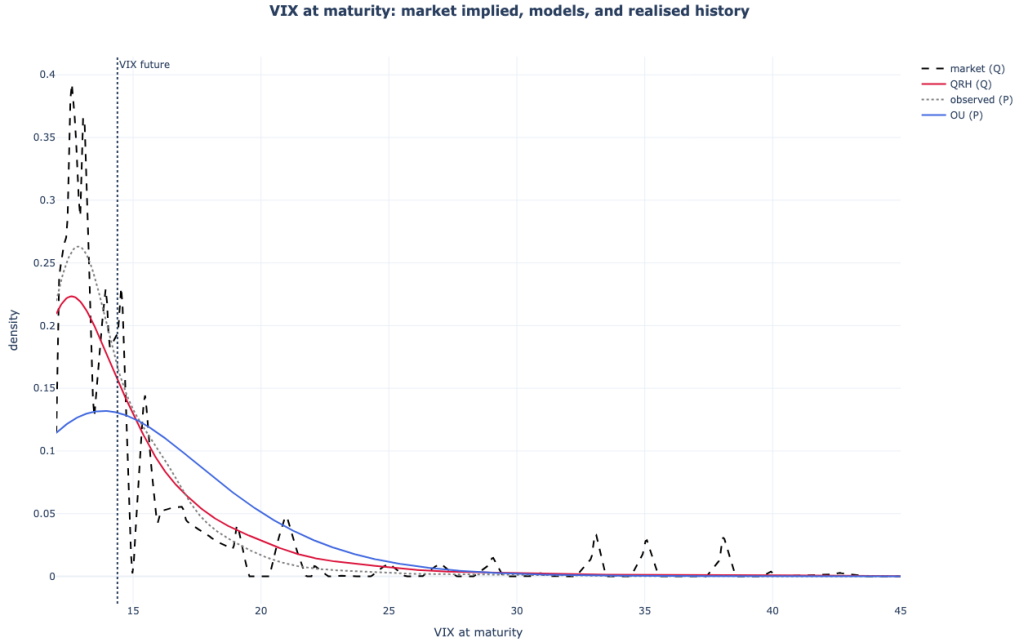


Figure 4.3: Terminal VIX densities at 17/07/2024. Market-implied and QRH are risk-neutral, the historical and log-OU densities are physical. All densities are renormalised over $[12, 45]$.

Also, under the QRH and extended tradable universe, we note that both portfolios are now bounded below by a positive wealth, \$8,254 at $\lambda = 2$ and \$90,132 at $\lambda = 20$, in comparison to one observed in Chapter 3 where $\lambda = 2$ ran down to $-\$1.58\text{m}$. This suggests our agents can now reduce their downside by trading in SPX options on the VIX claim where the originally specified tradable universe of VIX options alone did not allow them to.

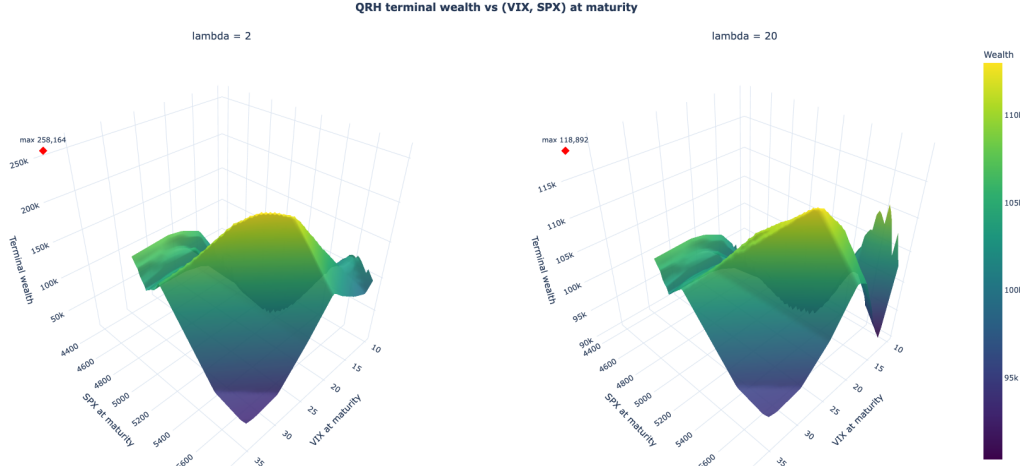


Figure 4.4: Terminal wealth over the joint (VIX_T, S_T) scenarios for $\lambda = 2$ (left) and $\lambda = 20$ (right).

	Min (\$)	Max (\$)	Mean (\$)	Median (\$)	Std. Dev.	Kurtosis
$\lambda = 2$	8,254	258,164	111,964	112,462	21,991	3.53
$\lambda = 20$	90,132	118,892	102,498	102,272	2,959	1.17

Table 4.3: Summary statistics of terminal wealth under QRH with the extended tradable set, 5,000 scenarios.

4.4.2 Indifference Prices: Model Effect vs. Tradable Universe Extension Effect

Indifference prices for the same digital claim (3.20) used in Chapter 3 are computed at strikes $0, 5, \dots, 50$ with $\lambda = 20$, $\hat{w} = \$100,000$ and zero initial wealth, from three configurations. The first is the log-OU law with VIX options only, which is the Chapter 3 result. The second is the QRH law with the same VIX options. The third is the QRH law with SPX options added. The second configuration isolates the effect of the model and the third isolates the effect of the tradable universe. Results are in Table 4.4 and Figure 4.5. The spread is non-negative in all three configurations at every strike, so Proposition 2.1 is satisfied.

The indifference prices agree closely across all three configurations, and the spread carries the information about the difference between the two models and the extended tradable universe. Two trends stand out. The first is the spread spike at $K = 15$ for all three configurations but at different mag-

	$K=10$	$K=15$	$K=20$	$K=25$	$K=30$	$K=35$	$K=40$	$K=45$	$K=50$
<i>Sell price π_s</i>									
OU, VIX only	995.17	280.33	44.22	14.31	9.02	5.22	4.10	2.93	2.41
QRH, VIX only	997.40	261.91	42.75	15.26	8.27	5.60	3.18	2.21	2.21
QRH, VIX+SPX	991.85	266.18	42.38	15.07	8.37	5.96	3.86	2.82	2.82
<i>Spread $\pi_s - \pi_b$</i>									
OU, VIX only	0.93	5.58	0.76	0.16	0.12	0.12	0.08	0.05	0.05
QRH, VIX only	0.43	8.26	0.90	0.48	0.45	0.37	0.23	0.14	0.14
QRH, VIX+SPX	0.87	8.52	0.91	0.09	0.10	0.08	0.07	0.01	0.01
Paths in money	99.0%	33.2%	8.5%	3.1%	1.5%	1.0%	0.5%	0.3%	0.3%

Table 4.4: Indifference prices and bid–ask spreads for the digital claim, $\lambda = 20$. Prices at $K = 0$ and $K = 5$ are \$1,000 in all three configurations and are omitted.

nitudes, which separates the two models. The second is that above $K = 25$ the spread stays wider under the second configuration (QRH + VIX only) than the third (QRH + VIX and SPX), which separates the two tradable universes since both share the same law.

The spike in spread can be explained in two parts. At $K = 15$ which is the closest strike to the sample median of 14.86 under log-OU and 13.65 under QRH, where the payoff of the digital is mostly uncertain $K \in [10, 20]$. At the farther ends of the distribution, the payoff of the digital option claim is more certain and cheap to hedge, so the spread collapses at both ends. The second part is the difference in magnitude, \$8.26 under QRH against \$5.58 under log-OU and the QRH median is the lower of the two, so this is not just uncertainty about the payoff. It comes from the shape of the two distributions in Figure 4.3 and Table 4.5. The QRH law carries far more weight in the upper tail, so more of the claim’s payoff falls in states the hedge cannot reach and hence more residual risk.

	$K=20$	$K=25$	$K=30$	$K=40$	$K=45$
QRH, $\mathbb{P}(\text{VIX}_T > K)$	8.5%	3.1%	1.5%	0.5%	0.3%
Log-OU, $\mathbb{P}(\text{VIX}_T > K)$	13.69%	2.69%	0.453%	0.0114%	0.0019%
Ratio	0.6	1.2	3.3	44	158

Table 4.5: upper-tail probabilities of the terminal VIX under the two laws, on the raw samples. The QRH row is the Monte Carlo frequency over the 5,000 scenarios and the log-OU row is evaluated from the law (3.10) (3.10).

The second trend is attributed to the extension of the tradable universe. This is because above $K = 25$ the spread falls from \$0.48 to \$0.09 at $K = 25$ and from \$0.23 to \$0.07 at $K = 40$, a drop of 3-5x, while at the peak the two QRH configurations are similar. Above $K = 25$ the claim pays only in the tail of the VIX distribution, where the VIX quotes are wide and thin and the hedge cannot be completed. Those are also the states in where the SPX index falls sharply, and deep SPX puts are quoted there with more depth. The extended universe is what closes the spread gap where the market was the most incomplete, and contributes little near the money where the VIX quotes are sufficient enough.

Putting the spread results into context, replacing log-OU with QRH moves the terminal law towards the one implied by the market, and widens the spread accordingly.

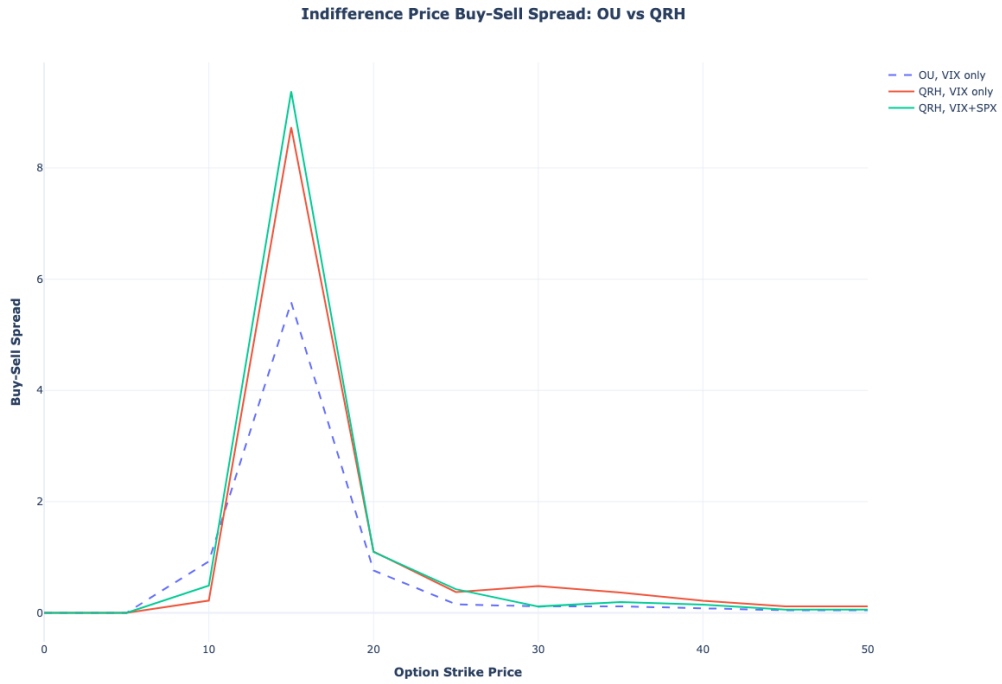


Figure 4.5: Indifference bid–ask spread for the digital claim under the three configurations.

Chapter 5

Conclusion

This project set out to price and hedge a terminal claim on the VIX index from the perspective of a single agent facing an incomplete market, and then, as an extension, ask whether the law under which that price was computed was an adequate description of the market the hedge is actually traded in.

The first half of this project(Chapter 3) establishes the baseline where we modelled the log-VIX as a discrete-time OU process estimated by least squares from 34 years of daily data, showed that the entropic objective is a convex risk measure in the sense of Definition 2.1, and reformulated the budget and depth constraints of the limit order book as linear inequalities, so that the resulting discretised problem is an exponential conic program with a provable epi-convergence guarantee [49]. Cash invariance of the entropic measure reduces each indifference price to a difference of two convex solves, which removes the search over wealth that a general utility would require. The resulting prices for a digital claim behave as the theory predicts, spreads are never crossed, they peak where the payoff is most uncertain, and they collapse at both ends of the strike range.

The second half replaces the log-OU law with the joint risk-neutral law of (S_T, VIX_T) implied by a QRH model, calibrated to the SPX smile and four VIX smiles on the quote date by a two-stage optimizer search, and re-solved the same optimisation problem over Monte Carlo scenarios drawn from that law with the tradable universe extended from 118 VIX contracts to 402 after the addition of SPX options. Solving the three configurations on identical data separates the effect of the model from the effect of the tradable set, and the separation is the main result of the project.

Aside from the primary objectives set out in the project brief [52], the

main findings here come from the comparisons in Section 4.4. We observed that the QRH law closely fits the market-implied terminal distribution of the VIX and SPX, which allows the agent to price the claim under more realistic market conditions. An attribute of the QRH law is its heavier right tail, which is consistent with the literature and with the stylised facts of the VIX, and this produced the two main observations: (1) the relatively larger spike in the spread against the OU; and (2) the persistently wider spread at the right tail end ($K > 25$) under QRH with VIX options only, against QRH with VIX and SPX. The extension of the traded universe to include SPX options served as a means for the agent to hedge the residual risk when pricing the digital claim at strikes in the right tail.

We also highlight some limitations of the implementation and setup of this project, and equally the improvement points and extensions that follow. Most notable is the static hedging setup used throughout, which is more tractable but less realistic, and it would be worth exploring the consequences and feasibility of the same methodology in a multi-period or dynamic hedging setup. It would also be worth exploring a non-linear market impact function in place of the proportional cost of (3.11), which is the setting Pennanen [48] and non-zero or asymmetric lending and borrowing rates, which the zero-rate assumption of the project brief currently rules out.

Appendix A

Python Code

All numerical work for this project is implemented in Python. The code is organised as two Jupyter notebooks, one per results chapter, sitting on top of a small library of reusable modules. The notebooks contain only the data loading, the calls into the library and the plotting; the models, the calibration and the optimisation live in the library.

- `ProjectTask.QRH.ipynb` reproduces Chapter 3 and Chapter 4, the OU parameter estimation, the quadrature grid, the portfolio solves at $\lambda = 2$ and $\lambda = 20$, and the indifference prices of the digital claim across strikes, forward variance fit, the QRH calibration, the joint simulation, and the three-configuration indifference price comparison.
- `optimizer.py` implements the `PortfolioOptimizer` class used in Chapter 3. It builds the payoff matrix from the quoted option chain, decomposes positions into non-negative buy and sell variables, imposes the depth constraints, absorbs the cost of the position into the objective through the cash term of Section 3.3.1, and expresses the entropic objective through `cvxpy.log_sum_exp`. It is agnostic to how the scenarios were generated, which is what allows the same design to serve the quadrature nodes of Chapter 3 and the Monte Carlo draws of Chapter 4.
- `qrh_utils.py` implements the quadratic rough hybrid scheme of Section 2.6.3, the kernel integrals (4.14), the terminal VIX map (4.13), the variance swap curve of (4.4) and the implied volatility inversion.
- `qrh_calibration.py` implements the forward variance fit (4.5), the calibration objective (4.15) and the two-stage search of Algorithm 1.

- `qrh_optimizer.py` adapts the portfolio problem to the joint scenario set and the extended tradable universe of Section 4.3.1.

Running either notebook requires `cvxpy`, `numpy`, `scipy` and `pandas`, together with a valid MOSEK licence.

A.1 ProjectTask.ipynb

```
1 # =====
2 # Chapter 3 -- Indifference pricing under the log-OU
3 # Exported from ProjectTask.ipynb (code cells only;
4 # -----
5
6 # --- Cell 1 -----
7 import numpy as np
8 import pandas as pd
9 import cvxpy as cp
10 from pricing_lib.optimizer import PortfolioOptimizer
11 from scipy.stats import norm
12 import statsmodels.api as sm
13 from scipy.stats import gaussian_kde, linregress, norm
14 import plotly.graph_objs as go
15 import plotly.figure_factory as ff
16 import matplotlib.pyplot as plt
17 from numpy.linalg import inv as np_inv
18 from plotly.subplots import make_subplots
19 import seaborn as sns
20 from datetime import datetime, date
21
22 # --- Cell 2 -----
23 # sns plot style
24 sns.set_style("whitegrid")
25 plt.rcParams['figure.figsize'] = (12, 8)
26 print(cp.installed_solvers())
27
28 # --- Cell 3 -----
29 # Read Data
30 vix_options_df = pd.read_csv("./Data/vix_options_data.
31                               csv")
32 vix_prices_df = pd.read_csv("./Data/vix_prices.csv")
33
34 # Column Formatting
35 vix_prices_df.rename(columns = {"DATE" : "date", "OPEN"
36                               : "px"}, inplace = True)
37 vix_options_df["sym"] = vix_options_df.
38                       underlying_symbol.apply(lambda x: x[1:]).values
39 vix_options_df.rename(columns = {"bid_size_1545": "
40                               bsize",
41                               "ask_size_1545" : "asize",
42                               "bid_1545": "bid",
43                               "ask_1545": "ask"}, inplace =
44                               True)
45 vix_options_df = vix_options_df[["quote_date", "
46                               expiration", "sym", "option_type", "strike", "
47                               bsize", "bid", "ask", "asize"]]
48
49 # --- Cell 4 -----
50 # Process the VIX prices data
51 vix_prices_df['date'] = pd.to_datetime(vix_prices_df['
52 date'])
53 vix_prices_df = vix_prices_df.sort_values(by='date')
54 vix_prices_df['VIX_log'] = np.log(vix_prices_df['px'])
55 vix_prices_df['delta_xi'] = vix_prices_df['VIX_log'].
56                             diff()
57 vix_prices_df['xi_lagged'] = vix_prices_df['VIX_log'].
58                             shift(1)
59 vix_prices_df = vix_prices_df.dropna()
60
61 # Performing OLS
62 regression = linregress(vix_prices_df['xi_lagged'],
63                          vix_prices_df['delta_xi'])
64 beta_1 = regression.slope
65 beta_0 = regression.intercept
66 pred_values = beta_1*vix_prices_df.xi_lagged.values +
67               beta_0
68 obs_values = vix_prices_df.delta_xi.values
69 residuals = obs_values - pred_values
70
71 # Calculate parameters a, xi_bar, and sigma
72 a = -beta_1
73 xi_bar = beta_0 / a
74 sigma = np.std(residuals)
75
76 print(f"OU Model Parameters:")
77
78 print("-" * 40)
79 print(f"{a':<10} | {xi_bar':<10} | {sigma':<10}")
80 print("-" * 40)
81 print(f"{a:<10.5f} | {xi_bar:<10.5f} | {sigma:<10.5f}")
82 print("-" * 40)
83
84 # Calculate date to maturity
85 start_date = date(2024, 6, 21) # t = 0
86 end_date = date(2024, 7, 17) # t = T
87 days = pd.date_range(start = start_date, end = end_date
88                       , freq = "B")
89 T = len(days)-1
90
91 # VIX value at initial date
92 VIX_0 = 13.20
93 xi_0 = np.log(VIX_0)
94
95 # Calculate the mean, median and std. dev. of xi_T
96 phi = 1 - a
97 mu_xi_T = xi_bar + (xi_0 - xi_bar)*(phi**T)
98 std_xi_T = np.sqrt(sigma**2 * (1 - phi**(2*T)) / (1 -
99               phi**2))
100 median_vix_T = np.exp(mu_xi_T)
101
102 print(f"Time to maturity (T) is {T} days.")
103 print(f"Mean and standard deviation of log-VIX at
104       maturity:")
105 print("-" * 40)
106 print(f"{E[xi_T]':<15} | {sigma(xi_T)':<15}")
107 print("-" * 40)
108 print(f"{mu_xi_T:<15.4f} | {std_xi_T:<15.6f}")
109 print("-" * 40)
110 print(f"Median of VIX at maturity: {median_vix_T:.6f}")
111
112 # --- Cell 5 -----
113 # Task 4: solving optimization for lambda = 2 and 20
114 NO_SCENARIOS = 100
115 portfolio_1 = PortfolioOptimizer(lob_data=
116                                 vix_options_df.copy(),
117                                 price_data=
118                                 vix_prices_df.copy(),
119                                 wealth = 1e5,
120                                 w_hat = 1e5,
121                                 init_claim = 0.0,
122                                 claim_sign = 1,
123                                 claim_strike = None,
124                                 risk_param = 2,
125                                 mu = mu_xi_T,
126                                 std = std_xi_T,
127                                 no_scenarios =
128                                 NO_SCENARIOS)
129 portfolio_1.initialize_params()
130
131 portfolio_2 = PortfolioOptimizer(lob_data=
132                                 vix_options_df,
133                                 price_data=
134                                 vix_prices_df,
135                                 wealth = 1e5,
136                                 w_hat = 1e5,
137                                 init_claim = 0.0,
138                                 claim_sign = 1,
139                                 claim_strike = None,
140                                 risk_param = 20,
141                                 mu = mu_xi_T,
142                                 std = std_xi_T,
143                                 no_scenarios =
144                                 NO_SCENARIOS)
145 portfolio_2.initialize_params()
146 portfolio_1.solve()
147 portfolio_2.solve()
148
149 # --- Cell 6 -----
150 # Plot terminal wealth vs terminal VIX scenarios
151 vix_px = portfolio_1.get_vix_prices()
152 terminal_wealth_2 = portfolio_1.get_terminal_wealth()
153 terminal_wealth_20 = portfolio_2.get_terminal_wealth()
```

```

134 fig = make_subplots(rows = 1, cols = 1, subplot_titles
    ↳ = ['Terminal Wealth vs VIX(T)'])
135 fig.add_trace(go.Scatter(y = terminal_wealth_2, x =
    ↳ vix_px, name = "lambda = 2"))
136 fig.add_trace(go.Scatter(y = terminal_wealth_20, x =
    ↳ vix_px, name = "lambda = 20"))
137 fig.add_vline(x = median_vix_T, line_dash = "dash",
138 ↳ annotation_text="Median VIX(T) ",
139 ↳ annotation_position="bottom right")
140 fig.update_yaxes(title_text="<b> Terminal Wealth </b>"
    ↳ , dtick = 2e5, tickprefix = '$', row=1, col=1,)
141 fig.update_xaxes(title_text="<b> VIX Price at Maturity
    ↳ </b>", row=1, col=1 )
142 fig.update_layout(height = 600)
143 fig.show()
144
145 # --- Cell 7 -----
146 # KDE
147 kde_plot, ax = plt.subplots()
148 pal = sns.color_palette()
149 sns.kdeplot(terminal_wealth_2, ax=ax, color=pal[0],
    ↳ label='lambda = 2')
150 sns.kdeplot(terminal_wealth_20, ax=ax, color=pal[1],
    ↳ label='lambda = 20')
151
152 # Histogram
153 ax.hist(terminal_wealth_2, density=True, alpha = 0.4)
154 ax.hist(terminal_wealth_20, density=True, alpha = 0.4)
155 ax.set_xlabel("Terminal Wealth at Maturity")
156 ax.set_ylabel("Kernel Density Estimate")
157 ax.legend()
158 plt.show()
159
160 # --- Cell 8 -----
161 # Summary Statistics
162 pd.options.display.float_format = '{:,.2f}'.format
163 summary_df = pd.DataFrame({"terminal_wealth_2":
    ↳ terminal_wealth_2, "terminal_wealth_20":
    ↳ terminal_wealth_20 })
164 summary_df = summary_df.agg(["min", "max", "mean", "
    ↳ median", "std", "kurtosis", "skew"]).transpose()
165 cash_pos_2 = portfolio_1.w - portfolio_1.Sx.value
166 cash_pos_20 = portfolio_2.w - portfolio_2.Sx.value
167 summary_df["Cash_Allocation"] = [cash_pos_2,
    ↳ cash_pos_20]
168 summary_df
169
170 # --- Cell 9 -----
171 def get_indifference_price( init_claim, strike, w_hat,
    ↳ side):
172     """
173     Function to evaluate indifferet price
174     """
175     NO_SCENARIOS = 100
176     RISK_PARAM = 20
177     if side == "BUY": # BUY CLAIM
178         # Base portfolio
179         P1 = PortfolioOptimizer(lob_data=
    ↳ vix_options_df.copy(), price_data=vix_prices_df.
    ↳ copy(),
180
    ↳ wealth = 0.0,
181
    ↳ w_hat = w_hat,
182
    ↳ init_claim =
    ↳ init_claim, claim_sign = 1, claim_strike = None,
    ↳ risk_param =
183
    ↳ RISK_PARAM, mu = mu_xi_T, std = std_xi_T,
    ↳ no_scenarios =
    ↳ NO_SCENARIOS)
184         # Portfolio that purchases claim
185         P2 = PortfolioOptimizer(lob_data=
    ↳ vix_options_df.copy(), price_data=vix_prices_df.
    ↳ copy(),
186
    ↳ wealth = 0.0,
187
    ↳ w_hat = w_hat,
188
    ↳ init_claim =
    ↳ init_claim, claim_sign = -1, claim_strike = strike
    ↳ ,
189
    ↳ RISK_PARAM, mu = mu_xi_T, std = std_xi_T,
    ↳ no_scenarios =
    ↳ NO_SCENARIOS)
190
191         elif side == "SELL": # SELL CLAIM
192
193             # portfolio that sells claim
194             P1 = PortfolioOptimizer(lob_data=
    ↳ vix_options_df.copy(), price_data=vix_prices_df.
    ↳ copy(),
195
    ↳ wealth = 0.0,
196
    ↳ w_hat = w_hat,
197
    ↳ init_claim =
    ↳ init_claim, claim_sign = 1, claim_strike = strike,
    ↳ risk_param =
198
    ↳ RISK_PARAM, mu = mu_xi_T, std = std_xi_T,
    ↳ no_scenarios =
    ↳ NO_SCENARIOS)
199             # Base portfolio
200             P2 = PortfolioOptimizer(lob_data=
    ↳ vix_options_df.copy(), price_data=vix_prices_df.
    ↳ copy(),
201
    ↳ wealth = 0.0,
202
    ↳ w_hat = w_hat,
203
    ↳ init_claim =
    ↳ init_claim, claim_sign = 1, claim_strike = None,
    ↳ risk_param =
204
    ↳ RISK_PARAM, mu = mu_xi_T, std = std_xi_T,
    ↳ no_scenarios =
    ↳ NO_SCENARIOS)
205             else:
206                 raise ValueError("Side given isn't valid")
207
208             P1.initialize_params()
209             P2.initialize_params()
210
211             P1.solve()
212             P2.solve()
213
214             indifference_px = w_hat*(P2.score() - P1.score())
215             return indifference_px
216
217 # --- Cell 10 -----
218 # Compute Indifference BUY/SELL prices for option
219 ↳ strikes
220 strikes = [5*i for i in range(11)]
221 buy_prices = []
222 sell_prices = []
223 for K in strikes:
224     buy_px = get_indifference_price(init_claim=0,
    ↳ strike=K, w_hat=1e5, side = "BUY")
225     buy_prices.append(-1*buy_px)
226     sell_px = get_indifference_price(init_claim=0,
    ↳ strike=K, w_hat=1e5, side = "SELL")
227     sell_prices.append(-1*sell_px)
228
229 # --- Cell 11 -----
230 # Plot buy-sell prices vs option strike
231 fig2 = go.Figure()
232 fig2.add_trace(go.Scatter(x=strikes, y=buy_prices, mode
    ↳ = 'lines', name='BUY'))
233 fig2.add_trace(go.Scatter(x=strikes, y=sell_prices,
    ↳ mode='lines', name='SELL'))
234 fig2.update_layout(
235     title="<b>Indifference price on Digital Option</b>"
    ↳ ,
236     title_x = 0.5,
237     xaxis_title="Digital Option Strike",
238     yaxis_title="Indifference Price",
239 )
240 fig2.show()
241
242 # --- Cell 12 -----
243 # Plot indifference price spread
244 spread = np.subtract(sell_prices, buy_prices)
245 fig3 = go.Figure()
246 fig3.add_trace(go.Scatter(x=strikes, y=spread, mode='
    ↳ lines', name='BUY'))
247 fig3.add_vline(x = median_vix_T, line_dash = "dash",
248 ↳ annotation_text="Med VIX(T) ",
249 ↳ annotation_position="bottom right")
250 fig3.update_layout(
251     title="<b>Indifference Price Buy-Sell Spread </b>"
    ↳ ,
252     title_x = 0.5,
253     xaxis_title="<b> Option Strike Price</b>",
254     yaxis_title="<b> Buy-Sell Spread </b>",
255 )
256 fig3.show()

```

```

256 # --- Cell 13 -----
257 P = portfolio_1.P
258 px = portfolio_1.get_vix_prices()
259 positions = portfolio_1.get_positions()
260 weights = portfolio_1.get_terminal_weights()
261 log_prices = portfolio_1.get_terminal_price_grid()
262 prices = portfolio_1.get_vix_prices()
263
264 # --- Cell 14 -----
265 fig4 = go.Figure()
266 fig4.add_trace(go.Scatter(x=log_prices, y=weights, mode=
    ↳ 'lines', name='BUY'))
267 fig4.update_layout(
268     title="Terminal Price Distribution",
269     xaxis_title="log VIX(T) prices",
270     yaxis_title="P.D.F weight",
271 )
272 fig4.show()
273
274 # --- Cell 15 -----
275 fig5 = go.Figure()
276 fig5.add_trace(go.Scatter(x=prices, y=weights, mode=
    ↳ 'lines', name='BUY'))
277 fig5.update_layout(
278     title="Terminal Price Distribution",
279     xaxis_title="log VIX(T) prices",
280     yaxis_title="P.D.F weight",
281 )
282 fig5.show()

```

A.2 ProjectTask_QRH.ipynb

```

1 # =====
2 # Chapter 4 -- Joint SPX-VIX law: calibration and
3 #   ↳ extended hedging
4 # Exported from ProjectTask_QRH.ipynb (code cells only;
5 #   ↳ outputs omitted).
6 # =====
7 # --- Cell 1 -----
8 import numpy as np
9 import pandas as pd
10 import cvxpy as cp
11 import statsmodels.api as sm
12 import seaborn as sns
13 import matplotlib.pyplot as plt
14 import plotly.graph_objs as go
15 import plotly.figure_factory as ff
16 from plotly.subplots import make_subplots
17 from datetime import datetime, date
18 from numpy.linalg import inv as np_inv
19 from scipy.interpolate import griddata,
20 #   ↳ PchipInterpolator
21 from scipy.stats import norm, gaussian_kde, linregress
22 #   ↳ , jarque_bera, skew,
23 #   ↳ kurtosis, probplot)
24 from pricing.lib.optimizer import PortfolioOptimizer
25 from pricing.lib.qrh_calibration import QRH_Calibration
26 from pricing.lib.qrh_optimizer import
27 #   ↳ PortfolioOptimizerQRH, process_snapshot_data
28 from pricing.lib.qrh_utils import (RunParams,
29 #   ↳ init_logging, KernelParams, mid,
30 #   ↳ get_fwd_and_discount
31 #   ↳ , implied_vols,
32 #   ↳ black_scholes_px,
33 #   ↳ impute_y0, simulate_qrh,
34 #   ↳ vix_at_T, objective
35 #   ↳ as qrh_objective,
36 #   ↳ rmse as qrh_rmse)
37 # --- Cell 2 -----
38 init_logging()
39 sns.set_style("whitegrid")
40 plt.rcParams['figure.figsize'] = (12, 8)
41 print(cp.installed_solvers())
42 # --- Cell 3 -----
43 # Read Data
44 vix_options_df = pd.read_csv("./Data/vix_options_data.
45 #   ↳ csv")
46 vix_prices_df = pd.read_csv("./Data/vix_prices.csv")
47 # Column Formatting
48 vix_prices_df.rename(columns = {"DATE" : "date", "OPEN"
49 #   ↳ : "px"}, inplace = True)
50 vix_options_df["sym"] = vix_options_df.
51 #   ↳ underlying_symbol.apply(lambda x: x[1:]).values
52 vix_options_df.rename(columns = {"bid_size_1545": "
53 #   ↳ bsize",
54 #   ↳ "ask_size_1545" : "asize",
55 #   ↳ "bid_1545": "bid",
56 #   ↳ "ask_1545": "ask"}, inplace =
57 #   ↳ True)
58 vix_options_df = vix_options_df[["quote_date", "
59 #   ↳ expiration", "sym", "option_type", "strike", "
60 #   ↳ bsize", "bid", "ask", "asize"]]
61 # --- Cell 4 -----
62 # Process the VIX prices data
63 vix_prices_df['date'] = pd.to_datetime(vix_prices_df['
64 #   ↳ date'])
65 vix_prices_df = vix_prices_df.sort_values(by='date')
66 vix_prices_df['VIX_log'] = np.log(vix_prices_df['px'])
67 vix_prices_df['delta_xi'] = vix_prices_df['VIX_log'].
68 #   ↳ diff()
69 vix_prices_df['xi_lagged'] = vix_prices_df['VIX_log'].
70 #   ↳ shift(1)
71 vix_prices_df = vix_prices_df.dropna()
72 # --- Cell 5 -----
73 # Performing OLS
74 regression = linregress(vix_prices_df['xi_lagged'],
75 #   ↳ vix_prices_df['delta_xi'])
76 beta_1 = regression.slope
77 beta_0 = regression.intercept
78 pred_values = beta_1*vix_prices_df.xi_lagged.values +
79 #   ↳ beta_0
80 obs_values = vix_prices_df.delta_xi.values
81 residuals = obs_values - pred_values
82 # Calculate parameters a, xi_bar, and sigma
83 a = -beta_1
84 xi_bar = beta_0 / a
85 sigma = np.std(residuals)
86 print(f"OU Model Parameters:")
87 print(f"a: {a:.4f} | xi_bar: {xi_bar:.4f} | sigma: {sigma:.4f}")
88 print(f"xi_bar: {xi_bar:.4f} | sigma: {sigma:.4f}")
89 print(f"a: {a:.4f} | xi_bar: {xi_bar:.4f} | sigma: {sigma:.4f}")
90 print(f"xi_bar: {xi_bar:.4f} | sigma: {sigma:.4f}")
91 # Calculate date to maturity
92 start_date = date(2024, 6, 21) # t = 0
93 end_date = date(2024, 7, 17) # t = T
94 days = pd.date_range(start = start_date, end = end_date
95 #   ↳ , freq = "B")
96 T = len(days)-1
97 # VIX value at initial date
98 VIX_0 = 13.20
99 xi_0 = np.log(VIX_0)
100 # Calculate the mean, median and std. dev. of xi_T
101 phi = 1 - a
102 mu_xi_T = xi_bar + (xi_0 - xi_bar)*(phi**T)
103 std_xi_T = np.sqrt(sigma**2 * (1 - phi**(2*T)) / (1 -
104 #   ↳ phi**2))
105 median_vix_T = np.exp(mu_xi_T)
106 print(f"Time to maturity (T) is {T} days.")
107 print(f"Mean and standard deviation of log-VIX at
108 #   ↳ maturity:")
109 print(f"xi_T: {xi_T:.4f} | sigma(xi_T): {sigma(xi_T):.4f}")
110 print(f"xi_T: {xi_T:.4f} | sigma(xi_T): {sigma(xi_T):.4f}")
111 print(f"mu_xi_T: {mu_xi_T:.4f} | std_xi_T: {std_xi_T:.4f}")
112 print(f"xi_T: {xi_T:.4f} | sigma(xi_T): {sigma(xi_T):.4f}")
113 print(f"Median of VIX at maturity: {median_vix_T:.6f}")
114 # --- Cell 5 -----
115 # Goodness of fit of the OU / AR(1) model fitted above
116 t_stat = regression.slope / regression.stderr
117 r2 = regression.rvalue ** 2
118 jb, jb_p = jarque_bera(residuals)
119 ac1_sq = np.corrcoef(residuals[1:] ** 2, residuals
120 #   ↳ [:-1] ** 2)[0, 1]
121 roll_sd = pd.Series(residuals).rolling(250).std().
122 #   ↳ dropna()
123 print(f"R^2: {r2:.4f}")
124 print(f"t-stat on a: {t_stat:.4f}")
125 print(f"half-life (days): {np.log(2)/a:.4f}")
126 print(f"residual skew: {skew(residuals):.4f}")
127 print(f"residual excess kurtosis: {kurtosis(
128 #   ↳ residuals):.4f}")
129 print(f"Jarque-Bera p-value: {jb_p:.4f}")
130 print(f"autocorr of resid^2 (lag 1): {ac1_sq
131 #   ↳ :.4f}")
132 print(f"rolling 250d sd, min - max: {roll_sd.min
133 #   ↳ ():.4f} - {roll_sd.max():.4f}")
134 print(f"fitted constant sigma: {sigma:.4f}")
135 probplot(residuals, dist="norm", plot=plt)
136 plt.title("QQ plot: OU residuals vs normal")
137 plt.show()
138 # --- Cell 6 -----

```



```

250 buy_prices = []
251 sell_prices = []
252 for K in strikes:
253     buy_px = get_indifference_price(init_claim=0,
254     ↪ strike=K, w_hat=1e5, side = "BUY")
255     buy_prices.append(-1*buy_px)
256     sell_px = get_indifference_price(init_claim=0,
257     ↪ strike=K, w_hat=1e5, side = "SELL")
258     sell_prices.append(-1*sell_px)
259
260 # --- Cell 12 -----
261 # Plot buy-sell prices vs option strike
262 fig2 = go.Figure()
263 fig2.add_trace(go.Scatter(x=strikes, y=buy_prices, mode
264     ↪ ='lines', name='BUY'))
265 fig2.add_trace(go.Scatter(x=strikes, y=sell_prices,
266     ↪ mode='lines', name='SELL'))
267 fig2.update_layout(
268     title="<b>Indifference price on Digital Option</b>"
269     ↪ ,
270     title_x = 0.5,
271     xaxis_title="Digital Option Strike",
272     yaxis_title="Indifference Price",
273 )
274 fig2.update_layout(template="plotly_white")
275 fig2.show()
276
277 # --- Cell 13 -----
278 # Plot indifference price spread
279 spread = np.subtract(sell_prices, buy_prices)
280 fig3 = go.Figure()
281 fig3.add_trace(go.Scatter(x=strikes, y=spread, mode='
282     ↪ lines', name='BUY'))
283 fig3.add_vline(x = median_vix_T, line_dash = "dash",
284     ↪ annotation_text="Med VIX(T) ",
285     ↪ annotation_position="bottom right")
286 fig3.update_layout(
287     title="<b>Indifference Price Buy-Sell Spread </b>",
288     title_x = 0.5,
289     xaxis_title="<b> Option Strike Price</b>",
290     yaxis_title="<b> Buy-Sell Spread </b>",
291 )
292 fig3.update_layout(template="plotly_white")
293 fig3.show()
294
295 # --- Cell 14 -----
296 P = portfolio_1.P
297 px = portfolio_1.get_vix_prices()
298 positions = portfolio_1.get_positions()
299 weights = portfolio_1.get_terminal_weights()
300 log_prices = portfolio_1.get_terminal_price_grid()
301 prices = portfolio_1.get_vix_prices()
302
303 # --- Cell 15 -----
304 fig4 = go.Figure()
305 fig4.add_trace(go.Scatter(x=log_prices, y=weights, mode
306     ↪ ='lines', name='BUY'))
307 fig4.update_layout(
308     title="Terminal Price Distribution",
309     xaxis_title="log VIX(T) prices",
310     yaxis_title="P.D.F weight",
311 )
312 fig4.update_layout(template="plotly_white")
313 fig4.show()
314
315 # --- Cell 16 -----
316 fig5 = go.Figure()
317 fig5.add_trace(go.Scatter(x=prices, y=weights, mode='
318     ↪ lines', name='BUY'))
319 fig5.update_layout(
320     title="Terminal Price Distribution",
321     xaxis_title="log VIX(T) prices",
322     yaxis_title="P.D.F weight",
323 )
324 fig5.update_layout(template="plotly_white")
325 fig5.show()
326
327 # Part 3: Quadratic Rough Heston
328
329 # --- Cell 17 -----
330 SNAPSHOT = " ./Data/spx_vix_snapshot.csv"
331 run_params = RunParams(data_path=SNAPSHOT, quote_dt="
332     ↪ 2024-06-21", expiry_dt="2024-07-17", n_paths=60
333     ↪ _000)
334 qrh = QRH_Calibration(run_params=run_params)
335 qrh.run()
336
337 # --- Cell 18 -----
338 # Tradables for the 17/07/2024 expiry. No moneyness or
339     ↪ otm filter: the feasible
340 # set in (P) is defined by the quoted sizes alone, so
341     ↪ the optimizer decides.
342 vix_only_df = process_snapshot_data(SNAPSHOT, expiry="
343     ↪ 2024-07-17", symbols=("VIX",))
344 spx_vix_df = process_snapshot_data(SNAPSHOT, expiry="
345     ↪ 2024-07-17")
346
347 # the VIX leg must be the identical contract set to
348     ↪ Part 2 for the comparison to mean anything
349 same = set(zip(vix_options_df.option_type,
350     ↪ vix_options_df.strike)) == \
351     ↪ set(zip(vix_only_df.option_type, vix_only_df.
352     ↪ strike))
353 print(f"Part 2 VIX tradables: {len(vix_options_df)}"
354     ↪ Part 3 VIX tradables: {len(vix_only_df)}"
355     ↪ f" identical set: {same}")
356 print(spx_vix_df.groupby("sym").agg(n=("strike", "size"
357     ↪ ), lo=("strike", "min"), hi=("strike", "max")))
358 spx_vix_df.head()
359
360 # --- Cell 19 -----
361 # Task 4 (QRH): solving optimization for lambda = 2 and
362     ↪ 20
363 NO_SCENARIOS_QRH = 5000
364 portfolio_1_qrh = PortfolioOptimizerQRH(lob_data=
365     ↪ spx_vix_df.copy(),
366     qrh_calibration = qrh,
367     wealth = 1e5,
368     w_hat = 1e5,
369     init_claim = 0.0,
370     claim_sign = 1,
371     claim_strike = None,
372     risk_param = 2,
373     no_scenarios =
374     ↪ NO_SCENARIOS_QRH)
375 portfolio_1_qrh.initialize_params()
376
377 portfolio_2_qrh = PortfolioOptimizerQRH(lob_data=
378     ↪ spx_vix_df.copy(),
379     qrh_calibration = qrh,
380     wealth = 1e5,
381     w_hat = 1e5,
382     init_claim = 0.0,
383     claim_sign = 1,
384     claim_strike = None,
385     risk_param = 20,
386     no_scenarios =
387     ↪ NO_SCENARIOS_QRH)
388 portfolio_2_qrh.initialize_params()
389 portfolio_1_qrh.solve()
390 portfolio_2_qrh.solve()
391
392 print(f"lambda=2 status={portfolio_1_qrh.PROBLEM.
393     ↪ status} score={portfolio_1_qrh.score():.5f}")
394 print(f"lambda=20 status={portfolio_2_qrh.PROBLEM.
395     ↪ status} score={portfolio_2_qrh.score():.5f}")
396
397 # --- Cell 20 -----
398 # Optimal positions under QRH, with the quoted depth
399     ↪ they trade against
400 for name, p in [("lambda = 2", portfolio_1_qrh), ("
401     ↪ lambda = 20", portfolio_2_qrh)]:
402     pos = pd.DataFrame({"sym": spx_vix_df.sym, "
403     ↪ option_type": spx_vix_df.option_type,
404     ↪ "strike": spx_vix_df.strike,
405     ↪ "bid": spx_vix_df.bid, "bsize":
406     ↪ spx_vix_df.bsize,
407     ↪ "ask": spx_vix_df.ask, "asize":
408     ↪ spx_vix_df.asize,
409     ↪ "position": p.get_positions()})
410 # x_plus is capped by asize (buying), x_minus by
411     ↪ bsize (selling), so the
412 # relevant depth depends on the sign of the
413     ↪ position
414 pos["depth"] = np.where(pos.position >= 0, pos.
415     ↪ asize, pos.bsize)

```

```

382 pos["pct_of_depth"] = 100 * np.abs(pos.position) /
    ↳ pos.depth
383
384 active = pos[np.abs(pos.position) > 1e-6].copy()
385 at_limit = (active.pct_of_depth > 99.9).sum()
386 print(f"{name}: {len(active)} active of {p,
    ↳ no_assets} cost Sx = {p.Sx.value:,.0f}"
387       f" at the size limit: {at_limit}")
388 print(active.round(2).to_string(index=False), "\n")
389
390 # --- Cell 21 -----
391 # Terminal wealth surface plot
392 fig4 = make_subplots(rows=1, cols=2,
393                      specs=[{"type": "surface"}, {"
    ↳ type": "surface"}]],
394                      subplot_titles=("lambda = 2", "
    ↳ lambda = 20"))
395
396 scene = dict(xaxis_title="VIX at maturity", yaxis_title
    ↳ ="SPX at maturity",
397              zaxis_title="Terminal wealth")
398
399 for col, (name, p) in enumerate([("lambda = 2",
    ↳ portfolio_1_qrh),
400                                ("lambda = 20",
    ↳ portfolio_2_qrh)], start=1):
401     vix, spx, wealth = p.get_vix_prices(), p.
    ↳ get_spx_prices(), p.get_terminal_wealth()
402
403     vi = np.linspace(np.percentile(vix, 1), np.
    ↳ percentile(vix, 99), 60)
404     si = np.linspace(np.percentile(spx, 1), np.
    ↳ percentile(spx, 99), 60)
405     VI, SI = np.meshgrid(vi, si)
406     Z = griddata((vix, spx), wealth, (VI, SI), method="
    ↳ linear")
407     fig4.add_trace(go.Surface(x=VI, y=SI, z=Z,
    ↳ colorscale="Viridis", opacity=0.9,
408                          showscale=(col == 2),
    ↳ colorbar=dict(title="Wealth")),
409                  row=1, col=col)
410     j = int(np.argmax(wealth))
411     inside = (vi[0] <= vix[j] <= vi[-1]) and (si[0] <=
    ↳ spx[j] <= si[-1])
412     fig4.add_trace(go.Scatter3d(x=[vix[j]], y=[spx[j]],
    ↳ z=[wealth[j]],
413                               mode="markers+text",
414                               marker=dict(size=6,
    ↳ color="red", symbol="diamond"),
415                               text=[f" max {wealth[j]
    ↳ :,.0f}"],
416                               textposition="top
    ↳ center", textfont=dict(size=11),
417                               showlegend=False), row
    ↳ =1, col=col)
418
419     print(f"{name}: max {wealth[j]:,.0f} at VIX {vix[j]
    ↳ :,.2f}, SPX {spx[j]:,.0f}"
420           f" | min {wealth.min():,.0f} | inside
    ↳ plotted box: {inside}")
421
422 fig4.update_layout(title="<b>QRH terminal wealth vs (
    ↳ VIX, SPX) at maturity</b>",
423                   title_x=0.5, height=850, scene=scene
    ↳ , scene2=scene)
424 fig4.update_layout(template="plotly_white")
425 fig4.show()
426
427 # --- Cell 22 -----
428 # Terminal Wealth Plot scattered
429 fig4b = make_subplots(rows=1, cols=2,
430                      specs=[{"type": "scene"}, {"type
    ↳ ": "scene"}]],
431                      subplot_titles=("lambda = 2", "
    ↳ lambda = 20"))
432 for col, p in enumerate([portfolio_1_qrh,
    ↳ portfolio_2_qrh], start=1):
433     vix, spx, wealth = p.get_vix_prices(), p.
    ↳ get_spx_prices(), p.get_terminal_wealth()
434     fig4b.add_trace(go.Scatter3d(x=vix, y=spx, z=wealth
    ↳ , mode="markers",
435                                marker=dict(size=1.6,
    ↳ color=wealth, colorscale="Viridis",

```

```

showscale
    ↳ =(col == 2)), showlegend=False),
    ↳ row=1, col=col)
437
438 fig4b.update_layout(title="<b>QRH terminal wealth,
    ↳ simulated paths</b>", title_x=0.5,
439                    height=650, scene=scene, scene2=
    ↳ scene)
440 fig4b.update_layout(template="plotly_white")
441 fig4b.show()
442
443 # --- Cell 23 -----
444 def get_indifference_price_qrh( init_claim, strike,
    ↳ w_hat, side, lob_data=None):
445     """
446     """
447     NO_SCENARIOS = 5000
448     RISK_PARAM = 20
449     spx_vix_df = lob_data if lob_data is not None else
    ↳ globals()["spx_vix_df"]
450     if side == "BUY": # BUY CLAIM
451         # Base portfolio
452         P1 = PortfolioOptimizerQRH(lob_data=spx_vix_df
    ↳ .copy(), qrh_calibration = qrh,
453                                wealth = 0.0,
    ↳ w_hat = w_hat,
454                                init_claim =
    ↳ init_claim, claim_sign = 1, claim_strike = None,
455                                risk_param =
    ↳ RISK_PARAM, no_scenarios = NO_SCENARIOS)
456         # Portfolio that buys claim
457         P2 = PortfolioOptimizerQRH(lob_data=spx_vix_df
    ↳ .copy(), qrh_calibration = qrh,
458                                wealth = 0.0,
    ↳ w_hat = w_hat,
459                                init_claim =
    ↳ init_claim, claim_sign = -1, claim_strike = strike
    ↳ ,
460                                risk_param =
    ↳ RISK_PARAM, no_scenarios = NO_SCENARIOS)
461     elif side == "SELL": # SELL CLAIM
462         # portfolio that sells claim
463         P1 = PortfolioOptimizerQRH(lob_data=spx_vix_df
    ↳ .copy(), qrh_calibration = qrh,
464                                wealth = 0.0,
    ↳ w_hat = w_hat,
465                                init_claim =
    ↳ init_claim, claim_sign = 1, claim_strike = strike,
466                                risk_param =
    ↳ RISK_PARAM, no_scenarios = NO_SCENARIOS)
467         # Base portfolio
468         P2 = PortfolioOptimizerQRH(lob_data=spx_vix_df
    ↳ .copy(), qrh_calibration = qrh,
469                                wealth = 0.0,
    ↳ w_hat = w_hat,
470                                init_claim =
    ↳ init_claim, claim_sign = 1, claim_strike = None,
471                                risk_param =
    ↳ RISK_PARAM, no_scenarios = NO_SCENARIOS)
472     else:
473         raise ValueError("Invalid Side!")
474
475     P1.initialize_params()
476     P2.initialize_params()
477
478     P1.solve()
479     P2.solve()
480
481     indifference_px = w_hat*(P2.score() - P1.score())
482     return indifference_px
483
484 # --- Cell 24 -----
485 # Indifference prices under QRH, for two different
    ↳ tradable universes
486
487 qrh_prices = {}
488 for label, lob in [("QRH, VIX only", vix_only_df), ("
    ↳ QRH, VIX+SPX", spx_vix_df)]:
489     buys, sells = [], []
490     for K in strikes:
491         buys.append(-1*get_indifference_price_qrh(
    ↳ init_claim=0, strike=K, w_hat=1e5,
492                                side=
    ↳ "BUY", lob_data=lob))

```

```

493     sells.append(-1*get_indifference_price_qrh(
494         ↪ init_claim=0, strike=K, w_hat=1e5,
495         ↪ side
496         ↪ ="SELL", lob_data=lob))
497     qrh_prices[label] = (buys, sells)
498     print(f"{label}: done")
499     buy_prices_qrh, sell_prices_qrh = qrh_prices["QRH, VIX+
500         ↪ SPX"]
501     # --- Cell 25 -----
502     fig5 = go.Figure()
503     fig5.add_trace(go.Scatter(x=strikes, y=buy_prices,
504         ↪ mode='lines', name='BUY - OU, VIX only', line=
505         ↪ dict(dash='dash'))))
506     fig5.add_trace(go.Scatter(x=strikes, y=sell_prices,
507         ↪ mode='lines', name='SELL - OU, VIX only', line=
508         ↪ dict(dash='dash'))))
509     for label, (b, sl) in qrh_prices.items():
510         fig5.add_trace(go.Scatter(x=strikes, y=b, mode='
511         ↪ lines', name=f'BUY - {label}'))
512         fig5.add_trace(go.Scatter(x=strikes, y=sl, mode='
513         ↪ lines', name=f'SELL - {label}'))
514     fig5.update_layout(
515         title="Indifference price on Digital Option: OU
516         ↪ vs QRH",
517         title_x = 0.5,
518         xaxis_title="Digital Option Strike",
519         yaxis_title="Indifference Price",
520     )
521     fig5.update_layout(template="plotly_white")
522     fig5.show()
523     # Buy-sell spread under both models
524     fig6 = go.Figure()
525     fig6.add_trace(go.Scatter(x=strikes, y=np.subtract(
526         ↪ sell_prices, buy_prices), mode='lines', line=
527         ↪ dict(dash='dash'))))
528     for label, (b, sl) in qrh_prices.items():
529         fig6.add_trace(go.Scatter(x=strikes, y=np.subtract(
530         ↪ sl, b), mode='lines', name=label))
531     fig6.update_layout(
532         title="Indifference Price Buy-Sell Spread: OU vs
533         ↪ QRH",
534         title_x = 0.5,
535         xaxis_title="Option Strike Price",
536         yaxis_title="Buy-Sell Spread",
537     )
538     fig6.update_layout(template="plotly_white", height=850,
539         ↪ width=1200)
540     fig6.show()
541     # --- Cell 26 -----
542     T_VIX = 26 / 365
543     # smooth the smile, reprice a dense call curve off it,
544     ↪ then differentiate twice
545     raw = pd.read_csv(SNAPSHOT)
546     chain = raw[(raw.underlying_symbol == "VIX") & (raw.
547         ↪ expiration == "2024-07-17")].copy()
548     chain["mid"] = mid(chain)
549     F_vix, D_vix = get_fwd_and_discount(chain)
550     otm = pd.concat([chain[(chain.option_type == "P") & (
551         ↪ chain.strike < F_vix)],
552         ↪ chain[(chain.option_type == "C") & (
553         ↪ chain.strike >= F_vix)]]).sort_values("strike")
554     otm["otm"] = (otm.bid_1545 >= 0.05) & (otm.strike <= 60)]
555     K_m = otm.strike.values.astype(float)
556     iv_m = implied_vols(otm["mid"].values / D_vix, F_vix,
557         ↪ K_m, T_VIX, otm.option_type.values == "C")
558     ok = np.isfinite(iv_m) & (iv_m > 0)
559     K_m, iv_m = K_m[ok], iv_m[ok]
560     grid = np.linspace(max(K_m.min(), 12), 45, 600)
561     calls = black_scholes_px(F_vix, grid, T_VIX,
562         ↪ PchipInterpolator(np.log(K_m /
563         ↪ F_vix), iv_m)(np.log(grid / F_vix)),
564         ↪ np.ones_like(grid, dtype=bool))
565     dens_mkt = np.maximum(np.gradient(np.gradient(calls,
566         ↪ grid), grid), 0)
567     def kde_positive(sample):
568         ↪ return gaussian_kde(np.log(sample))(np.log(grid)) /
569         ↪ grid
570     vix_obs = vix_prices_df.px.shift(-T)[(vix_prices_df.px
571         ↪ >= 12) & (vix_prices_df.px <= 15)].dropna().values
572     densities = {
573         "market (Q)": dens_mkt,
574         "QRH (Q)": kde_positive(portfolio_1_qrh.
575         ↪ get_vix_prices()),
576         "observed (P)": kde_positive(vix_obs),
577         "OU (P)": norm.pdf(np.log(grid), mu_xi_T,
578         ↪ std_xi_T) / grid,
579     }
580     densities = {k: v / np.trapezoid(v, grid) for k, v in
581         ↪ densities.items()} # common support
582     fig7 = go.Figure()
583     for lab, colour, dash in [("market (Q)", "black", "dash
584         ↪ ", ("QRH (Q)", "crimson", "solid"),
585         ↪ ("observed (P)", "grey", "dot
586         ↪ ", ("OU (P)", "royalblue", "solid"))]:
587         fig7.add_trace(go.Scatter(x=grid, y=densities[lab],
588         ↪ mode="lines", name=lab,
589         ↪ line=dict(color=colour,
590         ↪ dash=dash, width=2)))
591     fig7.add_vline(x=F_vix, line_dash="dot",
592         ↪ annotation_text="VIX future")
593     fig7.update_layout(title="VIX at maturity: market
594         ↪ implied, models, and realised history",
595         ↪ title_x=0.5, xaxis_title="VIX at
596         ↪ maturity", yaxis_title="density", height=450)
597     fig7.update_layout(template="plotly_white", height=850)
598     fig7.show()
599     print(f"all densities renormalised over VIX = {grid
600         ↪ [0]:.0f} to {grid[-1]:.0f}"
601         ↪ f" (VIX future F = {F_vix:.3f})")
602     print(f"{'':<14}{mean':>8}{median':>9}{sd':>8}{P
603         ↪ ('>20'):>10}{P(>30)':>10}")
604     for lab, d in densities.items():
605         m = np.trapezoid(d * grid, grid)
606         sdv = np.sqrt(np.trapezoid(d * (grid - m) ** 2,
607         ↪ grid))
608         cdf = np.cumsum(d) * np.gradient(grid)
609         print(f"{lab:<14}{m:>8.2f}{grid[np.searchsorted(cdf
610         ↪ , 0.5)]:>9.2f}{sdv:>8.2f}"
611         ↪ f"{100*np.trapezoid(d[grid>20], grid[grid
612         ↪ >20]):>9.2f}%"
613         ↪ f"{100*np.trapezoid(d[grid>30], grid[grid
614         ↪ >30]):>9.2f}%")
615     # --- Cell 27 -----
616     # One simulated (VIX, SPX) path under the calibrated
617     ↪ QRH model
618     N_PATHS, STEPS_PER_DAY, SEED, PATH_ID = 5, 8, 7, 0
619     days = (pd.Timestamp("2024-07-17") - qrh.quote_date).
620         ↪ days
621     n_steps = days * STEPS_PER_DAY
622     delta = (days / 365.0) / n_steps
623     params = qrh.calibrated.as_array()
624     kernel = KernelParams(*params[:3])
625     c = params[3]
626     grid = np.arange(n_steps + 1) * delta
627     y0_curve, _ = impute_y0(grid, qrh.fwd_var_fn, kernel, c
628         ↪ )
629     spx_by_step, shocks = simulate_qrh(kernel, c, y0_curve,
630         ↪ n_steps * delta, n_steps,
631         ↪ N_PATHS, SEED,
632         ↪ xi_target=qrh.
633         ↪ fwd_var_fn(grid))
634     spx_fwd = spx_vix_df.loc[spx_vix_df.sym == "SPX", "
635         ↪ forward"].iloc[0]
636     spx_path = spx_fwd * np.vstack([np.ones(N_PATHS)] + [
637         ↪ spx_by_step[n] for n in range(1, n_steps + 1)])
638     vix_at = np.unique(np.linspace(1, n_steps, 1000).
639         ↪ astype(int))
640     vix_path = np.full((n_steps + 1, N_PATHS), np.nan)
641     u = np.linspace(0, 30/365, 1000)

```

```

609 vix_path[0] = np.sqrt(np.trapezoid(qrh.fwd_var_fn(u), u
    ↪ ) / (30/365)) * 100
610 for n in vix_at:
611     vix_path[n], _ = vix_at_T(shocks[:,n], kernel, c, n
    ↪ * delta, n, qrh.fwd_var_fn)
612
613 t_days = grid * 365
614 fig8 = make_subplots(specs=[[{"secondary_y": True}]]
615 fig8.add_trace(go.Scatter(x=t_days, y=vix_path[:,
    ↪ PATH_ID], name="VIX",
616                         mode="lines+markers", marker=
    ↪ dict(size=3),
617                         connectgaps=True, line=dict(
    ↪ color="crimson")), secondary_y=False)
618 fig8.add_trace(go.Scatter(x=t_days, y=spx_path[:,
    ↪ PATH_ID], name="SPX",
619                         line=dict(color="royalblue"))
    ↪ , secondary_y=True)
620 fig8.update_layout(title="<b>Simulated QRH path: VIX
    ↪ and SPX</b>", title_x=0.5,
621                    xaxis_title="days from 21/06/2024",
    ↪ height=850)
622 fig8.update_yaxes(title_text="VIX", secondary_y=False)
623 fig8.update_yaxes(title_text="SPX", secondary_y=True)
624 fig8.update_layout(template="plotly_white")
625 fig8.show()
626
627 # --- Cell 28 -----
628 # Model vs market implied volatilities at the
    ↪ 17/07/2024 expiry: SPX (left), VIX (right)
629 fit = qrh.objective(qrh.calibrated.as_array(), qrh.
    ↪ obj_config, detail=True)
630
631 panels = [(qrh.spx_smile, fit["iv_spx"], f"SPX {
    ↪ run_params.expiry_dt}"),
632            (qrh.vix_smiles[0], fit["iv_vix"][0], f"VIX {
    ↪ qrh.vix_expiries[0]}")]
633
634 fig10 = make_subplots(rows=1, cols=2, subplot_titles=[f"
    ↪ {lab} ({sm.T*365:.0f}d)" for sm, _, lab in
    ↪ panels])
635 for col, (smile, model_iv, lab) in enumerate(panels,
    ↪ start=1):
636     k = np.log(smile.K / smile.F)
637     fig10.add_trace(go.Scatter(x=k, y=smile.iv * 100,
    ↪ mode="markers", name="market",
638                             legendgroup="mkt",
639                             marker=dict(color="black
    ↪ ", size=6)), row=1, col=col)
640     fig10.add_trace(go.Scatter(x=k, y=model_iv * 100,
    ↪ mode="lines", name="QRH",
641                             legendgroup="qrh",
642                             line=dict(color="crimson
    ↪ ", width=2)), row=1, col=col)
643     fig10.add_annotation(text=f"rmse {qrh_rmse(model_iv
    ↪ , smile.iv)*100:.2f}%",
644                           x=k.min(), y=smile.iv.max() *
    ↪ 100, showarrow=False,
645                           xanchor="left", font=dict(size
    ↪ =11, color="grey"),
    ↪ row=1, col=col)
646
647 fig10.update_xaxes(title_text="log(K / F)")
648 fig10.update_yaxes(title_text="implied vol (%)")
649 fig10.update_layout(title="<b>Model vs Market Implied
    ↪ Volatilities - 17/07/2024 expiry</b>",
650                    title_x=0.5, height=850, width
    ↪ =1200)
651 fig10.update_layout(template="plotly_white")
652 fig10.show()
653
654 print(f"H={qrh.calibrated.H:.4f} nu={qrh.calibrated.nu
    ↪ :.4f} "
655       f"lambda={qrh.calibrated.lmd:.4f} c={qrh.
    ↪ calibrated.c:.6f}")

```

A.3 optimizer.py

```

1 import numpy as np
2 import pandas as pd
3 import cvxpy as cp
4 from scipy.stats import norm
5
6 class PortfolioOptimizer:
7
8     def __init__(self, lob_data : pd.DataFrame,
9         ↳ price_data: pd.DataFrame, init_claim, claim_strike
10         ↳ ,
11             claim_sign, wealth, w_hat, risk_param,
12         ↳ mu, std, no_scenarios = 100):
13         """
14         Portfolio Optimizer
15         """
16         self.w = wealth
17         self.w_hat = w_hat
18         self._lambda = risk_param
19         self.lob_data = lob_data
20         self.no_assets = len(self.lob_data)
21         self.px_data = price_data
22         self.solver = 'MOSEK'
23         self.no_scenarios = no_scenarios
24         self.mu = mu # mean of log VIX prices
25         self.std = std # Standard deviation of log VIX
26         ↳ prices
27
28         # Claim details
29         self.claim_side = claim_sign
30         self.init_claim = init_claim
31         self.claim_strike = claim_strike
32
33         # Optimizer Variables
34         self.terminal_prices = None
35         self.terminal_weights = None
36         self.P = None # Payoff matrix
37         self.c = None # Claim value
38         self.x = None # Risky assets
39         self.x_minus = None # Buy Qty
40         self.x_plus = None # Sell Qty
41         self.Sx = None # Transaction Cost
42
43         # Optimizer Solver
44         self.PROBLEM = None
45         self.CONSTRAINT = None
46         self.OBJECTIVE = None
47
48     def initialize_params(self):
49
50         self.init_portfolio_data()
51         self.init_optimizer_variables()
52         self.define_optimizer()
53
54     def init_portfolio_data(self):
55         """
56         Defines portfolio data based on dataset
57         """
58         self.terminal_prices = self.
59         ↳ get_terminal_price_grid()
60         self.terminal_weights = self.
61         ↳ get_terminal_weights()
62         self.c = self.populate_claim()
63
64         # Populate payoff matrix
65         self.P = self.get_payoff_matrix()
66
67     def init_optimizer_variables(self):
68         """
69         """
70         # Decision variable
71         self.x_plus = cp.Variable(self.no_assets,
72         ↳ nonneg=True)
73         self.x_minus = cp.Variable(self.no_assets,
74         ↳ nonneg=True)
75         self.x = self.x_plus - self.x_minus
76         ask_px = self.lob_data.ask.values
77
78         bid_px = self.lob_data.bid.values
79         # Trading cost function
80         self.Sx = cp.sum(cp.multiply(ask_px, self.
81         ↳ x_plus) - cp.multiply(bid_px, self.x_minus))
82         # self.cashflow = self.c - self.P @ self.x
83
84     def define_optimizer(self):
85         # Defining objectives and constraints
86         W = self.c - (self.P @ self.x) - (self.w -
87         ↳ self.Sx)
88         log_weights = np.log(self.terminal_weights)
89         inner_expr = (self._lambda/self.w_hat)*W +
90         ↳ log_weights
91         self.OBJECTIVE = cp.Minimize((1/self._lambda)*
92         ↳ cp.log_sum_exp(inner_expr))
93         # Defining constraints
94         self.CONSTRAINT = []
95         self.CONSTRAINT.append( self.x == self.x_plus -
96         ↳ self.x_minus)
97         self.CONSTRAINT.append( self.x_minus <= self.
98         ↳ lob_data.bsize.values )
99         self.CONSTRAINT.append( self.x_plus <= self.
100         ↳ lob_data.asize.values)
101         # self.CONSTRAINT.append( self.Sx <= self
102         ↳ .w)
103         # Defining the optimization problem
104         self.PROBLEM = cp.Problem(self.OBJECTIVE, self.
105         ↳ CONSTRAINT)
106
107     def get_payoff_matrix(self):
108         """
109         Calculates the payoff matrix for all assets and
110         ↳ scenarios.
111         """
112         assets_df = self.lob_data[['option_type', '
113         ↳ strike', 'bid']]
114
115         # Initialize P matrix, cols are assets and rows
116         ↳ are scenarios
117         P = np.zeros((self.no_scenarios, self.no_assets
118         ↳ ))
119
120         # Populating P matrix
121         for j in range(self.no_assets):
122             asset = assets_df.iloc[j]
123             vix_at_maturity = np.exp(self.
124             ↳ terminal_prices)
125             if asset['option_type'] == 'C':
126                 P[:, j] = np.maximum(vix_at_maturity -
127                 ↳ asset['strike'], 0)
128             elif asset['option_type'] == 'P':
129                 P[:, j] = np.maximum(asset['strike'] -
130                 ↳ vix_at_maturity, 0)
131             else:
132                 raise ValueError("Invalid Option Type")
133
134         return P
135
136     def populate_claim(self):
137         """
138         Creates vector of claim values corresponding to
139         ↳ all scenarios
140         """
141         claim_values = np.zeros(self.no_scenarios)
142
143         # initial claim value
144         if self.init_claim:
145             claim_values += self.init_claim
146
147         # Add Barrier Option Payoff
148         if self.claim_strike is not None:
149             vix_prices = np.exp(self.terminal_prices)
150             indicator = (vix_prices > self.claim_strike
151             ↳ ).astype(int)
152             claim_payoff = 1000 * indicator
153
154         # The claim side can be +1 (buy) or -1 (
155         ↳ sell)

```

```

130         claim_values += self.claim_side *
↪ claim_payoff
131
132         return claim_values
133
134     def get_terminal_price_grid(self):
135         """
136         Construction of log VIX price grid for all
↪ scenarios
137         """
138         xi_0 = self.mu - 9 * self.std
139         delta_xi = (18 * self.std) / (self.no_scenarios
↪ - 1)
140         xi_T = [xi_0 + i * delta_xi for i in range(self
↪ .no_scenarios)]
141         return xi_T
142
143     def get_terminal_weights(self):
144         """
145         Evaluates the probability of all price scenario
146         """
147         delta_epsilon = (18 * self.std) / (self.
↪ no_scenarios - 1)
148         p_eps_M = norm.pdf(self.terminal_prices, loc=
↪ self.mu, scale=self.std)
149         p_M = p_eps_M * delta_epsilon
150
151         return p_M
152
153     def solve(self, isverbose= False, iswarm = False):
154         self.PROBLEM.solve(verbose=isverbose, solver=
↪ self.solver, warm_start=iswarm)
155
156     def score(self):
157         return self.PROBLEM.value
158
159     def get_terminal_wealth(self, without_cash = False)
↪ :
160         if without_cash:
161             return self.P @ self.x.value
162         if self.x.value is not None:
163             return (self.P @ self.x.value) + (self.w -
↪ self.Sx.value)
164         else:
165             raise print("Error: No Terminal Wealth")
166             return None
167
168     def get_vix_prices(self):
169         return np.exp(self.terminal_prices)
170
171     def get_positions(self):
172         return self.x.value

```

A.4 qrh_utils.py

```

1 from __future__ import annotations
2
3 import logging
4 from argparse import ArgumentParser
5 from dataclasses import dataclass
6 from typing import Optional, Sequence, Union
7
8 import numpy as np
9 import pandas as pd
10 from numpy.typing import NDArray
11 from scipy.integrate import quad
12 from scipy.interpolate import PchipInterpolator
13 from scipy.optimize import minimize
14 from scipy.special import gamma as _Gamma, gammainc,
15     ↪ ndtr
16
17 npf64 = np.float64
18
19 DELTA_VIX = 30.0 / 365.0
20 PENALTY = 1e3
21 BOUNDS = [(0.005, 0.45), (0.05, 3.00), (0.50, 20.0), (1
22     ↪ e-5, 0.030)]
23 PARAM_NAMES = ["H", "nu", "lambda", "c"]
24 INIT_PARAMS = np.array([0.0964, 0.7620, 6.256, 4.557e
25     ↪ -3])
26 PAPER_RANGE = {"H": (0.0233, 0.2908), "nu": (0.3676,
27     ↪ 1.2942),
28     "lambda": (5.000, 6.398), "c": (2.308e
29     ↪ -3, 18.977e-3)}
30
31 @dataclass
32 class Smile:
33     K: NDArray[np.float64]
34     is_call: NDArray[np.bool_]
35     iv: NDArray[np.float64]
36     vega: NDArray[np.float64]
37     F: float
38     T: float
39
40 @dataclass
41 class SmoothCurve:
42     T: NDArray[np.float64]
43     Z: NDArray[np.float64]
44     vars_fitted: NDArray[np.float64]
45     vars_raw: NDArray[np.float64]
46     fit_errs: NDArray[np.float64]
47     eps: float
48     cond: float
49     n_tab: int = 20_000
50
51     def __post_init__(self) -> None:
52         self._tab_u = np.linspace(0.0, float(self.T
53     ↪ [-1]), self.n_tab)
54         self._tab_xi = phi_deriv(self.T[None, :], self.
55     ↪ _tab_u[:, None]) @ self.Z
56         self._xi_end = float(self._tab_xi[-1])
57
58     def __call__(self, u: Union[float, NDArray[np.
59     ↪ float64]]) -> NDArray[np.float64]:
60         # scalar callers exist (xi_0(0.0)), so promote
61     ↪ before interpolating
62         u = np.atleast_1d(np.asarray(u, float))
63         xi = np.interp(u, self._tab_u, self._tab_xi,
64     ↪ left=float(self._tab_xi[0]),
65     ↪ right=self._xi_end)
66         return np.maximum(xi, 1e-8)
67
68 @dataclass
69 class ObjConfig:
70     spx: Smile
71     spx_step: int
72     vix: list[Smile]
73     vix_steps: list[int]
74     vix_labels: list[str]
75     xi0_fn: SmoothCurve
76
77     n_steps: int
78     delta: float
79     n_paths: int = 60_000
80     n_quad: int = 16
81     seed: int = 20240621
82     w_vix: float = 1.0
83     w_fut: float = 5.0
84
85     @property
86     def tau_max(self) -> float:
87         return self.n_steps * self.delta
88
89 @dataclass
90 class CalibratedParams:
91     H: float
92     nu: float
93     lmd: float
94     c: float
95     obj: float
96     kappa_l2_sq: float
97     v0: float
98     elapsed: float
99
100     def as_array(self) -> NDArray[np.float64]:
101         return np.array([self.H, self.nu, self.lmd,
102     ↪ self.c])
103
104     def bound_flags(self) -> dict[str, str]:
105         out = {}
106         for name, x, (lo, hi) in zip(PARAM_NAMES, self.
107     ↪ as_array(), BOUNDS):
108             span = hi - lo
109             out[name] = ("at lower bound - not
110     ↪ identified" if x <= lo + 0.01 * span
111     ↪ else "at upper bound - not
112     ↪ identified" if x >= hi - 0.01 * span
113     ↪ else "ok")
114         return out
115
116 @dataclass
117 class KernelParams:
118     H: float
119     nu: float
120     lam: float
121
122     def __post_init__(self) -> None:
123         self.H, self.nu, self.lam = float(self.H),
124     ↪ float(self.nu), float(self.lam)
125         self.alpha = self.H + 0.5
126         self.prefac = self.nu / _Gamma(self.alpha)
127         self.nu_hat = self.nu * np.sqrt(_Gamma(2.0 *
128     ↪ self.H)) / _Gamma(self.alpha)
129         self.r = self.nu_hat ** 2 / (2.0 * self.lam) **
130     ↪ (2.0 * self.H)
131
132     @property
133     def admissible(self) -> bool:
134         return self.r < 1.0
135
136     @property
137     def memory_days(self) -> float:
138         return 365.0 / self.lam
139
140     def __repr__(self) -> str:
141         return (f"KernelParams(H={self.H:.4f}, nu={self
142     ↪ .nu:.4f}, "
143     ↪ f"lam={self.lam:.4f}, ||kappa||^2={self
144     ↪ .r:.4f}, "
145     ↪ f"memory={self.memory_days:.0f}d)")
146
147 @dataclass(frozen=True)
148 class RunParams:
149     data_path: str
150     quote_dt: str
151     expiry_dt: str

```

```

138     n_paths: int
139
140     @classmethod
141     def add_args(cls, ap: ArgumentParser) -> None:
142         ap.add_argument("--data-path", default="..Data
↳ /spx_vix_snapshot.csv")
143         ap.add_argument("--quote-dt", default="
↳ 2024-06-21")
144         ap.add_argument("--expiry-dt", default="
↳ 2024-07-17")
145         ap.add_argument("--n-paths", type=int, default
↳ =60_000)
146
147     def init_logging() -> None:
148         handler = logging.StreamHandler()
149         handler.setFormatter(logging.Formatter("%(levelname
↳ )s %(message)s"))
150         logging.basicConfig(level=logging.INFO, handlers=[
↳ handler])
151
152
153
154     def black_scholes_px(F: float,
155                         K: NDArray[np.float64],
156                         T: float,
157                         sig: NDArray[np.float64],
158                         is_call: NDArray[np.bool_]) ->
↳ NDArray[np.float64]:
159         sq = np.sqrt(T)
160         with np.errstate(divide="ignore", invalid="ignore")
↳ :
161             d1 = np.log(F / K) / (sig * sq) + 0.5 * sig *
↳ sq
162             d2 = d1 - sig * sq
163             call = F * ndtr(d1) - K * ndtr(d2)
164             # put is computed from call via put call parity
165             return np.where(is_call, call, call - F + K)
166
167
168     def black_vega(F: float,
169                  K: NDArray[np.float64],
170                  T: float,
171                  sig: NDArray[np.float64]) -> NDArray[np.
↳ float64]:
172         sq = np.sqrt(T)
173         d1 = np.log(F / K) / (sig * sq) + 0.5 * sig * sq
174         return F * np.exp(-0.5 * d1 ** 2) / np.sqrt(2.0 *
↳ np.pi) * sq
175
176
177     def implied_vols(prices: NDArray[np.float64],
178                     F: float,
179                     K: NDArray[np.float64],
180                     T: float,
181                     is_call: NDArray[np.bool_],
182                     n_iter: int = 72,
183                     lb: float = 1e-4,
184                     ub: float = 8.0) -> NDArray[np.float64
↳ ]:
185         intrinsic = np.where(is_call, np.maximum(F - K,
↳ 0.0), np.maximum(K - F, 0.0))
186         upper = np.where(is_call, F, K)
187         valid = np.isfinite(prices) & (prices > intrinsic +
↳ 1e-12) & (prices < upper - 1e-12)
188
189         a, b = np.full(K.shape, lb), np.full(K.shape, ub)
190         for _ in range(n_iter):
191             m = 0.5 * (a + b)
192             too_low = black_scholes_px(F, K, T, m, is_call)
193             < prices
194             a = np.where(too_low, m, a)
195             b = np.where(too_low, b, m)
196             return np.where(valid, 0.5 * (a + b), np.nan)
197
198     def mid(d: pd.DataFrame) -> pd.Series:
199         return 0.5 * (d["bid_1545"] + d["ask_1545"])
200
201
202     def get_fwd_and_discount(quote_data: pd.DataFrame) ->
↳ tuple[float, float]:
203         pivot = quote_data.pivot_table(index="strike",
↳ columns="option_type",
204
205         values="mid",
206         aggfunc="first").dropna()
207         pivot = pivot[(pivot["C"] > 0) & (pivot["P"] > 0)]
208         if len(pivot) < 3:
209             raise ValueError("not enough two-sided strikes
↳ for put-call parity")
210         strikes = pivot.index.values.astype(float)
211         call_put_diff = (pivot["C"] - pivot["P"]).values
212         strike_near_atm = strikes[np.argmin(np.abs(
↳ call_put_diff))]
213         mask = (strikes > 0.75 * strike_near_atm) & (
↳ strikes < 1.35 * strike_near_atm)
214         if mask.sum() < 3:
215             mask = np.ones_like(strikes, dtype=bool)
216             # C - P = D*(F - K), so slope = -D and intercept =
↳ D*F
217             slope, intercept = np.polyfit(strikes[mask],
↳ call_put_diff[mask], 1)
218             discount = -slope
219             return float(intercept / discount), float(discount)
220
221     def var_swap_single(k: NDArray[np.float64],
222                        vol: NDArray[np.float64],
223                        T: float) -> tuple[float, dict]:
224         sigma = vol * np.sqrt(T)
225         d2 = -k / sigma - sigma / 2.0
226         y = ndtr(d2)
227
228         order = np.argsort(y)
229         y_s, w_s = y[order], sigma[order] ** 2
230
231         # pchip needs strictly increasing abscissae
232         keep = np.concatenate([(True), np.diff(y_s) > 1e
↳ -14]))
233         y_s, w_s = y_s[keep], w_s[keep]
234         if y_s.size < 3:
235             return np.nan, {}
236
237         # w(T) = integral_0^1 w_bs(y) dy, y = N(d2); tails
238         < held flat in variance
239         interior = quad(PchipInterpolator(y_s, w_s), y_s
↳ [0], y_s[-1], limit=200)[0]
240         left = w_s[0] * y_s[0]
241         right = w_s[-1] * (1.0 - y_s[-1])
242         total = interior + left + right
243         return total, dict(y_min=float(y_s[0]), y_max=float
↳ (y_s[-1]),
244                             tail_frac=float((left + right) /
↳ total))
245
246     def var_swap_curve(data: pd.DataFrame, quote_date: pd.
↳ Timestamp,
247                        symbol: str = "SPX", t_min: float =
↳ 3.0 / 365.0,
248                        min_bid: float = 0.05,
249                        max_rel_spread: float = 0.75) -> pd.DataFrame:
250         df_all = data[data.underlying_symbol == symbol].
↳ copy()
251         df_all["mid"] = mid(df_all)
252
253         rows = []
254         for exp, df in df_all.groupby("expiration"):
255             tau = float((pd.Timestamp(exp) - quote_date).
↳ days / 365.0)
256             if tau < t_min:
257                 continue
258             try:
259                 forward, discount = get_fwd_and_discount(df
↳ )
260             except Exception:
261                 continue
262
263             otm = pd.concat([
264                 df[(df.option_type == "P") & (df.strike <
↳ forward)],
265                 df[(df.option_type == "C") & (df.strike >=
↳ forward)],
266                 ].sort_values("strike")
267             otm = otm[(otm["bid_1545"] >= min_bid) & (otm["
↳ ask_1545"] > 0)]
268             otm = otm[(otm["ask_1545"] - otm["bid_1545"])

```

```

267     otm = otm[otm["implied_volatility_1545"] > 1e
268     if len(otm) < 8:
269         continue
270
271     strikes = otm["strike"].values.astype(float)
272     is_call = (otm["option_type"].values == "C")
273     log_moneyness = np.log(strikes / forward)
274     vol_mid = otm["implied_volatility_1545"].values
275     .astype(float)
276
277     # snapshot has one iv column (mid), so re-imply
278     bid/ask vols from prices
279     vol_bid = implied_vols(otm["bid_1545"].values /
280     discount, forward, strikes, tau, is_call)
281     vol_ask = implied_vols(otm["ask_1545"].values /
282     discount, forward, strikes, tau, is_call)
283     ok = np.isfinite(vol_bid) & np.isfinite(vol_ask)
284     vol_bid = np.where(ok, vol_bid, vol_mid)
285     vol_ask = np.where(ok, vol_ask, vol_mid)
286
287     w_mid, parts = var_swap_single(log_moneyness,
288     vol_mid, tau)
289     if not np.isfinite(w_mid):
290         continue
291     w_bid, _ = var_swap_single(log_moneyness,
292     vol_bid, tau)
293     w_ask, _ = var_swap_single(log_moneyness,
294     vol_ask, tau)
295
296     rows.append(dict(expiration=exp, tau=tau, days=
297     int(round(tau * 365)),
298     F=forward, D=discount,
299     n_options=len(otm), w_mid=w_mid,
300     vs_mid=w_mid / tau, vs_bid=
301     w_bid / tau,
302     vs_ask=w_ask / tau, **parts))
303
304     out = pd.DataFrame(rows).sort_values("tau").
305     reset_index(drop=True)
306     if out.empty:
307         raise RuntimeError(f"no usable {symbol}
308         expiries found")
309     out["vs_vol_mid"] = np.sqrt(np.maximum(out["vs_mid"]
310     ], 0))
311     out["quote_width_vol"] = 0.5 * (np.sqrt(np.maximum(
312     out["vs_ask"], 0))
313     - np.sqrt(np.
314     maximum(out["vs_bid"], 0)))
315     return out
316
317 def phi(tau: NDArray[np.float64], x: NDArray[np.float64]
318     ) -> NDArray[np.float64]:
319     m = np.minimum(x, tau)
320     return 1.0 - m ** 3 / 6.0 + x * tau * (2.0 + m) /
321     2.0
322
323 def phi_deriv(tau: NDArray[np.float64], x: NDArray[np.
324     float64]) -> NDArray[np.float64]:
325     m = np.minimum(x, tau)
326     return tau - m ** 2 / 2.0 + tau * m
327
328 def var_curve_smooth(expiries: NDArray[np.float64],
329     tot_vars: NDArray[np.float64],
330     eps: Union[float, NDArray[np.
331     float64]] = 0.03,
332     ridge: float = 1e-10) ->
333     SmoothCurve:
334     n = expiries.size
335     # eps may be one allowance for all expiries or one
336     per expiry
337     eps_vec = np.full(n, float(eps)) if np.isscalar(eps)
338     else eps
339
340     A = phi(expiries[None, :], expiries[:, None])
341     # regularize gram matrix so its invertible
342     A = A + ridge * np.trace(A) / n * np.eye(n)
343     cond = float(np.linalg.cond(A))
344
345     # perturbation (s+e)^2 T ~ w + 2 sqrt(w) @ sqrt(T)
346     scale = 2.0 * np.sqrt(tot_vars) * np.sqrt(expiries)
347
348     def obj_and_grad(err: NDArray[np.float64]) -> tuple
349     [float, NDArray[np.float64]]:
350         v = tot_vars + scale * err
351         z = np.linalg.solve(A, v)
352         return float(v @ z), 2.0 * z * scale
353
354     res = minimize(obj_and_grad, np.zeros(n), jac=True,
355     method="L-BFGS-B",
356     bounds=list(zip(-eps_vec, eps_vec)),
357     options=dict(maxiter=500, ftol=1e
358     -14))
359
360     fitted = tot_vars + scale * res.x
361     return SmoothCurve(T=expiries, Z=np.linalg.solve(A,
362     fitted),
363     vars_fitted=fitted, vars_raw=
364     tot_vars,
365     fit_errs=np.sqrt(fitted /
366     expiries) - np.sqrt(tot_vars / expiries),
367     eps=float(np.max(eps_vec)), cond
368     =cond)
369
370 def kappa(tau: Union[float, NDArray[np.float64]],
371     p: KernelParams) -> NDArray[np.float64]:
372     tau = np.asarray(tau, float)
373     out = np.zeros_like(tau)
374     m = tau > 0.0
375     out[m] = p.prefac * tau[m] ** (p.alpha - 1.0) * np.
376     exp(-p.lam * tau[m])
377     return out
378
379 def K00(tau: Union[float, NDArray[np.float64]],
380     p: KernelParams) -> NDArray[np.float64]:
381     tau = np.asarray(tau, float)
382     out = np.zeros_like(tau)
383     m = tau > 0.0
384     out[m] = p.r * gammainc(2.0 * p.H, 2.0 * p.lam *
385     tau[m])
386     return out
387
388 def K0_kappa(tau: Union[float, NDArray[np.float64]],
389     p: KernelParams) -> NDArray[np.float64]:
390     tau = np.asarray(tau, float)
391     out = np.zeros_like(tau)
392     m = tau > 0.0
393     out[m] = p.nu * p.lam ** (-p.alpha) * gammainc(p.
394     alpha, p.lam * tau[m])
395     return out
396
397 def K0_resolvent(tau: Union[float, NDArray[np.float64]
398     ],
399     p: KernelParams,
400     n_max: int = 512,
401     tol: float = 1e-14) -> NDArray[np.
402     float64]:
403     tau = np.asarray(tau, float)
404     out = np.zeros_like(tau)
405     m = tau > 0.0
406     if not m.any():
407         return out
408     x = 2.0 * p.lam * tau[m]
409     acc = np.zeros_like(x)
410     rn = 1.0
411     for n in range(1, n_max + 1):
412         rn *= p.r
413         if rn < tol:
414             break
415         term = rn * gammainc(2.0 * p.H * n, x)
416         acc += term
417         if term.max() < tol:
418             break
419     out[m] = acc
420     return out

```

```

396
397 def b_star(n_steps: int, delta: float, p: KernelParams)
398     ↪ -> NDArray[np.float64]:
399     edges = K00(np.arange(0, n_steps + 2, dtype=float)
400     ↪ * delta, p)
401     out = np.zeros(n_steps + 2)
402     out[1:] = np.sqrt(np.maximum(np.diff(edges), 0.0) /
403     ↪ delta)
404     return out
405
406 def a_coeff(u_vals: NDArray[np.float64],
407             n_steps: int,
408             delta: float,
409             p: KernelParams) -> NDArray[np.float64]:
410     k_edge = np.arange(0, n_steps + 1, dtype=float) *
411     ↪ delta
412     vals = K00(u_vals[:, None] - k_edge[None, :], p)
413     a_sq = (vals[:, :-1] - vals[:, 1:]) / delta
414     return np.sqrt(np.maximum(a_sq, 0.0)).T
415
416 def impute_y0(horizons: NDArray[np.float64],
417               xi0_fn: SmoothCurve,
418               p: KernelParams,
419               c: float,
420               n_cells: int = 240) -> tuple[NDArray[np.
421     ↪ float64], float]:
422     """
423     y0(u)^2 = xi0(u) - var[Y_u] - c, with var[Y_u] =
424     ↪ int_0^u xi0(u-v) kappa(v)^2 dv
425     """
426     cell_fractions = np.linspace(0.0, 1.0, n_cells + 1)
427     lag_edges = horizons[:, None] * cell_fractions[None
428     ↪ , :]
429     # exact kappa^2 mass per cell, not width x kappa^2(
430     ↪ mid): kappa^2 ~ v^(2H-1) is
431     # singular at v = 0 but its integral K00 is finite
432     ↪ and known in closed form
433     cell_kernel_mass = np.diff(K00(lag_edges, p), axis
434     ↪ =1)
435     lag_midpoints = 0.5 * (lag_edges[:, :-1] +
436     ↪ lag_edges[:, 1:])
437     fwd_variance_at_lag = xi0_fn((horizons[:, None] -
438     ↪ lag_midpoints).ravel())
439     y_variance = np.sum(cell_kernel_mass
440     ↪ * fwd_variance_at_lag.reshape(
441     ↪ lag_midpoints.shape), axis=1)
442
443     fwd_variance = xi0_fn(horizons)
444     y0_squared = fwd_variance - y_variance - c
445     # y0^2 < 0 means no real y0 reproduces the market
446     ↪ curve here
447     y0_violation = float(np.mean(np.maximum(-y0_squared
448     ↪ , 0.0))
449     ↪ / max(np.mean(fwd_variance), 1
450     ↪ e-12))
451     return np.sqrt(np.maximum(y0_squared, 0.0)),
452     ↪ y0_violation
453
454 def simulate_qrh(p: KernelParams,
455                 c: float,
456                 y0_curve: NDArray[np.float64],
457                 T: float,
458                 n_steps: int,
459                 n_paths: int,
460                 seed: int,
461                 record_steps: Sequence[int] = (),
462                 xi_target: Optional[NDArray[np.float64
463     ↪ ]] = None
464     ) -> tuple[dict[int, NDArray[np.
465     ↪ float64]], NDArray[np.float64]]:
466     delta = T / n_steps
467     theta = npf64(1.0 / (2.0 * p.H + 1.0))
468     c = npf64(c) # min variaince
469     one = npf64(1.0)
470     numerical_floor = npf64(1e-10)
471     bstar_wts = b_star(n_steps, delta, p)
472     # reversed copy the b_star weights
473     bstar_rev = np.ascontiguousarray(bstar_wts[::-1],
474     ↪ dtype=npf64)
475
476     n_weights = bstar_wts.size
477     kappa_integral = float(K0_kappa(delta, p))
478     kappa_sq_integral = float(K00(delta, p))
479     beta_ux = npf64(kappa_integral / delta)
480     # sigma_chi = sqrt(V_bar * delta) and sigma_eps =
481     ↪ sqrt(V_bar * [K00 - K0^2/delta]);
482     sqrt_delta = npf64(np.sqrt(delta))
483     resid_vol_unit = npf64(np.sqrt(kappa_sq_integral -
484     ↪ kappa_integral ** 2 / delta))
485
486     rng = np.random.default_rng(seed)
487     z_return = rng.standard_normal((n_steps, n_paths),
488     ↪ dtype=npf64)
489     z_kernel = rng.standard_normal((n_steps, n_paths),
490     ↪ dtype=npf64)
491
492     Xchi = np.zeros((n_steps, n_paths), npf64)
493     forward_variance = xi_target
494     X_n = np.zeros(n_paths, npf64)
495     y_prev = np.full(n_paths, y0_curve[0], npf64)
496     V_prev = y_prev ** 2 + c
497     spx_by_step, record_at = {}, set(int(s) for s in
498     ↪ record_steps)
499
500     for n in range(1, n_steps + 1):
501         if n == 1:
502             y_hat = np.full(n_paths, y0_curve[1], npf64
503             ↪ )
504         else:
505             # Y_n thats F_n-1 adapted
506             y_hat = y0_curve[n] + (bstar_rev[n_weights
507             ↪ - 1 - n: n_weights - 2] @ Xchi[:, n - 1])
508
509             # Conditional Variance
510             Vbar_n = (theta * y_hat ** 2 + (one - theta) *
511             ↪ y_prev ** 2 + c)
512             np.maximum(Vbar_n, numerical_floor, out=Vbar_n)
513             if forward_variance is not None:
514                 # Rescale to match the fwd variance
515                 Vbar_n *= npf64(min(max(forward_variance[n]
516             ↪ / max(float(Vbar_n.mean()), 1e-12), 0.5), 2.0))
517                 Vbar_n_sqrt = np.sqrt(Vbar_n, out=Vbar_n)
518
519             # Return Shock
520             Xchi_n = Vbar_n_sqrt * (sqrt_delta * z_return[n
521             ↪ - 1])
522             Y_n = y_hat + beta_ux * Xchi_n + Vbar_n_sqrt *
523             ↪ (resid_vol_unit * z_kernel[n - 1])
524             V_n = (Y_n ** 2) + c
525             X_n -= npf64(1/4) * delta * (V_n + V_prev) +
526             ↪ Xchi_n
527             Xchi[n - 1] = Xchi_n
528             y_prev, V_prev = Y_n, V_n
529             if n in record_at:
530                 spx_by_step[n] = np.exp(X_n)
531
532     return spx_by_step, Xchi
533
534 def vix_at_T(chi: NDArray[np.float64],
535              p: KernelParams,
536              c: float,
537              T: float,
538              n_steps: int,
539              xi0_fn: SmoothCurve,
540              n_quad: int = 16) -> tuple[NDArray[np.
541     ↪ float64], float]:
542     delta = T / n_steps
543     x, wq = np.polynomial.legendre.leggauss(n_quad)
544     u = 0.5 * DELTA_VIX * (x + 1.0)
545     wq = 0.5 * DELTA_VIX * wq
546
547     y0_fwd, y0_violation = impute_y0(T + u, xi0_fn, p,
548     ↪ c)
549     a = a_coeff(T + u, n_steps, delta, p)
550     y_fwd = y0_fwd[:, None] + a.T @ chi
551     integ = (y_fwd ** 2 + c) * (1.0 + K0_resolvent(
552     ↪ DELTA_VIX - u, p))[:, None]
553     return np.sqrt(np.maximum((wq @ integ) / DELTA_VIX,
554     ↪ 0.0)) * 100.0, y0_violation
555
556 def mc_option_prices(samples: NDArray[np.float64],

```

```

526         strikes: NDArray[np.float64],
527         is_call: NDArray[np.bool_]) ->
    ↪ NDArray[np.float64]:
528     # calc every strike at once  $E[(S-K)^+]$  = (sum of the
    ↪ block above  $K - K * n\_samples$ ) /  $n\_paths$ 
529     sorted_samples = np.sort(samples)
530     n_paths = sorted_samples.size
531     running_sum = np.concatenate([[0.0], np.cumsum(
    ↪ sorted_samples)])
532     n_below = np.searchsorted(sorted_samples, strikes,
    ↪ side="right")
533     call_prices = (running_sum[n_paths] - running_sum[
    ↪ n_below]
534                    - strikes * (n_paths - n_below)) /
    ↪ n_paths
535     put_prices = (strikes * n_below - running_sum[
    ↪ n_below]) / n_paths
536     return np.where(is_call, call_prices, put_prices)
537
538
539 def mc_implied_vols(samples: NDArray[np.float64],
540                    smile: Smile) -> tuple[NDArray[np.
    ↪ float64], float]:
541     # taking sample mean as the forward makes call -
    ↪ put = mean(S) - K hold
542     model_forward = float(np.mean(samples))
543     prices = mc_option_prices(samples, smile.K, smile.
    ↪ is_call)
544     model_iv = implied_vols(prices, model_forward,
    ↪ smile.K, smile.T, smile.is_call)
545     return model_iv, model_forward
546
547
548 # -----
    ↪ objective
549
550 def vega_weighted_mse(model_iv: NDArray[np.float64],
551                    smile: Smile) -> Optional[float]:
552     # mean squared model-vs-market iv error over one
    ↪ smile, weighted by vega
553     priced_ok = np.isfinite(model_iv)
554     if priced_ok.sum() < 4:
555         return None
556     vega_weights = smile.vega[priced_ok] / smile.vega[
    ↪ priced_ok].sum()
557     iv_errors = model_iv[priced_ok] - smile.iv[
    ↪ priced_ok]
558     return float(np.sum(vega_weights * iv_errors ** 2))
559
560
561 def simulate_spx_and_vix(params: NDArray[np.float64],
562                        cfg: ObjConfig
563                        ) -> tuple[NDArray[np.float64]
    ↪ ], list[NDArray[np.float64]], float]:
564     kernel = KernelParams(*params[:3])
565     min_variance = params[3]
566     time_grid = np.arange(cfg.n_steps + 1) * cfg.delta
567     y0_curve, grid_violation = impute_y0(time_grid, cfg
    ↪ .xi0_fn, kernel, min_variance)
568
569     spx_by_step, return_shocks = simulate_qrh(
570         kernel, min_variance, y0_curve, cfg.tau_max,
    ↪ cfg.n_steps, cfg.n_paths,
571         cfg.seed, record_steps=[cfg.spx_step],
    ↪ xi_target=cfg.xi0_fn(time_grid))
572
573     vix_samples, y0_violations = [], [grid_violation]
574     for vix_step in cfg.vix_steps:
575         samples, fwd_violation = vix_at_T(return_shocks
    ↪ [:vix_step], kernel, min_variance,
576                                         vix_step *
    ↪ cfg.delta, vix_step,
577                                         cfg.xi0_fn,
    ↪ cfg.n_quad)
578         vix_samples.append(samples)
579         y0_violations.append(fwd_violation)
580
581     return spx_by_step[cfg.spx_step], vix_samples,
    ↪ float(np.sum(y0_violations))
582
583
584 def objective(params: NDArray[np.float64],
585             cfg: ObjConfig,
586             detail: bool = False) -> Union[float,
    ↪ dict]:
587     kernel = KernelParams(*params[:3])
588     if not kernel.admissible:
589         return PENALTY * (1.0 + kernel.r)
590     try:
591         spx_samples, vix_samples, y0_violation =
    ↪ simulate_spx_and_vix(params, cfg)
592
593         spx_model_iv, spx_model_fwd = mc_implied_vols(
    ↪ spx_samples, cfg.spx)
594         spx_error = vega_weighted_mse(spx_model_iv, cfg
    ↪ .spx)
595         if spx_error is None:
596             return PENALTY
597
598         vix_errors, vix_model_ivs, vix_model_fwds,
    ↪ future_errors = [], [], [], []
599         for smile, samples in zip(cfg.vix, vix_samples)
    ↪ :
600             model_iv, model_fwd = mc_implied_vols(
    ↪ samples, smile)
601             smile_error = vega_weighted_mse(model_iv,
    ↪ smile)
602             if smile_error is None:
603                 return PENALTY
604             vix_errors.append(smile_error)
605             vix_model_ivs.append(model_iv)
606             vix_model_fwds.append(model_fwd)
607             future_errors.append((model_fwd / smile.F -
    ↪ 1.0) ** 2)
608
609             vix_error = float(np.mean(vix_errors))
610             future_error = float(np.mean(future_errors))
611             y0_penalty = 10.0 * y0_violation
612             total_error = (spx_error + cfg.w_vix *
    ↪ vix_error
613                          + cfg.w_fut * future_error +
    ↪ y0_penalty)
614
615             if detail:
616                 return dict(F=total_error, r_spx=spx_error,
    ↪ r_vix=vix_error,
617                          r_fut=future_error, pen=
    ↪ y0_penalty,
618                          iv_spx=spx_model_iv, iv_vix=
    ↪ vix_model_ivs,
619                          fut_vix=vix_model_fwds, F_spx=
    ↪ spx_model_fwd,
620                          S_spx=spx_samples, vix_sims=
    ↪ vix_samples)
621             return float(total_error)
622     except Exception:
623         return PENALTY
624
625
626 def model_vix_spot(params: NDArray[np.float64], cfg:
    ↪ ObjConfig) -> float:
627     no_shocks = np.zeros((1, 1))
628     vix, _ = vix_at_T(no_shocks, KernelParams(*params
    ↪ [:3]), params[3],
629                      1e-8, 1, cfg.xi0_fn, n_quad=48)
630     return float(vix[0])
631
632
633 def rmse(model_iv: NDArray[np.float64], mkt_iv: NDArray
    ↪ [np.float64]) -> float:
634     m = np.isfinite(model_iv)
635     return float(np.sqrt(np.mean((model_iv[m] - mkt_iv[
    ↪ m]) ** 2)))

```

A.5 qrh_calibration.py

```

1 from __future__ import annotations
2
3 import logging
4 import time
5 from argparse import ArgumentParser
6 from typing import Optional
7
8 import numpy as np
9 import pandas as pd
10 from scipy.optimize import differential_evolution,
11     minimize
12
13 from pricing_lib.qrh_utils import (
14     BOUNDS, INIT_PARAMS, PARAM_NAMES, PAPER_RANGE,
15     CalibratedParams, KernelParams, ObjConfig,
16     RunParams, Smile,
17     black_vega, get_fwd_and_discount, implied_vols,
18     init_logging,
19     mid, model_vix_spot, objective, rmse,
20     var_curve_smooth, var_swap_curve,
21 )
22
23 N_VIX = 4
24 MIN_OPTIONS = 8
25 N_STEPS = 1 # stage 1, differential
26     evolution
27 N_PATHS = 15_000
28 N_STEPS_POLISH = 2 # stage 2; polish path count
29     comes from --n-paths
30 MAX_ITER = 30
31 POP_SIZE = 10
32 POLISH_ITER = 50
33 EPS_BOUNDS = (0.002, 0.03)
34 MARKET_VIX = 13.20 # VIX index at 15:45 on the
35     quote date, from the snapshot
36
37 class QRH_Calibration:
38
39     def __init__(self, run_params: RunParams) -> None:
40         self.run_params = run_params
41         self.quote_date = pd.Timestamp(run_params.
42             quote_dt)
43         self.data: pd.DataFrame = pd.DataFrame()
44         self.var_swaps: Optional[pd.DataFrame] = None
45         self.fwd_var_fn = None
46         self.vix_expiries: list[str] = []
47         self.spx_smile: Optional[Smile] = None
48         self.vix_smiles: list[Smile] = []
49         self.obj_config: Optional[ObjConfig] = None
50         self.calibrated: Optional[CalibratedParams] =
51             None
52
53     def build_fwd_var_curve(self) -> None:
54         vs = var_swap_curve(self.data, self.quote_date,
55             "SPX")
56         # error margin for each expiry from its own re-
57         # implied bid/ask width
58         eps = np.clip(vs["quote_width_vol"].values, *
59             EPS_BOUNDS)
60         self.fwd_var_fn = var_curve_smooth(vs.tau.
61             values, vs.w_mid.values, eps=eps)
62         self.var_swaps = vs
63         v0 = float(self.fwd_var_fn(0.0)[0])
64         logging.info(f"fwd var curve: {len(vs)} knots,
65             cond={self.fwd_var_fn.cond:.2e}, "
66             f"V_0={v0:.6f} ({np.sqrt(v0) *
67             100:.2f}% vol)")
68
69     def build_smile(self, symbol: str, expiry: str,
70         k_lo: float, k_hi: float,
71         min_bid: float = 0.05, max_spread:
72         float = 0.60) -> Smile:
73         tau = float((pd.Timestamp(expiry) - self.
74             quote_date).days / 365.0)
75         quotes = self.data[(self.data.underlying_symbol
76             == symbol)
77             & (self.data.expiration ==
78             expiry)].copy()
79         quotes["mid"] = mid(quotes)
80         forward, discount = get_fwd_and_discount(quotes)
81
82         # Filter for valid OTM quotes
83         otm = pd.concat([
84             quotes[(quotes.option_type == "P") & (
85             quotes.strike < forward)],
86             quotes[(quotes.option_type == "C") & (
87             quotes.strike >= forward)],
88             otm.sort_values("strike")
89             otm = otm[(otm["bid_1545"] >= min_bid) & (otm["
90             ask_1545"] > 0)]
91             otm = otm[(otm["ask_1545"] - otm["bid_1545"])
92             / otm["mid"] < max_spread]
93             otm = otm[(otm["strike"] >= k_lo * forward) & (
94             otm["strike"] <= k_hi * forward)]
95
96         strikes = otm["strike"].values.astype(float)
97         is_call = (otm["option_type"].values == "C")
98         iv = implied_vols(otm["mid"].values / discount,
99             forward, strikes, tau, is_call)
100         valid = np.isfinite(iv) & (iv > 1e-3)
101         strikes, is_call, iv = strikes[valid], is_call[
102             valid], iv[valid]
103
104         # Normalise with forward price
105         if symbol == "SPX":
106             strikes, forward = strikes / forward, 1.0
107             return Smile(K=strikes, is_call=is_call, iv=iv,
108                 vega=black_vega(forward, strikes,
109                 tau, iv), F=forward, T=tau)
110
111     def build_smiles(self) -> None:
112         self.spx_smile = self.build_smile("SPX", self.
113             run_params.expiry_dt, 0.85, 1.10)
114         # check vix expiries and keep the first N_VIX
115         # with a usable smile.
116         self.vix_smiles, self.vix_expiries = [], []
117         for expiry in sorted(self.data[self.data.
118             underlying_symbol == "VIX"].expiration.unique()):
119             if pd.Timestamp(expiry) <= self.quote_date:
120                 continue
121             smile = self.build_smile("VIX", expiry,
122                 0.60, 2.50)
123             if len(smile.K) >= MIN_OPTIONS:
124                 self.vix_smiles.append(smile)
125                 self.vix_expiries.append(expiry)
126                 if len(self.vix_expiries) == N_VIX:
127                     break
128             logging.info(f"vix expiries: {'', '.join(self.
129                 vix_expiries)}")
130
131     def build_objective_config(self) -> None:
132         self.obj_config = self.make_config(
133             N_STEPS_POLISH, self.run_params.n_paths)
134
135     def make_config(self, steps_per_day: int, n_paths:
136         int) -> ObjConfig:
137         def days(e):
138             return (pd.Timestamp(e) - self.quote_date).
139                 days
140
141         spx_expiry = self.run_params.expiry_dt
142         max_days = max(days(e) for e in [spx_expiry] +
143             self.vix_expiries)
144         n_steps = max_days * steps_per_day
145         return ObjConfig(
146             spx=self.spx_smile, spx_step=days(
147             spx_expiry) * steps_per_day,
148             vix=self.vix_smiles,
149             vix_steps=[days(e) * steps_per_day for e in
150             self.vix_expiries],
151             vix_labels=self.vix_expiries, xi0_fn=self.
152             fwd_var_fn,
153             n_steps=n_steps, delta=(max_days / 365.0) /
154             n_steps, n_paths=n_paths)

```

```

115
116     def calibrate(self) -> CalibratedParams:
117         start = time.time()
118
119         coarse = self.make_config(N_STEPS, N_PATHS)
120         rng = np.random.default_rng(7)
121         init = rng.uniform([b[0] for b in BOUNDS], [b
↪ [1] for b in BOUNDS],
122                             size=(POP_SIZE * len(BOUNDS)
↪ , len(BOUNDS)))
123         init[0] = np.clip(INIT_PARAMS, [b[0] for b in
↪ BOUNDS], [b[1] for b in BOUNDS])
124
125         logging.info(f"stage 1 differential evolution:
↪ {coarse.n_steps} steps, "
126                     f"{coarse.n_paths:,} paths")
127         de = differential_evolution(
128             objective, bounds=BOUNDS, args=(coarse,),
↪ init=init,
129             maxiter=MAX_ITER, tol=1e-6, mutation=(0.4,
↪ 1.0), recombination=0.75,
130             seed=7, polish=False)
131
132         logging.info(f"stage 2 polish: {self.obj_config
↪ .n_steps} steps, "
133                     f"{self.obj_config.n_paths:,}
↪ paths (from F={de.fun:6f})")
134         polish = minimize(objective, de.x, args=(self.
↪ obj_config,),
135                             method="Nelder-Mead", bounds=
↪ BOUNDS,
136                             options=dict(maxiter=
↪ POLISH_ITER, xatol=1e-4, fatol=1e-10))
137
138         x, fun = (polish.x, polish.fun) if polish.fun <
↪ de.fun else (de.x, de.fun)
139         p = KernelParams(*x[:3])
140         self.calibrated = CalibratedParams(
141             H=float(x[0]), nu=float(x[1]), lmd=float(x
↪ [2]), c=float(x[3]),
142             obj=float(fun), kappa_l2_sq=p.r, v0=float(
↪ self.fwd_var_fn(0.0)[0]),
143             elapsed=time.time() - start)
144         return self.calibrated
145
146     def summary(self) -> dict:
147         cal = self.calibrated
148         cfg = self.obj_config
149         detail = objective(cal.as_array(), cfg, detail=
↪ True)
150         flags = cal.bound_flags()
151
152         lines = ["=" * 60]
153         for name, val in zip(PARAM_NAMES, cal.as_array
↪ ()):
154             lo, hi = PAPER_RANGE[name]
155             note = "" if flags[name] == "ok" else f"
↪ <<< {flags[name]}"
156             lines.append(f" {name:<8s}{val:>11.6f} (
↪ paper {lo:.4f} - {hi:.4f}){note}")
157             lines += [
158                 f" {'|k||^2':<8s}{cal.kappa_l2_sq:>11.6f}
↪ ",
159                 f" {'V_0':<8s}{cal.v0:>11.6f} {np.sqrt(
↪ cal.v0) * 100:.2f}% vol",
160                 f" {'VIX_0':<8s}{model_vix_spot(cal.
↪ as_array(), cfg):>11.2f} ",
161                 f"vs {MARKET_VIX:.2f} market",
162                 f" {'F':<8s}{cal.obj:>11.3e} spx {detail
↪ ['r_spx']:.1e} | "
163                 f"vix {detail['r_vix']:.1e} | fut {detail['
↪ r_fut']:.1e} | pen {detail['pen']:.1e}",
164                 "-" * 60,
165                 f" SPX {cfg.spx.T * 365:>4.0f}d rmse {
↪ rmse(detail['iv_spx'], cfg.spx.iv) * 100:5.2f}%"
166                 f" ({len(cfg.spx.K):3d} opts)",
167                 ]
168             for lab, sm, iv, fm in zip(self.vix_expiries,
↪ cfg.vix,
169                                     detail["iv_vix"],
170                                     detail["fut_vix"]):
171                 lines.append(f" VIX {sm.T * 365:>4.0f}d
↪ rmse {rmse(iv, sm.iv) * 100:5.2f}%"
172                             f" ({len(sm.K):3d} opts)
↪ future {fm:6.3f} vs {sm.F:6.3f} {lab}")
173             lines += [f" {cal.elapsed:.1f}s", "-" * 60]
174             logging.info("\n".join(lines))
175             return detail
176
177     def run(self) -> CalibratedParams:
178         self.data = pd.read_csv(self.run_params.
↪ data_path)
179         self.build_fwd_var_curve()
180         self.build_smiles()
181         self.build_objective_config()
182         self.calibrate()
183         self.summary()
184         return self.calibrated
185
186 if __name__ == "__main__":
187     init_logging()
188     parser = ArgumentParser()
189     RunParams.add_args(parser)
190     qrh = QRH_Calibration(
191         RunParams(**vars(parser.parse_args())))
192
193     qrh.run()
194
195     print(qrh.calibrated)

```

A.6 qrh_optimizer.py

```

1 import numpy as np
2 import pandas as pd
3 import cvxpy as cp
4
5 from pricing_lib.qrh_utils import ObjConfig,
    ↳ get_fwd_and_discount, mid, simulate_spx_and_vix
6
7
8 class PortfolioOptimizerQRH:
9
10     def __init__(self, lob_data: pd.DataFrame,
11     ↳ qrh_calibration, init_claim, claim_strike,
12     ↳ claim_sign, wealth, w_hat, risk_param,
13     ↳ expiry="2024-07-17",
14     ↳ no_scenarios=5000, steps_per_day=2,
15     ↳ seed=20240621):
16         """
17         Portfolio Optimizer under the calibrated
18     ↳ Quadratic Rough Heston model,
19         scenarios come from Monte Carlo.
20         """
21         self.w = wealth
22         self.w_hat = w_hat
23         self._lambda = risk_param
24         self.lob_data = lob_data
25         self.no_assets = len(self.lob_data)
26         self.qrh = qrh_calibration
27         self.solver = 'MOSEK'
28         self.no_scenarios = no_scenarios
29
30         # Simulation details
31         self.expiry = expiry
32         self.steps_per_day = steps_per_day
33         self.seed = seed
34         self.spx_forward = None
35
36         # Claim details
37         self.claim_side = claim_sign
38         self.init_claim = init_claim
39         self.claim_strike = claim_strike
40
41         # Optimizer Variables
42         self.spx_prices = None
43         self.vix_prices = None
44         self.terminal_weights = None
45         self.P = None # Payoff matrix
46         self.c = None # Claim value
47         self.x = None # Risky assets
48         self.x_minus = None # Buy Qty
49         self.x_plus = None # Sell Qty
50         self.Sx = None # Transaction Cost
51
52         # Optimizer Solver
53         self.PROBLEM = None
54         self.CONSTRAINT = None
55         self.OBJECTIVE = None
56
57     def initialize_params(self):
58
59         self.init_portfolio_data()
60         self.init_optimizer_variables()
61         self.define_optimizer()
62
63     def init_portfolio_data(self):
64         self.spx_prices, self.vix_prices = self.
    ↳ get_terminal_price_grid()
65         self.terminal_weights = self.
    ↳ get_terminal_weights()
66         self.c = self.populate_claim()
67
68         # Populate payoff matrix
69         self.P = self.get_payoff_matrix()
70
71     def init_optimizer_variables(self):
72         # Decision variable
73         self.x_plus = cp.Variable(self.no_assets,
    ↳ nonneg=True)
74         self.x_minus = cp.Variable(self.no_assets,
    ↳ nonneg=True)
75         self.x = self.x_plus - self.x_minus
76
77         ask_px = self.lob_data.ask.values
78         bid_px = self.lob_data.bid.values
79         # cost function
80         self.Sx = cp.sum(cp.multiply(ask_px, self.
    ↳ x_plus) - cp.multiply(bid_px, self.x_minus))
81
82     def define_optimizer(self):
83         # Defining objectives and constraints
84         W = self.c - (self.P @ self.x) - (self.w - self
    ↳ .Sx)
85         log_weights = np.log(self.terminal_weights)
86         inner_expr = (self._lambda / self.w_hat) * W +
    ↳ log_weights
87         self.OBJECTIVE = cp.Minimize((1 / self._lambda)
    ↳ * cp.log_sum_exp(inner_expr))
88         # Defining constraints
89         self.CONSTRAINT = []
90         self.CONSTRAINT.append( self.Sx <= self.w )
91         self.CONSTRAINT.append( self.x == self.x_plus -
    ↳ self.x_minus)
92         self.CONSTRAINT.append( self.x_minus <= self.
    ↳ lob_data.bsize.values )
93         self.CONSTRAINT.append( self.x_plus <= self.
    ↳ lob_data.asize.values)
94         # Defining the optimization problem
95         self.PROBLEM = cp.Problem(self.OBJECTIVE, self.
    ↳ CONSTRAINT)
96
97     def get_payoff_matrix(self):
98         assets_df = self.lob_data[['sym', 'option_type
    ↳ ', 'strike']]
99
100         # Initialize P matrix, cols are assets and rows
    ↳ are scenarios
101         P = np.zeros((self.no_scenarios, self.no_assets
    ↳ ))
102
103         # Populating P matrix
104         for j in range(self.no_assets):
105             asset = assets_df.iloc[j]
106             underlying = self.spx_prices if asset['sym
    ↳ ']' == 'SPX' else self.vix_prices
107             if asset['option_type'] == 'C':
108                 P[:, j] = np.maximum(underlying - asset
    ↳ ['strike'], 0)
109             elif asset['option_type'] == 'P':
110                 P[:, j] = np.maximum(asset['strike'] -
    ↳ underlying, 0)
111             else:
112                 raise ValueError("Invalid Option Type")
113
114         return P
115
116     def populate_claim(self):
117         """
118         Creates vector of claim values corresponding to
119     ↳ all scenarios
120         """
121         claim_values = np.zeros(self.no_scenarios)
122
123         # initial claim value
124         if self.init_claim:
125             claim_values += self.init_claim
126
127         # Add Barrier Option Payoff
128         if self.claim_strike is not None:
129             indicator = (self.vix_prices > self.
    ↳ claim_strike).astype(int)
130             claim_payoff = 1000 * indicator
131
132         # The claim side can be +1 (buy) or -1 (
    ↳ sell)
133         claim_values += self.claim_side *
    ↳ claim_payoff

```

```

130         return claim_values
131
132     def get_terminal_price_grid(self):
133         """
134         Simulates SPX and VIX at the option maturity
135         ↳ off one set of QRH paths
136         """
137         qrh = self.qrh
138         days = (pd.Timestamp(self.expiry) - qrh.
139         ↳ quote_date).days
140         n_steps = days * self.steps_per_day
141
142         config = ObjConfig(
143             spx=qrh.spx_smile, spx_step=n_steps,
144             vix=qrh.vix_smiles[:1], vix_steps=[n_steps
145             ↳ ],
146             vix_labels=[self.expiry], xi0_fn=qrh.
147             ↳ fwd_var_fn,
148             n_steps=n_steps, delta=(days / 365.0) /
149             ↳ n_steps,
150             n_paths=self.no_scenarios, seed=self.seed)
151
152         spx_samples, vix_samples, _ =
153         ↳ simulate_spx_and_vix(qrh.calibrated.as_array(),
154         ↳ config)
155         self.spx_forward = self.get_spx_forward()
156         return self.spx_forward * spx_samples,
157         ↳ vix_samples[0]
158
159     def get_spx_forward(self):
160         spx = self.lob_data.loc[self.lob_data.sym == '
161         ↳ SPX', 'forward']
162         return float(spx.iloc[0]) if len(spx) else 1.0
163
164     def get_terminal_weights(self):
165         return np.full(self.no_scenarios, 1.0 / self.
166         ↳ no_scenarios)
167
168     def solve(self, isverbose=False, iswarm=False):
169         for solver in [self.solver, cp.CLARABEL, cp.SCS
170         ↳ ]:
171             try:
172                 self.PROBLEM.solve(verbose=isverbose,
173                 ↳ solver=solver, warm_start=iswarm)
174                 if self.x.value is not None:
175                     return
176             except Exception:
177                 continue
178             raise RuntimeError("no solver converged")
179
180     def score(self):
181         return self.PROBLEM.value
182
183     def get_terminal_wealth(self, without_cash = False)
184
185     ↳ :
186     if without_cash:
187         return self.P @ self.x.value
188     if self.x.value is not None:
189         return (self.P @ self.x.value) + (self.w -
190         ↳ self.Sx.value)
191     else:
192         raise print("Error: No Terminal Wealth")
193         return None
194
195     def get_vix_prices(self):
196         return self.vix_prices
197
198     def get_spx_prices(self):
199         return self.spx_prices
200
201     def get_positions(self):
202         return self.x.value
203
204     def process_snapshot_data(file_path, expiry="2024-07-17
205     ↳ ", symbols=("SPX", "VIX")):
206         df = pd.read_csv(file_path)
207         df["mid"] = mid(df)
208         df["sym"] = df.underlying_symbol.str[1:]
209
210         frames = []
211         for sym in symbols:
212             chain = df[(df.sym == sym) & (df.expiration ==
213             ↳ expiry)].copy()
214             chain["forward"], _ = get_fwd_and_discount(
215             ↳ chain)
216
217             can_buy = (chain.ask_1545 > 0) & (chain.
218             ↳ ask_size_1545 > 0)
219             can_sell = (chain.bid_1545 > 0) & (chain.
220             ↳ bid_size_1545 > 0)
221             chain = chain[can_buy | can_sell]
222             chain["rel_spread"] = (chain.ask_1545 - chain.
223             ↳ bid_1545) / chain["mid"]
224             chain = chain.sort_values("rel_spread").
225             ↳ drop_duplicates(["option_type", "strike"])
226             frames.append(chain)
227
228         lob_data = pd.concat(frames).rename(columns={
229             ↳ "bid_size_1545": "bsize", "ask_size_1545": "
230             ↳ asize",
231             ↳ "bid_1545": "bid", "ask_1545": "ask"})
232         lob_data = lob_data[["expiration", "sym", "
233             ↳ option_type", "strike",
234             ↳ "bsize", "bid", "ask", "asize"
235             ↳ ], "forward"]]
236         return lob_data.sort_values(["sym", "strike"]).
237         ↳ reset_index(drop=True)

```

Bibliography

- [1] Philippe Artzner, Freddy Delbaen, Jean-Marc Eber, and David Heath. Coherent measures of risk. *Mathematical Finance*, 9(3):203–228, 1999.
- [2] M. Avellaneda and A. Papanicolaou. Statistics of vix futures and applications to trading volatility exchange-traded products. *International Journal of Theoretical and Applied Finance*, 22(01):1850061, 2019.
- [3] T. Josef B. Hans, G. Lukas and W. Ben. Deep hedging. *Quantitative Finance* 19(8), pages 1,271–1,291, 2019.
- [4] Kerry Back. *Asset Pricing and Portfolio Choice Theory*. Oxford University Press, 2010.
- [5] Dirk Becherer. Rational hedging and valuation of integrated risks under constant absolute risk aversion. *Insurance: Mathematics and Economics*, 33(1):1–28, 2003.
- [6] Mikkel Bennedsen, Asger Lunde, and Mikko S. Pakkanen. Hybrid scheme for Brownian semistationary processes. *Finance and Stochastics*, 21(4):931–965, 2017.
- [7] Lorenzo Bergomi. *Stochastic Volatility Modeling*. Chapman & Hall / CRC, 2016.
- [8] Pierre Blanc, Jonathan Donier, and Jean-Philippe Bouchaud. Quadratic Hawkes processes for financial prices. *Quantitative Finance*, 17(2):171–188, 2017.
- [9] Florian Bourgey and Jim Gatheral. Quadratic rough Heston: SPX, VIX, and the SSR. Working paper, SSRN, 2026. SSRN Abstract ID 5239929. Code available at <https://github.com/jgatheral/QuadraticRoughHeston>.

- [10] Stephen Boyd and Lieven Vandenbergh. *Convex Optimization*. Cambridge University Press, Cambridge, 2004. Available online.
- [11] Douglas T. Breeden and Robert H. Litzenberger. Prices of state-contingent claims implicit in option prices. *The Journal of Business*, 51(4):621–651, 1978.
- [12] R. Carmona. *Indifference Pricing : Theory and Applications*. Princeton University Press, 2009.
- [13] Peter Carr and Dilip Madan. Towards a theory of volatility trading. In Robert A. Jarrow, editor, *Volatility: New Estimation Techniques for Pricing Derivatives*, pages 417–427. Risk Books, London, 1998.
- [14] Cboe Exchange, Inc. Cboe volatility index (vix) white paper. Technical report, Cboe Exchange, Inc., 2019. Accessed 2025-08-23.
- [15] Cboe Global Indices, LLC. Cboe® volatility index (vix®) methodology. Technical report, Cboe Global Indices, LLC, Chicago, IL, July 2019. Major version 1; Last revised: 2019-07-26.
- [16] A. Chen, T. Nguyen, and N. Sorensen. Indifference pricing under sahara utility. *Journal of Computational and Applied Mathematics*, 2020.
- [17] Robert C. Dalang, Andrew Morton, and Walter Willinger. Equivalent martingale measures and no-arbitrage in stochastic securities market models. *Stochastics and Stochastic Reports*, 29(2):185–201, 1990.
- [18] Aditi Dandapani, Paul Jusselin, and Mathieu Rosenbaum. From quadratic Hawkes processes to super-Heston rough volatility models with Zumbach effect. *Quantitative Finance*, 21(8):1235–1247, 2021.
- [19] M. H. A. Davis, V. G. Panas, and T. Zariphopoulou. European option pricing with transaction costs. *SIAM Journal on Control and Optimization*, 31(2):470–493, 1993.
- [20] Freddy Delbaen, Peter Grandits, Thorsten Rheinländer, Dominick Samperi, Martin Schweizer, and Christophe Stricker. Exponential hedging and entropic penalties. *Mathematical Finance*, 12(2):99–123, 2002.
- [21] Freddy Delbaen and Walter Schachermayer. A general version of the fundamental theorem of asset pricing. *Mathematische Annalen*, 300(1):463–520, 1994.

- [22] Jérôme Detemple and Carlton Osakwe. The valuation of volatility options. *Review of Finance*, 4(1):21–50, 04 2000.
- [23] Nicole El Karoui and Marie-Claire Quenez. Dynamic programming and pricing of contingent claims in an incomplete market. *SIAM Journal on Control and Optimization*, 33(1):29–66, 1995.
- [24] Hans Föllmer and Alexander Schied. Convex measures of risk and trading constraints. *Finance and Stochastics*, 6(4):429–447, 2002.
- [25] Hans Föllmer and Alexander Schied. *Stochastic Finance: An Introduction in Discrete Time*. Walter de Gruyter, Berlin, 3rd edition, 2011.
- [26] Marco Frittelli and Emanuela Rosazza Gianin. Putting order in risk measures. *Journal of Banking & Finance*, 26(7):1473–1486, 2002.
- [27] Jim Gatheral. Efficient simulation of affine forward variance models. Working paper, SSRN, 2022. SSRN Abstract ID 4018422.
- [28] Jim Gatheral and Florian Bourgey. QuadraticRoughHeston: reference implementation <https://github.com/jgatheral/QuadraticRoughHeston>. GitHub repository, 2026. Companion code to [9].
- [29] Jim Gatheral, Paul Jusselin, and Mathieu Rosenbaum. The quadratic rough Heston model and the joint S&P 500/VIX smile calibration problem. *Risk*, May 2020. arXiv:2001.01789.
- [30] Jim Gatheral and Martin Keller-Ressel. Affine forward variance models. *Finance and Stochastics*, 23(3):501–533, 2019.
- [31] Paul Glasserman. *Monte Carlo Methods in Financial Engineering*. Springer, New York, 2004.
- [32] Andreas Grünbichler and Francis A. Longstaff. Valuing futures and options on volatility. *Journal of Banking & Finance*, 20(6):985–1001, July 1996.
- [33] O. Gueant. *The Financial Mathematics of Market Liquidity*. Chapman & Hall / CRC, 2016.
- [34] Julien Guyon. The joint S&P 500/VIX smile calibration puzzle solved. *Risk*, April 2020. SSRN Abstract ID 3568480.

- [35] J. Michael Harrison and David M. Kreps. Martingales and arbitrage in multiperiod securities markets. *Journal of Economic Theory*, 20(3):381–408, 1979.
- [36] J. Michael Harrison and Stanley R. Pliska. Martingales and stochastic integrals in the theory of continuous trading. *Stochastic Processes and their Applications*, 11(3):215–260, 1981.
- [37] V. Henderson. Valuation of claims on nontraded assets using utility maximization. *Mathematical Finance, Vol. 12, No.4*:351–373, 2002.
- [38] V. Henderson and D. G. Hobson. Real options with constant relative risk aversion. *Journal of Economic Dynamics & Control* 27 (2002) 329–355, 2002.
- [39] Steven L. Heston. A closed-form solution for options with stochastic volatility with applications to bond and currency options. *The Review of Financial Studies*, 6(2):327–343, 1993.
- [40] S. D. Hodges. Optimal replication of contingent claims under transaction costs. *The Review of Futures Markets Vol. 8*, 1989.
- [41] T. Pennanen J. Armstrong and U. Rakwongwan. Pricing index options by static hedging under finite liquidity. *International Journal of Theoretical and Applied Finance*, 2018.
- [42] Dmitry Kramkov and Walter Schachermayer. The asymptotic elasticity of utility functions and optimal investment in incomplete markets. *The Annals of Applied Probability*, 9(3):904–950, 1999.
- [43] Dmitry Kramkov and Walter Schachermayer. Necessary and sufficient conditions in the problem of optimal investment in incomplete markets. *The Annals of Applied Probability*, 13(4):1504–1516, 2003.
- [44] Robert C. Merton. Lifetime portfolio selection under uncertainty: The continuous-time case. *The Review of Economics and Statistics*, 51(3):247–257, 1969.
- [45] MOSEK ApS. *MOSEK Portfolio Optimization Cookbook*, release 1.6.0 edition, August 2025. Accessed 27 Aug 2025.
- [46] Michael K. Osinowo. Hedging & indifference pricing of terminal claims on the VIX. MSc project submitted for 7CCMFM50, Department of Mathematics, King’s College London. Examined September 2025; superseded by the present report, 2025.

- [47] G. A. Paleologo. *The Elements of Quantitative Investing*. Wiley Finance, 2025.
- [48] T. Pennanen. Optimal investment and contingent claim valuation in illiquid markets. *Finance Stoch.*, 18(4):733–754, 2014.
- [49] T. Pennanen and M. Koivu. Epi-convergent discretizations of stochastic programs via integration quadratures. *Numerische Mathematik*, 100(1):141–163, 2005.
- [50] T. Pennanen and U. Rakwongwan. Optimal semi-static hedging in illiquid markets. <https://arxiv.org/pdf/2008.01463>, 2020.
- [51] Teemu Pennanen and Ari-Pekka Perkkiö. Convex duality in optimal investment and contingent claim valuation in illiquid markets. *Finance and Stochastics*, 22(4):733–771, 2018.
- [52] Teemu Pennanen and Spyridon Pougkakiotis. Fm50: Pricing and hedging volatility index options — project description. Project brief for the MSc in Financial Mathematics, 2024. Static portfolio optimization and indifference pricing via quadrature and convex optimization.
- [53] John W. Pratt. Risk aversion in the small and in the large. *Econometrica*, 32(1/2):122–136, 1964.
- [54] Richard Rouge and Nicole El Karoui. Pricing via utility maximization and entropy. *Mathematical Finance*, 10(2):259–276, 2000.
- [55] A. Roux and Z. Xu. Optimal investment and contingent claim valuation with exponential disutility under proportional transaction costs. *International Journal of Theoretical and Applied Finance Vol. 25, No. 04n05*, 2022.
- [56] Birgit Rudloff and Firdevs Ulus. Certainty equivalent and utility indifference pricing for incomplete preferences via convex vector optimization. *Mathematics and Financial Economics*, 15(2):397–430, 2021.
- [57] Alexander Shapiro, Darinka Dentcheva, and Andrzej Ruszczyński. *Lectures on Stochastic Programming: Modeling and Theory*. SIAM, Philadelphia, 2009.
- [58] P. Yamphram, P. Sutthimat, and U. Rakwongwan. Pricing and hedging index options under mean-variance criteria in incomplete markets. *Computation*, 11(2), 2023.

- [59] Yutong Yan, Wei Zhang, Yahua Yin, and Weidong Huo. An ornstein–uhlenbeck model with the stochastic volatility process and tempered stable process for vix option pricing. *Mathematical Problems in Engineering*, 2022(1):4018292, 2022.
- [60] Ya-Hua Yin, Fu-min Zhu, and Zun-Xin Zheng. Pricing vix options based on mean-reverting models driven by information. *The North American Journal of Economics and Finance*, 74:102203, 2024.